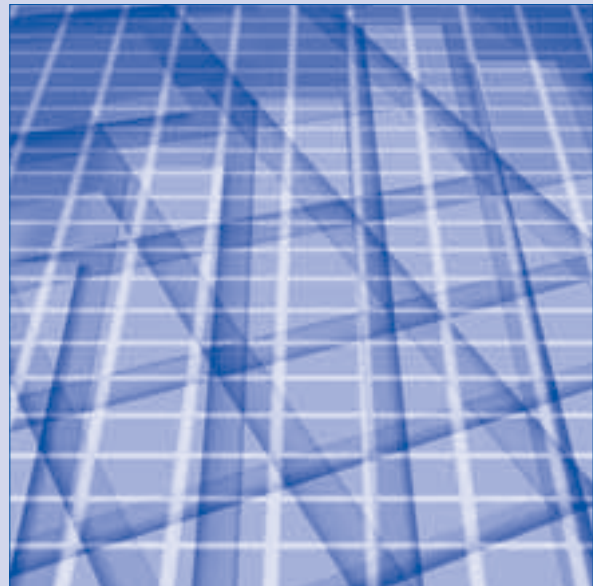




M359 Block 1

UNDERGRADUATE COMPUTING

Relational databases: theory and practice



Databases in context

Block
1

This publication forms part of an Open University course M359 *Relational databases: theory and practice*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)870 333 4340, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure: tel. +44 (0)1908 858785; fax +44 (0)1908 858787; email ouwenvq@open.ac.uk

Sybase, iAnywhere and SQL Anywhere are trademarks of Sybase, Inc.; Java is a trademark of Sun Microsystems, Inc. Other product and company names may appear in the M359 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall, Milton Keynes
MK7 6AA

First published 2006. Second edition 2008

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>.

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by S&P Enterprises (rfod) Limited, Glos.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 2945 0

CONTENTS

1	The evolution of data management	6
1.1	Introduction	6
1.2	Historical data management	6
1.3	Modern data processing	13
1.4	Some basic definitions and descriptions	18
1.5	Databases and DBMSs	22
1.6	Are all database systems the same?	27
1.7	Specialist data processing systems	35
1.8	Glimpsing the future of data management	39
1.9	Enduring issues in data management	41
1.10	Summary	43
2	Data management – why it matters	44
2.1	Introduction	44
2.2	Data as an asset	46
2.3	Data quality	49
2.4	Securing data assets	50
2.5	Legal and ethical issues in data management	51
2.6	What can happen when it goes wrong?	54
2.7	Corporate data management strategy	64
2.8	Summary	66
3	Database management system concepts	68
3.1	Introduction	68
3.2	The three-schema architecture	69
3.3	The functions of a DBMS	72
3.4	Inside the DBMS	77
3.5	Information system architectures	81
3.6	People and databases	90
3.7	User interaction and database access	93
3.8	Summary	100
4	Developing database systems	101
4.1	Introduction	101
4.2	A development life cycle	103
4.3	Requirements gathering	106
4.4	Analysis	107

4.5	Design	108
4.6	Implementation	110
4.7	Testing	113
4.8	Maintenance	114
4.9	Summary	114
5	Conceptual data models	116
5.1	Introduction	116
5.2	The domain of discourse and our system	117
5.3	Using conceptual data models	119
5.4	Understanding conceptual data models	120
5.5	The University conceptual data model	120
5.6	The Hospital conceptual data model	137
5.7	Summary	141
Block summary		142
Solutions to Exercises		143
Index		156

M359 COURSE TEAM

This course was produced by the following team (affiliated to The Open University, unless otherwise stated):

Course team

Kevin Waugh Course Team Chair and Author
Ian Cooke Author
Mike Newton Author
Judith Segal Author
Steven Self Author
Alistair Willis Author
Kay Bromley Academic Editor
Ralph Greenwell Course Manager and Accessibility Consultant

External assessor

Barry Lowden University of Essex

Critical readers

Sue Barrass
Peter Blachford
Terry Burbidge
Pauline Butcher
Pauline Curtis
Hugh Darwen
Ivan Dunn
Gillian Mills
Ron Rogerson

LTS Media team

Andrew Seddon Media Project Manager
Steve Rycroft Editor
Andrew Whitehead Designer and Graphic Artist
Phillip Howe Compositor
Kamy Yazdanjoo Software Developer
Sue Stavert Technical Testing Team

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics and Computing.

1

The evolution of data management

1.1 Introduction

We are living in the most documented, recorded, and analysed period in our history. Purchases that we make can be recorded by our credit card company; a simple walk down our local high street may be monitored by a CCTV camera; travel by car, train or aeroplane leaves behind a virtual trail of information. There are numerous actions in our daily lives that will produce a data record that is of value to somebody. Nearly all of the data that our activities generate will be recorded in a database as a structured data record – perhaps only for a very brief period, but often such data is stored for an indefinite period. Some of that data collection may never ever be accessed, some might be searched to answer a specific question, some will be summarised and presented in a report, but all of it will be stored until it is deleted or accidentally destroyed.

Much of the wealth of the world economy is tied into the creation, storage, management and distribution of information. While it is not the purpose of this course to look at the information systems or the economics of data management, it is important to identify that the knowledge economy is a large part of modern society. Consequently, society now places great dependence on good data management practices. In this course we will consider how to ensure that data is held and managed in effective, safe database systems, while making sure that the data can be used to maximum effect by the applications that require it, and kept secure from those not authorised to access it.

Throughout the course we will be looking at some of the underpinning technology in the information management environment. Specifically, we will consider the security, storage, management of and access to data of interest to an organisation. We will not be involved in considering how that data is to be used by the organisation; but we will ensure that we can maximise the uses to which that data can be put by ensuring the data is accessible to those who require it. We won't be involved in the analysis that decides what data is required by an organisation, but instead we will focus on deciding how best to represent that data within a relational database system and how to ensure that only appropriate changes are made to the data.

In a later section of this block we will look at how an organisation makes use of the data it collects. We will consider that the data both incurs cost and adds value to that organisation, and how it is important that the organisation can maximise the value while minimising the cost. This example will highlight the importance of good database development by offering a look at some of the problems that can arise if a database is not adequately designed and implemented. We will use this example to identify some of the key issues that we will pursue in the theory and practice of relational databases, which are the topics of *Blocks 2, 3 and 4* of the course material.

1.2 Historical data management

Recording data is not a recent, 'computer generation' phenomenon – there are examples of human record keeping systems going back to ancient times. We can look back at some examples of historical record keeping systems to identify key points about data and data processing systems that are fundamental to the development of modern views on data

Generally we use the term technology to encompass both hardware and software.

and information management. In the pages that follow, the historical detail is not the important focus; what is important is that we have become more sophisticated in the requirements we place on our record keeping systems. However, despite this increased sophistication, we will see that some fundamental requirements remain unchanged.

Pre-paper records

Archaeologists have identified record keeping artefacts that have been dated as far back as the rise of Cro-Magnon man, around 30,000 years ago. An animal jaw-bone with fifty-five notches marked in groups of five was discovered in the 1930s in what was then known as Czechoslovakia and dated at over 20,000 years old. Similarly, in Lebanon a bone piece about 10 cm in length covered in four columns of markings, with 170 incisions in total, was found to date from around 12,000 BC. These are undoubtedly primitive records, probably a simple counting record, and they have one desirable quality important in any record keeping system – the records have proved to be durable. However, the notched bones lack another important property of record keeping systems. No one today really knows what they represented – the number of days since last rainfall, meals eaten, number of tribe members, goats owned, calendars and many other interpretations are possible.



Administrative tablet of clay, Mesopotamian/Sumerian, 3100–2900 BC. The earliest tablets with written inscriptions represent the work of administrators, perhaps of large temple institutions, recording the allocation of rations or the movement and storage of goods. © TopFoto.co.uk

Things had improved in this respect by the time of the Sumerian civilisation, around 3000 BC. From this time period the remains of clay tablets have been found which use direct pictographic representations to record what was being counted; 25 pictograms of a sheep representing 25 sheep, for example. So we have record keeping in which both the things being counted and the actual count of these things is being recorded. A second key property of modern record keeping systems is evident here – there needs to be agreement as to what the record represents. It is worth noting that a record of 25 sheep or 34 oil jars doesn't tell us anything about what that record was used for. It could represent details of debt collection, taxes, stock taking, shopping lists and so on. However, as a durable and understandable piece of data it stands independently of the use to which it was put. Our ability to interpret the application of historical data records improves as we move to more modern day artefacts.

For the final property of pre-written record keeping we can look at a much more recent example: the 13th century and the task of keeping records of taxes and payments between English noblemen and the King. The tally stick became the main record of tax assessment and payments for the Treasury. A tally stick is a narrow length of wood, on



Tally sticks, circa 1440 – these were a method of accounting for loans and payments used by the Treasury. Each stick contains notches to show the amount deposited, the name of the lender, the county, the time of year when the money can be repaid, and the year. Tallies were used by the exchequer until 1826.

© Science Museum/Science and Society Picture Library

which a series of notches are carved to represent a monetary amount. The Crown Exchequer gave a detailed description of the marks and distances between marks and the amounts they represented so that they represented a common notation. The stick is then split vertically into two pieces so that each piece has the same notches in alignment. One piece of stick would be held by each of the parties involved in the transaction. For example, the King's tax assessor would visit a nobleman and make an assessment of taxes to be paid to the King. A pair of tally sticks would record this assessment. One of the two tally sticks was given to the nobleman as a record of his debt; the other would be held by the tax assessor as an effective counterfoil for the tax assessment. When the time came to pay taxes the nobleman would take his tally sticks and cash to the King's representative (usually at Easter and Michaelmas) where 'tallying-up' would take place. If there were any disputes regarding the amounts owed, the two halves of the tally stick could be compared for evidence of tampering. The tally stick has the desirable property that the data recorded is trustworthy. It cannot be changed by one of the stick holders as the other stick would not 'tally' when aligned for comparison, so we have the basis for a simple check on the validity of the record being held.

The tally became the first example of the government bond, allowing the King to raise immediate finances based on the expectation of receiving payment at a later date. Since the tax assessment and the tally that recorded it was a guarantee that at tallying-up time the King would have a certain amount of cash, it was possible for the government to sell the tally to a buyer at a discount. Later, after the taxes had been paid, the holder of the

tally could go back to the government to redeem the tally at full value – a fixed-term, fixed interest, government bond. This is an example of another enduring data management property – the data recorded could be used for more than one purpose, and in its tally stick form it was indeed accessible for different purposes.

As a record keeping system, and the basis for an information system recording financial transactions, the tally system was very successful. The sticks were durable, small and easily transported and stored. The meaning of the notches was clear, or could easily be interpreted by reference to an agreed definition. Tallying-up was a simple process that ensured that the records could be trusted and it could be applied even by those who couldn't read or write.

Paper-based record keeping and data processing

Early paper records would often lack structure; sometime early records of bills or stock records are just one long string of textual information running throughout a page, without any structural breaks such as paragraphs or columns. To locate specific details in an early land register, for example, would involve a linear search of the document until the required information was found.



Autograph memorandum of payments and receipts by Michelangelo. Reproduction is prohibited. By permission of the British Library

However, as paper records became more common and their use extended, the mechanisms for presenting information improved with the use of lists, structured records, tables and so on, making it easier to interpret and manipulate. Of course, like the notches on the tally stick, it is necessary to ensure that a structured written record has an understandable and clear interpretation in order to be useful. With a move to paper record keeping, and the general increase in literacy of the population, it becomes possible for large and varied amounts of data to be recorded and managed.



Parish register showing structured entries for burials, 1539–1541, from the register of St Peter's church in Dunwich, Suffolk.
Heritage Image Partnership. The British Library

Manual data processing

As the number of literate and numerate people generating paper-based records grew, the problem of how to make use of this data – how to process the paper records – needed to be addressed. We begin to see the emergence of data clerks, filing systems, reports and manual record keeping systems around the 1700s. These depended on the abilities of people trained to manage paper records to perform specific, often repetitive tasks – copy clerks who manually transferred records from one ledger to another, accounting clerks who maintained double-entry bookkeeping records, filing clerks who stored and retrieved ledgers, papers and so on, or who sorted using pigeon-hole desks.



Early Post Office filing clerks, stamping and sorting letters, 1875. Plate taken from the *Illustrated London News* (Vol. 75/1, p.160). Each letter that passed through the postal service had to be stamped by hand, so rows of clerical workers were needed to process the growing volume of correspondence.

© Science Museum/Science and Society Picture Library

Automated data processing

Even with large numbers of people manually moving, sorting, summarising, filing and retrieving paper records, the needs of centralised governments soon overwhelmed the ability to process its data. Automated mechanical systems were required to meet large processing requirements.

The US government estimated it would take more than 10 years to manually process the data that would be collected in the 1890 census – faster processing was required. In 1881 Herman Hollerith started work on a machine to tabulate census data. Hollerith was successful in using electrical machines to count and sort punched cards – cards with holes in them. The 1890 census data was recorded on punched cards and processed



Early Hollerith punched card. Image shows a woman processing information gathered by the United States Census Bureau with a card puncher machine. This type of machine was used by the Bureau between 1890 and 1950.

© Science Photo Library

using the Hollerith tabulation system with the results published in 1892, only two years later. By comparison, the 1880 census had taken six years to process manually. Another quality of data management is becoming evident; despite the large volumes of data being recorded and managed, it is essential that it can be processed within a useful timescale.

Punched cards and mechanical and electrical tabulation systems utilising structured data records were used for many data processing purposes until the development of the electronic computer in the 1940s. Punched cards continued as a data entry and data storage medium until the late 1980s when fast and reliable magnetic media all but replaced them.

Early computerised records

The first use of digital computers, around 1940, was purely to perform calculations. The early stored-memory electronic computers used their memory to store programs and initial and intermediate values used in calculations. It was only later, as the amount of available electronic memory increased and with the development of persistent storage such as magnetic drum and disk memory, as well as the use of data entry peripherals such as punched tape and punched cards, that computers were considered as a data storage and data processing environment.

The punched-card systems and the electronic and magnetic memory of the early computers represented information in fundamentally the same way. A hole representing a yes/no answer to the question 'Does this person have two children?' can be encoded on a punched card. Electronically, the same yes or no answer can be recorded by a positive or negative voltage; magnetically, the field polarity is recorded on a magnetic material and related to a yes or no answer. So punched-card processing performed by Hollerith's tabulation system could be recreated in the electronic computers without the need for much innovation in the computing technology. However, computer technologies soon switched to using encoded information, coded representations for numbers and alphabetic characters, and the binary encoding of information rapidly replaced the yes/no representation used on the punched cards. The data still required a known structure and an agreed interpretation but a data card could now carry a string of characters encoding data; for example, '3 A 37345 30' might represent 'three children, homeowner, income of £37,345 per year, 30 years old'. These strings of characters were held in a single collection, known as a data file, that might contain thousands of such data strings and that could be written to disk or tape memory.

The use of general-purpose encoding to represent information within a computer effectively brings us up to the modern era of data processing. If a general-purpose computer can process encoded textual data, then to process interesting multimedia data formats such as web content, audio, video and so on, only requires the identification and use of suitable encoding mechanisms to produce data representations that can be stored in computer memory and the development of programs to manipulate that stored data.



EXERCISE 1.1

The above historical accounts of data management have raised several important issues in the handling and processing of data – state three of these issues.

1.3 Modern data processing

The modern era of data processing started with the direct computerisation of the punched-card style records used by tabulation machines; later, however, the limitations of such approaches were recognised. The database approach, which separates the physical representation of data from the mechanisms used to access the data, soon replaced file processing in electronic computer systems. We describe these approaches in the next few pages.

The file-based approach

The development of magnetic data storage on tape, disk or drum and the development of computer operating systems and file management systems led to the growth in **file-based storage** and manipulation of structured data. The early use of files continued to mimic the kind of processing that was performed with punched cards. Each different application required a different stack of punched cards to represent the input data. Each of these cards would have a specific layout of data encoded. This made it difficult to share or reuse the punched-card data for different purposes without re-punching a new stack of cards.

The early file-based systems, made up of data files and associated programs to process them, would have a file of data and one or more programs for each application. Each data file contained data records specific to that one application. Some applications might use more than one data file, and some applications could access the data that was also used by another application. Clearly there are benefits to be gained from having a sharable file that could be used by several applications; the main advantage is that data entry is a costly operation and sharing avoids the need for re-entering data. However, each application needs to know about the format and access mechanisms for each file – these details are known as the **interface** to the file. The interface to the file is a machine readable collection of format and access descriptions which could be coded in an application program or stored in a data file and accessed by an application program.

Consider the example shown in Figure 1.1, which illustrates a single file of student data and a single application process, a mailing list generator. The student file could be structured to have a record for each student, with each record having 90 characters encoded: the first 10 characters of the record representing the student's identifier, the next 30 characters representing the student's name, the next 40 the student's address and the final 10 characters their postcode.

We will look at a larger example of a university database later in the course, but for now we will consider some simple examples based around files of student data.

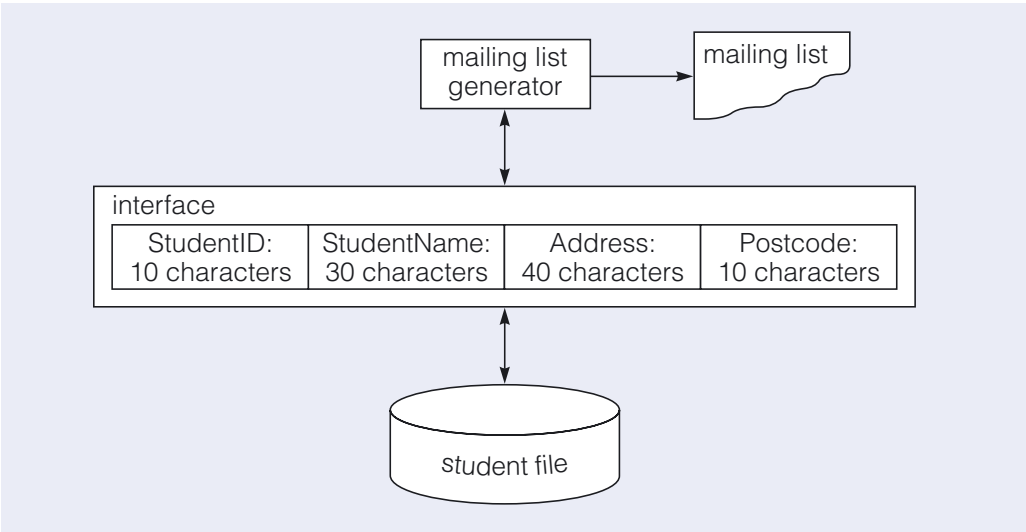


Figure 1.1 An application using an explicit interface to the student file

If we have just one application using this file, a mailing list generator in this case, then we can ensure that the file interface and the application are consistent if the file structure needs to change. Suppose that at some time in the future we decide that it is important to be able to select students based on the date of their first enrolment and to generate mailing lists based on this selection. The existing student data file does not have this data, so we must add it. First however, we must change the data file's structure and consequently we need to update the interface used by the mailing list generator to allow it to make use of the newly structured data. Of course, the mailing list generator would have to be reprogrammed to process this new date field, as seen in Figure 1.2, which is necessary to meet the new requirements for using the enrolment date.

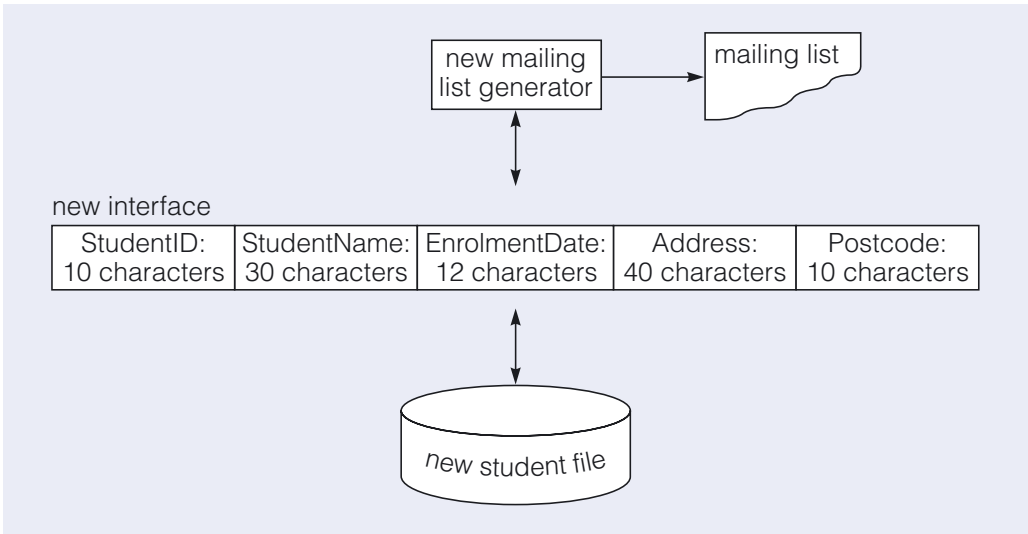


Figure 1.2 The application from Figure 1.1 using the new interface including the date of enrolment

To some extent this change to the mailing list generator program is necessary maintenance work (sometimes called **productive maintenance**, maintenance that leads to a benefit by meeting a new requirement, not just a continuation of the previous state). However, consider a more complex (and realistic example) where that same student file is also used by a second application, say a TMA results recording program. This program requires an interface to the student data file to process the data it contains. Let's assume that it won't make use of the new student enrolment data. The TMA recording program does not require any update as it has no new requirements to meet. Unfortunately, to meet the new mailing list generator requirements we have updated the data file's record structure, so we will need to change the shared interface and consequently we will need to update the TMA program as shown in Figure 1.3. This forced update to the TMA program is an example of **unproductive maintenance**.

With unproductive maintenance we are making changes to the program not because of any new requirements for that program but in order to continue using a shared data file that has been changed for some other reason. If we have more files and more applications sharing those files the unproductive maintenance becomes a potentially enormous overhead.

Unproductive maintenance is a consequence of the way in which file-based processing is closely tied to the physical structure of the data stored in the files. This is referred to as **data dependence**. A more detailed example can illustrate this. When a program accesses the student record shown in Figure 1.1, then 90 characters must be read for each full record. In those 90 characters the first 10 characters are the physical location for the student's identifier, and so on for the other data items. The 90-character record size and the layout of data items within those 90 characters forms the interface description of the file. If we added a two-character field at the very start of the record, say to represent a registration region number for that student, then we would need to

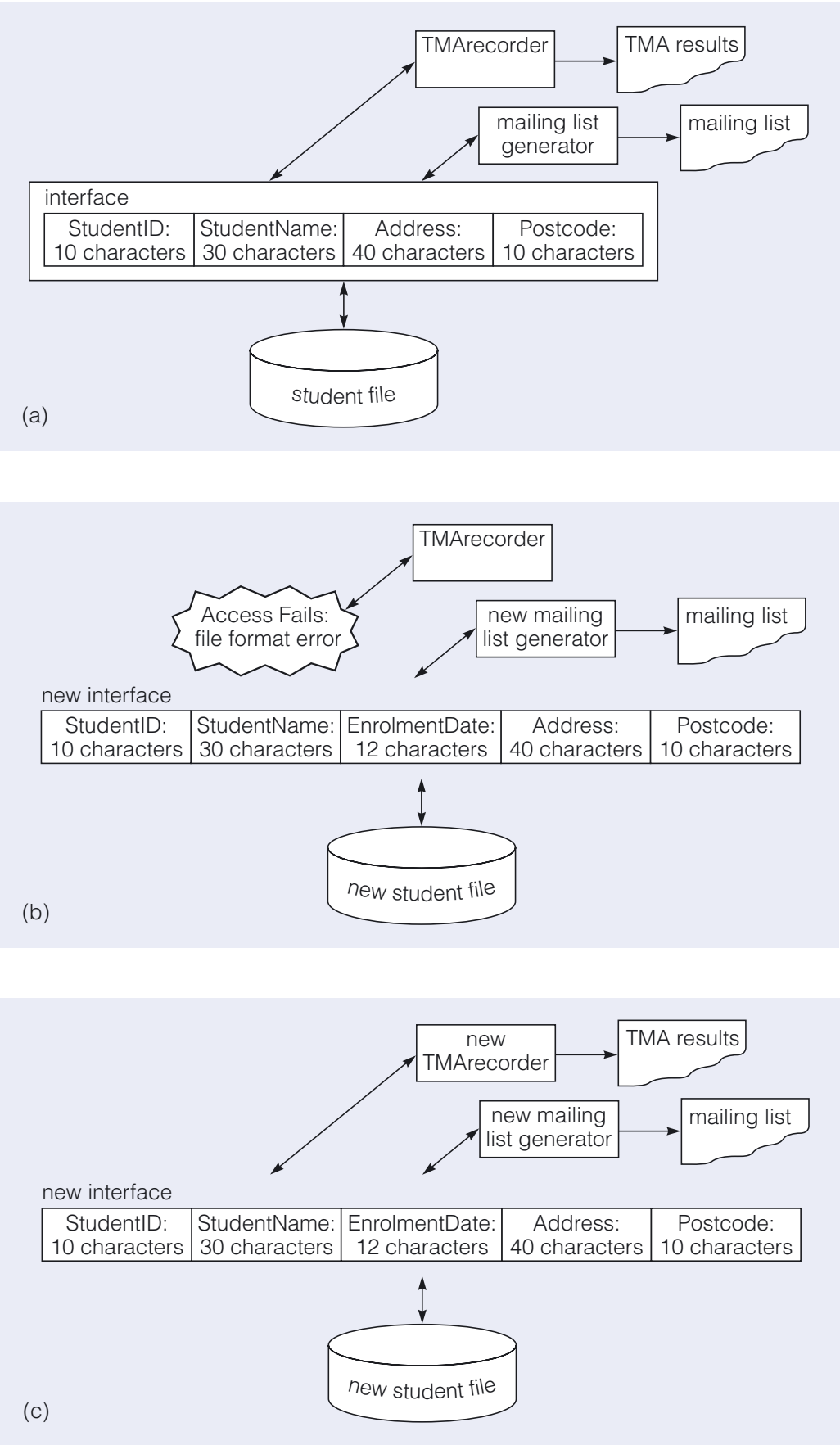


Figure 1.3 (a) Two applications share an interface to the student file. (b) One application requires the original interface to be changed to meet new requirements; this change 'breaks' the second application's use of the interface. (c) The second application must be updated in order to use the new interface

make each student record 92 characters long, and the student identifier would now be located in characters 3 to 12; so the interface would need to change accordingly. As a consequence, every program using the interface would need to be changed to reflect this physical layout change even if that program did not need to process the region number.

To avoid this unproductive maintenance the data could be duplicated in multiple files. For example, it would be possible to leave the student file shown in Figure 1.1 unchanged for use by the TMA recording application and create and update a copy of this file for use by the mailing list generator. There isn't then a need for unproductive maintenance beyond copying and renaming the data file.



EXERCISE 1.2

What is the weakness in having the same data about students, their identifiers, names, addresses and postcodes, stored in two separate files and processed by different applications?

Using multiple data copies of the file can still result in a form of unproductive maintenance. When the student data file used by the mailing list generator is updated, say a student moves house, then to maintain the consistency of the data it is necessary to also update the data file used by the TMA recording application to reflect this. Every copy of the data file needs to be updated each time a change is made to the data by any of the applications programs. With just a single copy of the data, in one file shared between applications, this is avoided.

When it was introduced, this new file-based method of data processing offered greater performance and increased flexibility compared to the previous manual and punched-card systems, and consequently the demands of users grew. This places a burden on the application programmers. In the file-based approach, every new requirement for a different query or report will involve a new or amended application program, and possibly a new or amended data file and consequent interface maintenance. Such amendments can be very costly and resource intensive; without some restriction, the constant flow of new requests could overwhelm the data processing department. This limitation highlights further properties of modern data management systems – the flexibility required when amending the data structures to meet new requirements, and a need to easily satisfy data processing requests.

So the file-based approach suffers from unproductive maintenance as a result of the data dependencies of using a file's physical structure within an application. In addition there can be other problems that occur in file-based processing related to the duplication of data in multiple files for different purposes and the need to generate a new application for each ad hoc request that a user wishes answered.

The database approach

The limits of the file-based approach mean that the **database approach** to data processing has almost entirely replaced file-based data processing systems.

This approach hides the physical representation of the data from the users (or more properly the users' application programs). A software system known as a **database management system** (DBMS) is responsible for all interactions with the data files and all interactions with the applications that use the data – see Figure 1.4. Requests from the application programs are presented to the DBMS. The DBMS is then free to perform whatever processing is required to satisfy the request and access the

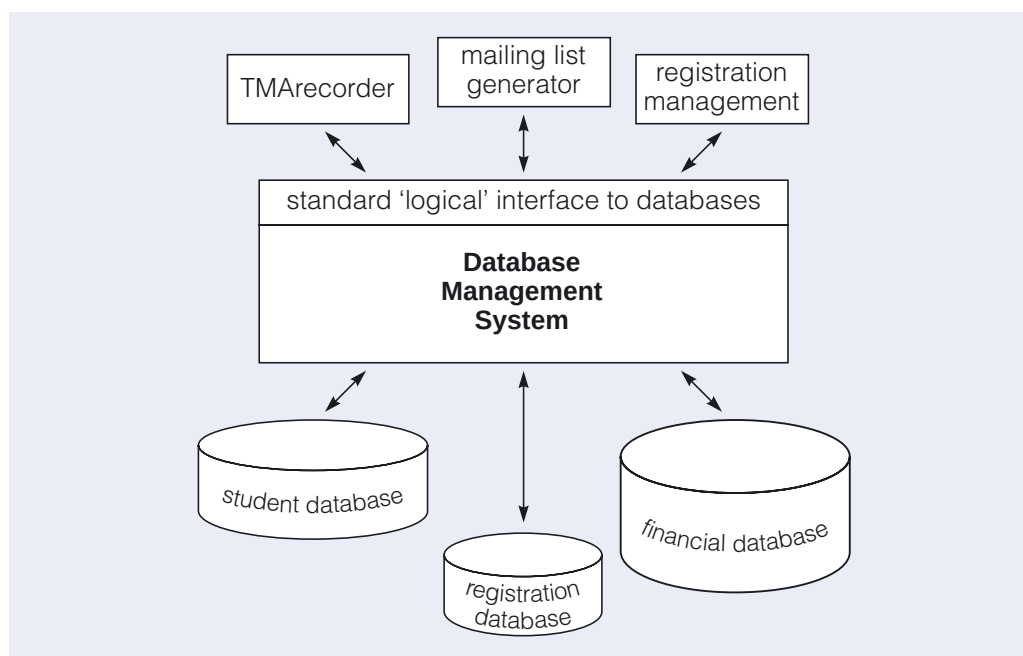


Figure 1.4 The DBMS approach to data management

necessary data files. The application program interacts only with the DBMS; each application therefore has one interface with the DBMS not one interface for each data file. Furthermore, the application program does not need to know how the data is stored in the files or how to manipulate the records in those files. The application program can use a **logical interface** rather than the **physical interface** of the file-based approach.

For example, an application might issue a request to `SELECT Postcode FROM Student`, in order to extract all the postcodes in the student data. The DBMS turns this logical request into the specific physical instructions required to find and return the postcode data. It can do this because it internally holds a **mapping** between the logical name, 'Postcode' (in the Student data), and the physical representation, characters 81–90 of each 90-character record in the student file. If there is a need to change the physical record structure (such as the earlier example of adding two characters for the region number) then the mapping can be changed so Postcode is now associated with characters 83–92 of the student file, but the logical description remains the same, 'Postcode', so the application does not have to be changed.

This separation of the physical storage of the data from the application processes that use it is referred to as **data independence** and is fundamental to the way in which we access and manipulate data in modern data processing. We will be revisiting this concept throughout the course.

The database approach to data storage and manipulation and the involvement of the DBMS is now central to the many and varied requirements that users have for data management. The DBMS hides most of the complex data requirements from the application programmer and other users while offering logical access to the content of collections of databases. The DBMS can be constructed and optimised to take account of the specialised tools and techniques required to support the properties and management options for the data, while ensuring a minimum of unproductive maintenance and maximum sharing of the stored data.

In the next couple of parts of this section we will look in detail at modern data management.



EXERCISE 1.3

- (a) Briefly describe the cause of the problem of unproductive maintenance in the file-based approach.
- (b) How does the database approach resolve this problem?

What history tell us about data management

We have given a very brief introduction to some of the principles of data storage and management systems that we will be considering later in the course. We have shown some of the historical development of data storage and management to highlight that these are enduring principles not solely related to modern computerised systems.

- ▶ The data records must be durable.
- ▶ The data records must be interpretable.
- ▶ The data records must be trustworthy and secure.
- ▶ Agreeing the structure and content of data collections can enable data sharing.
- ▶ Structured data can be processed automatically.
- ▶ Data structure representation must be flexible.
- ▶ Data access must be simple and logical, rather than tied to physical representations.
- ▶ Data collections can be used for many purposes; their processing must be timely to be useful.

These points will be revisited as we look in detail at relational database systems and the associated database management systems.

1.4 Some basic definitions and descriptions

Before we continue it's important that we consider some of the terminology that has been used so far in this block. In this early section of the course we have deliberately not been too specific in what we mean by data or information and we haven't tried to define the term database. However, as we progress with the course it will be increasingly important that we have agreed descriptions and definitions for these terms. Without some agreement we will end up in a very confused situation.

We will try to be consistent throughout the course in our use of the terminology – however, you might want to make use of a computing dictionary or dictionary of data processing and data management when working with other texts in this discipline.

Data vs Information

In some contexts data and information are considered synonymous terms. You can often find them appearing together in such sentences as 'Modern computers systems are concerned with the processing of data/information'. In fact in the **Information Systems** discipline there is a very strong distinction between data and information (and these are distinct from knowledge). Although we won't be working in the Information Systems discipline, it will help if we can distinguish where the data and information boundary lies, even if we can't pin it down exactly.

Data is a term given to the many recordable facts about the world(s) we inhabit. The world might be the physical world (a person's height, a mountain's height, the acceleration due to gravity at the Earth's surface, the wind speed on top of the Eiffel Tower at 1.30 pm on 3 May 2009), the financial world (a person's salary, the cost of an

item, the tax rate, the amount of the last credit card transaction you made), the art world (the name of the painter of a piece of artwork, the weight of a sculpture, the insurance value of a work, the date on which a painting was first shown in a gallery) and so on. Clearly there are an infinite number of possible facts we could record; very few of these would be of interest to us. We usually restrict the term data to be the collection of facts that are somehow of interest or relevant to us. Occasionally we will use data in a general sense, some collection of recordable facts, and at other times we will specifically mean the facts that we have decided are relevant to us for some specific purpose or purposes.

Note that we have not said anything about the accuracy of data, just about distinguishing those facts that are of interest to us. Consider that in the general pool of human-centred data we know that every person that has ever lived has a date of birth. For some people that date may not be known; but they still have one! For some organisation, say The Open University, the facts relevant to the organisation might be the date of birth of the employees of that organisation. But it may have a data item that says I was born on 11 May 2065. It's a recorded data item, it just happens to be inaccurate. We'll look at the need for accuracy and consistency in our stored data later in the course.

We've also not said anything about the representation of data – just that it is a collection of facts of interest. The representation of data can be chosen for specific purposes, or specific audiences of users. We can write down a calendar date in many different formats – 21/05/2008, 5/21/08, 21st May 2008, May 21st 2008, 21 May 2008, Twenty-first of May 2008, 15/5/1429 and Iyar, 16, 5768. However, they all represent the same actual day. We need something other than the data value and representation to be able to say what our data means.

Information is said to be meaningful data. Adding meaning is something that people do when they interpret data and use it to make decisions, judgements or inferences. Creating information is considered by many to be the process of putting data into a given context, relating it to other things and supplying a meaning to that data.

Consider that a clock may show both hands pointing towards the 12 – a piece of data. If we also know, or infer, that the clock is working and showing the correct time and is set to the local time zone, then we can deduce that this data means that it is 12 o'clock at that location (although midday or midnight still has to be determined!). On the other hand, if we don't include the inference that the clock is correct, we cannot say what the local time is (we have no information about the local time) but we still have the fact that the clock hands are at the 12 o'clock position.

This example of creating information from data explains why many people often think of data and information as being equivalent; they look at some data and automatically provide the interpretation of it that enables them to understand its meaning – the automatic interpretation of the data supplies its information content. After we have put data into context and give it meaning we can start making determinations as to the validity of the data.

EXERCISE 1.4

What is the context that allows us to confidently state that the value 11 May 2065 is not a valid date of birth for a current employee of The Open University?

Hint: you may need to think of lots of things that are in your standard set of assumptions about dates of birth for employees.

Did you know that the Milton Keynes Jesters ice hockey team beat the Oxford Bulls by 3 goals to 5 on 1 May 2005 – no, I didn't think so and it's unlikely to be relevant to you.



Notice that the information content of data in a given context is very dependent on the user of that data knowing what the intended context for that interpretation is. The date represented by 5/11/2005 depends on the context of its use. Is it intended to be 5 November (UK notation) or 11 May (US notation)? The meaning that we attribute to specific data will be independent of the interpretation that others may give it, unless there is some common agreed interpretation – that is, we don't all get the same meaning from a piece of data. For example, the failure of the NASA Mars Climate Orbiter in 1999 has been attributed to a mix-up between imperial and metric units, as applied to distance measurement data. This example represents a problem of misinterpretation that could have been resolved by checking that the originator of the data and the user of the data were in agreement as to its encoding and meaning.

It should be possible to avoid the worst problems that could arise by agreeing and documenting the meaning of data when that data is intended to be shared, and ensuring that the users of the shared data are trained in its meaning and interpretation. We return to this important point later in the course.

When designing a database to store and manipulate data it will be important that we understand the consequences of the interpretation we expect for it. Knowing the interpretation placed on the data to be stored will allow us to validate the data and ensure that only meaningful processing is applied to that data. Suppose your data included the value of two numbers: one of the values is '1200' and represents the number of books on a shelf; the other value, '2834', represents a telephone extension number. If you were writing a program to process these numbers, you would need to know whether they can be changed by arithmetic addition. In one case we have 1200 books on a shelf; viewing this number in its appropriate context would tell us that it is valid to increment it by one when a book is added to the shelf ($1200 + 1 \rightarrow 1201$) and to decrement (reduce) it by one when a book is removed ($1201 - 1 \rightarrow 1200$). In the second case, if we add one to the '2834' value, the result may be a valid telephone number, but this is not a meaningful way to process telephone numbers. Imposing limits on the types of operation that can be applied is one example of imposing **constraints** on the data held in a database. We will see in the second section of this block that constraints, of various kinds, form a large part of the definition of a database.

Some important issues arise from the relationship between data and information:

- ▶ Data can have more than one representation. We should be able to express data in different ways, according to our needs. One of the continuing themes of the course will be the consideration of different data representations for various kinds of data and communication purposes and the need for transformations between the representations.
- ▶ Data does not have any meaning in isolation. As we have shown above, any consideration of data as information can only be understood in a defined context, with definitions and descriptions to support an agreed interpretation. So, users of a computer system can view input or output data as information only if they have knowledge of the appropriate context. The converse of this represents a significant danger with any kind of computer system; if users do not have adequate knowledge of the context they cannot have a proper understanding of the meaning of the data. This leads to another theme of the course: to recognise the requirements for understanding data and to provide support for that understanding.
- ▶ Computers process data, not information. It is not possible for a computer system to have awareness of the context in which data acquires meaning. However, a computer system can be programmed to take account of some aspects of the meaning of data. To do so the designer of the system has to make appropriate

choices and give appropriate instructions in order to represent those aspects. For example, it would be possible to specify that a particular value represents a telephone number and that this can be processed only in particular ways.

- Computers can be used to store, manage and process data and so can underpin information systems by selecting and presenting data. However, for it to be treated as information it requires people to supply appropriate meaning and interpretation.

Database, database engine and database management system

In this course we will use the term **database** to refer to a collection of structured, persistent, shared data. Of course, we also need to define structured, persistent and shared for this definition to be meaningful.

Structured data implies that the data has a regular, defined, structure that supports the consistent interpretation of the data within the database and that this structure can be used to support the manipulation of the data. The structure allows a user to specify a data requirement in a way that could be consistently applied to the entire collection of data without having to consider every data item contained in the database. For example, counting the number of students with postcodes in the MK5 6EH area doesn't require the student identifiers or names to be searched since it is known where the postcode is stored in the student data files in the file-based approach we considered earlier.

Contrast this with a '**free text**' **storage system** such as could be used to store a large number of purely textual newspaper articles. In such a system the data is held as a string of words, broken into sentences, paragraphs, articles, and so on. However, this arrangement does not help us if we want to search for a specific postcode which could occur anywhere in the article, in the article heading, or any sentence of the article. To find any postcodes in a free text system it is necessary to search every word in every article. Later, when we consider **relational databases**, we will have certain additional formal requirements about the logical structure of the data that will give us benefits in processing and manipulating the data stored.

Exercise 1.5

It would be possible (although highly inadvisable) to store hospital records in a free text storage system. In such a system, for example, we might record a patient's admission by using a statement such as 'Patient Peter Jones was admitted to Clare Ward under the care of Dr Smith on 13 May 2005 for observation'.

- Assume that the system contained several such statements for different patients, wards, doctors and dates. Give an example of the problems that the unstructured nature of this data could lead to if you needed to satisfy a request such as: list the patients who are on Clare Ward.
- Why would a structured data record and a search using the structured record remove such problems?



Persistent data refers to data that will continue to be stored even when the applications creating and using the data are no longer running. This differs from data within a computer program that is deleted when the program terminates. Persistent data is usually stored within a computer file system on disk or some other long-term storage medium. Unless physically altered by a computer program the data should remain safely stored until it is next required.

Shared data ensures that a database is not limited to a specific application or use, but can be used by any user applications authorised to do so.

Another important term that we will use in the context of a database management system is **database engine**. We will be quite specific in the way in which these terms are

used in the course. We use the term database management system to represent the collection of software and procedures that enable the storage, manipulation and protection of a collection of databases. We will use the term database engine if we specifically mean a software program within a database management system that is responsible for accessing a database and manipulating data. The database engine will be the software responsible for processing a **query**, a request to the DBMS to access some data, and ensuring that database content is retrieved and altered to meet the requirements of that query. The query processed by the database engine may already have been processed by some parts of the database management system (which we consider in more detail later), for example to optimise its performance, or to check that the user presenting the query has appropriate authorisation to access the database.

Marketing material and press articles often use the term database to mean a computer system on which a database is stored or the software that manipulates the data (i.e. they refer to a DB2 database, or an Oracle database) when they really mean the DB2 database management system or the Oracle database management system. Remember that a database management system may be responsible for the management of many databases.



EXERCISE 1.6

Why would you never see a statement such as 'The database runs on all computers using the Microsoft Windows and UNIX operating systems' in this course material?

User processes

When referring to software that makes use of the DBMS we may use the terms **user process**, **user application** or **user program** synonymously. Out of preference we will use user processes to indicate clearly that these are outside the DBMS and so distinguish them from processes within the DBMS and from computer **operating system** processes in the host computer system. Where the context is clear we will just use **process**, **program** or **application**. We have done this in the earlier subsections of this block.

We will not be focusing on the type of software that is used to construct the user processes or the type of application that the user process represents. We will see later in the course that the use to which the data is being put and the manner in which it is presented to the end user is not the direct concern of the database designer or database implementer. What is important to the database designer is that the right data is held in the right way to support the user process requirements, and the implementer must ensure that the design is accurately and effectively realised in a given database management system.

Throughout this course we will mainly be concerned with manipulation of data to support data storage, update and retrieval applications. However, in this introduction we will look at some of the more complex ways in which data is being used in extended applications.

1.5 Databases and DBMSs

In this subsection we will concentrate on the specific issues in databases and database management systems. We will start by looking at some of the responsibilities of the DBMS with respect to the management and accessibility of the data so that the data is available to a wide range of user processes. We briefly introduce a way of grouping the responsibilities of the DBMS that will help us simplify our view of the DBMS. We finish this overview by considering some of the advantages and disadvantages of following a database approach rather than a file-based approach.

After we complete our overview of the database and DBMS we will examine several different forms of database structure then look at the underpinnings of the relational database systems that will be explored in the remainder of the course.

We have discussed several problems with the file-based approach; these include unproductive maintenance, the risk of inconsistency, and the need for an application programmer to know about the physical structure of the file.

In the database approach (Figure 1.4), the database is a single large collection of data which is accessible by all the user processes that need it. The database management system is used by all user processes that are required to enter, store and manipulate the data. (In fact it's more likely that the DBMS may be responsible for several databases which are independent of each other, but for simplicity at this stage we will consider just a single database being managed.) Since the DBMS has sole responsibility for data access, all activities related to data access requirements will reside within the DBMS and be accessed by the user processes. We will look in more detail at examples of the kinds of common tasks the DBMS can perform and the features of a modern DBMS in Section 3 of this block.

Consider the benefits of having a single large collection of data. If all the data is accessible then there is no need to include **redundant duplication** of the data and so we remove the risk of inconsistency. Later we will see that ensuring that there is no redundant duplication of data in the database is a key point in database design. Since all the data required by an organisation can be made available in the same database it becomes possible to cross-reference data items and ensure consistency between related data.

In the earlier description of the database approach you will recall that the presence of the DBMS permitted data independence. In order to achieve this, a database will include a description of the data it holds. This description is known as a **system catalog** or **data dictionary**. Data independence allows for the physical representation of the data to be changed without the need to change the logical representation. The data dictionary will be updated each time the physical structure is changed, to ensure that the DBMS can translate (map) between the logical name used in requests and the physical data stored in the computer system.

EXERCISE 1.7

We showed earlier that adding a field of data to a record structure can lead to unproductive maintenance of user processes in a file-based system. In the database approach, the same changes would require no updating of the user processes.

What changes would the deletion of a field of data from the database require in the user processes that utilise the database approach?

Would these changes represent productive or unproductive maintenance?

Maintaining the system catalog is a responsibility of the DBMS, more specifically the database engine within the DBMS, as it is the only software that can access the database's internal structures. However, since the DBMS is responsible for restructuring the database when requested to do so, it can ensure that the catalog is always consistent with the actual database structure.

The user processes will send different queries to the DBMS to access the data in the database. However, they can all use a common interface (a common language) for that request. In this way the programmer can learn a common database access language and the logical description of the data he wishes to access and leave the specific details of accessing the physical data to the DBMS.



The data correctness and consistency can also be managed directly by the DBMS for each database. The data in the database has been collected and structured for a purpose, and the designer and users have an agreed interpretation for the data. A designer can describe rules about the validity of the data that the user processes will attempt to insert or update in the database.

Some examples of this kind of validity rule might be:

- ▶ If a date of birth of an employee is being stored then dates in the future should not be permitted, and the age should be above the minimum age for employment.
- ▶ If a till receipt for a supermarket contains a product code then that product code should exist within that supermarket for the receipt to be meaningful.
- ▶ If a salary figure is to be changed for a grade of employees then a change of no more than 10% is permitted.
- ▶ The number of patients allocated to a hospital ward should not exceed the number of beds in that ward.
- ▶ A telephone number cannot have arithmetic operations applied to it.

Most complex databases have many rules, usually known as **constraints**, that define the representation and values permitted for valid data being entered into the database or that describe valid changes that can be made to existing data. It is possible for these rules to be stored in the system catalog managed by the DBMS. The programmer developing the user processes no longer needs to write code to enforce these rules as the DBMS can be responsible for enforcing these rules every time the database is accessed. In a similar way to enforcing correctness, the DBMS can control the **security access restrictions** (that is, which users can request what changes to what data). These can be centrally defined and the DBMS, the only software that can access the database, will enforce security access restrictions for all the user processes.

Centralising common activities in the DBMS so they can be used by multiple user processes is another typical function of the modern DBMS. An example of this might be to supply common processing tasks to several user processes – a program that calculates the postage due for shipping a catalogue order to a customer would perform the same calculation for a given customer and catalogue order, irrespective of the order originating in a mail-order processing application, web page application or telephone-sales application. So, rather than code the postage-due calculation in each of these user processes there are benefits to coding this program once and storing it within the database. The DBMS executes the code at the request of the user process and returns the result of the code's execution.

Finally, the DBMS has a role in the mapping activity between logical and physical data representations. This is another aspect of data independence and storing of program code within the database. The external user processes are not limited to accessing the data only in the internal formats used within the physical data storage system. The DBMS can take responsibility for translating between the external and internal representations and can use appropriate stored programs to achieve this. (Some operating systems perform this type of mapping when using regional settings to determine, for example, the date format.)

We can now show a more complete description of the database approach (presented in outline in Figure 1.4), with several more functional components, in Figure 1.5. The detail of these functions, where not previously discussed, will be considered in Section 3 of this block.

Any equivalent diagram for the file-based approach would show each user process containing code for each of these functions.

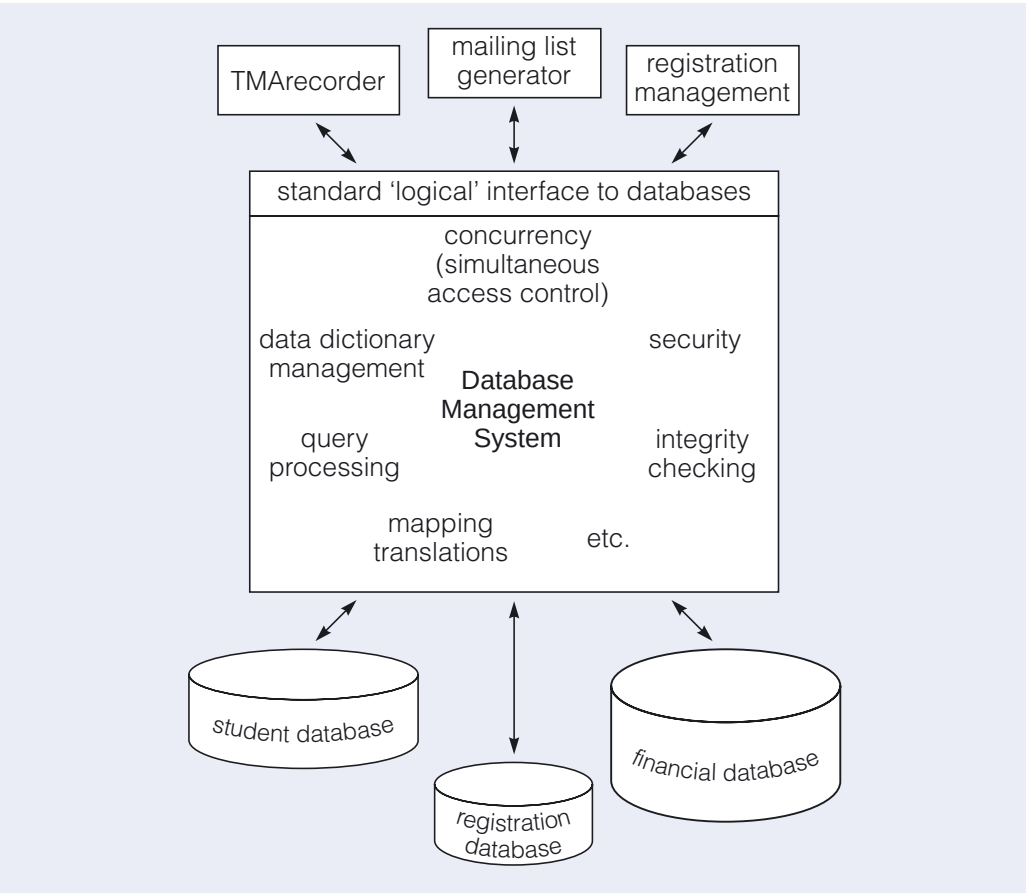


Figure 1.5 Common functions embedded within a DBMS

Separation of concerns within the DBMS approach

The previous paragraphs are about the way in which the DBMS can support the users of the database because of its central role in bridging between the user requirements and the data requirements. An alternative way of considering this is to think in terms of the separation of the concerns related to the physical storage, logical description and individual user requirements of the database system. Each of these three concerns can be considered independently provided that some mechanism exists to translate between them (see Figure 1.6).

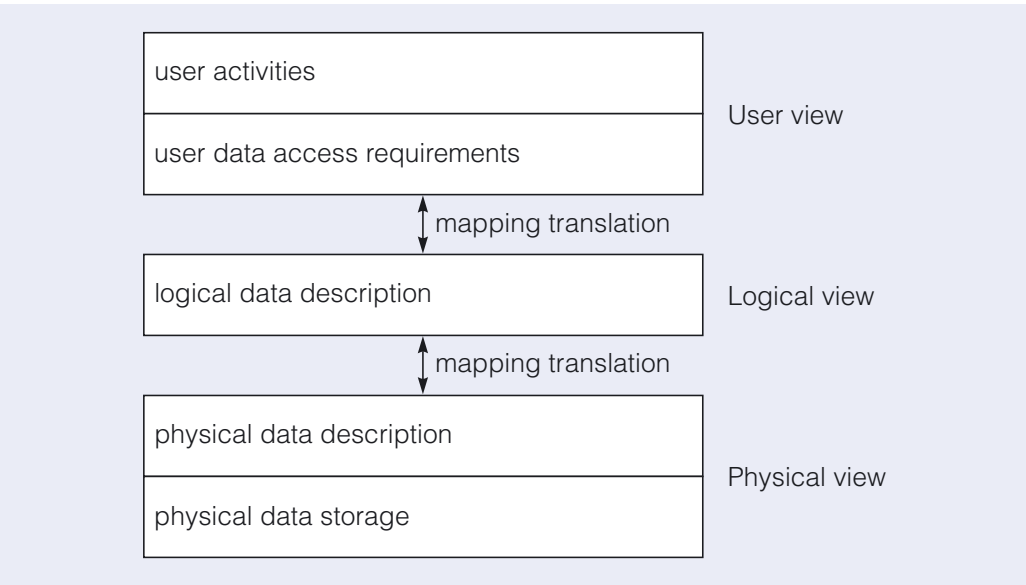


Figure 1.6 Separation of concerns within a DBMS

When a user process requests a data item it is of no direct consequence to the program how that data is physically stored, or what data processing must take place to deliver the data it requires. The user process makes a request using a logical description of the data it requires, and the resulting data is returned to the process (or an error message to indicate why the request was not performed). User processes usually only require some of the data managed by the DBMS, and may not know about the full range of data the DBMS could supply.

The DBMS will, however, need to maintain a full logical description of all the data that it is required to manage. This logical description of the database contents should be independent of both the way in which the computer system physically stores the data and the use to which the data is put by user processes. The data description, the rules for the validity of the data and the common processes that can be applied to the data are specifically concerned with the database itself, not the user processes or the details of physical storage.

At the physical level the hardware and software responsible for the low-level storage management (data files, programs, hard disks, networks and so on) is not concerned with the details of the user process making the request for data. It will receive instructions to store, or locate and retrieve actual data but these will be independent of where the requests originated, or how the result is to be used.

Of course, to make these three independent areas of concern operate as one system it is necessary to describe how they will interact and to allow the mapping (translation) between a user request, the logical expression of that request and the physical interaction with the hardware to satisfy that request. The typical DBMS will include software components to achieve this, although it is not always possible to clearly distinguish the separation between **user view**, **logical view** and **physical view** of data. Later, in Section 3, we will revisit this separation of concerns when we introduce the ANSI/SPARC architecture (also known as the three-schema architecture).

Advantages of the database and DBMS approach

We've introduced positive aspects of the database and DBMS approach to data management in some detail already. Before we look at some of the disadvantages we will summarise the advantages.

- ▶ Centralised functionality leads to consistency in the management of the data, its storage, validation, security, processing and control.
- ▶ Data can be shared reducing the need for redundant duplication.
- ▶ There is a reduction of the unproductive maintenance incurred when the requirements of data and programs are changed.
- ▶ The separation of concerns allows a focused way of considering data representations, requirements and processing.
- ▶ The separation of concerns simplifies the task for programmers developing user processes.
- ▶ Ad hoc querying (an unplanned, interactive style of query generation) becomes easily achievable because the DBMS can translate a logical user query into the more complex physical requirements of the underlying computer system.

Disadvantages of the database and DBMS approach

While there are many benefits to be gained from following the DBMS approach to managing data there are also some disadvantages. Most of the disadvantages relate to the inherent difficulty of managing a large, complex general purpose computer system.

- ▶ A DBMS is a very complex system. The brief examples we've used to describe the features that we expect in a modern DBMS do not provide a complete list, but they do outline a very complex collection of software. In such environments you would want well-trained and experienced systems managers and developers, and these can be difficult to find and expensive to employ and train.
- ▶ Risks of failure and security breaches are often severe. The sharing of a single copy of the database leads to the risk of having 'all your eggs in one basket'. If the DBMS fails then it fails for all users and all user processes until the failure is corrected. A security breach potentially leaves all data vulnerable to attack. Both of these are key concerns to large organisations.
- ▶ The hardware and software required to support a modern DBMS can be expensive. Early systems required large powerful processors and large amounts of disk space to support even modestly sized databases. This disadvantage has been reduced in recent years as the simultaneous rise in processing power and data storage capabilities has put powerful computers onto most desktops. Most banks, airlines, supermarkets and so on will be using database management systems to handle many terabytes of data on mainframe and minicomputer systems which require maintenance and support.

It should be realised that most modern DBMSs can be appropriately tailored to specific environments, and the size and complexity can be chosen to meet specific requirements. Achieving 100% availability of data and high levels of security can lead to expensive mainframe-based data management systems for organisations such as banks and airlines. However, it is also possible for basic desktop database systems to satisfy the requirements of a small social or sports club, for example.

A terabyte is 1024 gigabytes, or one trillion bytes, equivalent to over one million minutes of MP3-encoded music.

1.6 Are all database systems the same?

The issues we've discussed so far apply to all the ways in which databases are implemented, accessed and used. However, before we move on to some more specific examples of database systems we need to start thinking about the way in which we will work with our logical **data definitions** and our requests to the DBMS. The focus of this course will be on relational database systems but it is important to know that there are other database types in common use and that database technologies continue to develop. This subsection will include an overview of the relational database and some other database systems that you may hear about.

The development of database systems falls into three main eras. These are commonly referred to as the **pre-relational**, **relational** and **post-relational** systems, a consequence of the significance of the relational database model. The divide between pre-relational and relational occurred around the mid-1970s when the first commercial relational database systems began to appear. Relational databases have been the dominant trend in database technology up to the present day. The post-relational systems, until the late 1990s were mainly experimental and research systems. However, in the last decade the first commercial object-oriented database systems and XML-based systems have begun to appear. At the time of writing it is not clear that the newer systems will have the same general applicability that the relational systems have enjoyed. Even if these technologies prove to be successful it will be several decades before the legacy relational systems (and even some legacy pre-relational systems) are replaced.

Pre-relational systems

The earliest database systems offered very little support for data independence. The physical storage of the data was visible to the programmer, who needed to explicitly code the access mechanisms for that data storage. The pre-relational systems fall into two main types (although they are closely related): **hierarchical** and **network** systems. Both of these were developed in a largely ad hoc manner to meet specific data requirements and were implemented in very different ways. They only became formal 'models', with an agreed definition and standard behaviour, after abstracting away from the implemented systems to provide a common description. The hierarchical and network models lacked any theoretical foundations to support the programmer in producing the database design or in manipulating the data representations and constructing their queries. Consequently, it was difficult for a programmer to formally reason about the databases they were developing, which can be important when trying to predict program behaviour or test its performance.

Products built on these models became available in around 1968 and **legacy systems** using successors to these products are still in use in mainframe environments today. The continued use of these systems suggests that they had some positive points. Such benefits were mainly related to the high performance that could be achieved for certain applications because of the ability of the programmer to interact directly with the physical storage structure (and so avoid the translation between logical and physical representation that takes finite processing time). This, however, also makes it complex for a programmer to manage the data and to deal with ad hoc requests. The mechanisms used to store the physical data had the problem that it led to better performance for some applications and worse for others. For a simple example of this kind of dilemma, consider a university database in which it is relatively straightforward to generate a list of students registered for a specific course, but it might be a difficult task to produce a single alphabetic listing of all the students at the university from that same database.

It would take many pages to describe the hierarchical and network models in detail; indeed, these systems are beyond the scope of this course. However, for completeness, we can give an idea of the way in which the structure of data appears in these models, which will be useful for assessing the strengths and weaknesses of these approaches when manipulating data. In the hierarchical model, data is arranged in a hierarchy (more specifically a tree-structured hierarchy), as shown in Figure 1.7.

A way to think about this data model is to consider a simple filing cabinet structure. Suppose you wanted to record details about course presentations and students enrolled on the courses. One structure for this might be to have a filing cabinet for each year that courses are presented, and within the filing cabinet have a folder for each course. Within each course folder you could have the data for each student enrolled on that course written on a separate forms. Notice that if you wanted to process the records of all the students who were enrolled on M359 in 2007, for example, then you would go to the 2007 filing cabinet and the M359 folder, and the data about students would all be in that folder. However, if you wanted to know all the courses that a student, Junhi Smythe for instance, has enrolled on then you would need go to the first filing cabinet, first folder and look through that, then the second folder, then the third; eventually you would move to the second filing cabinet, and so on – the physical structure of the filing cabinet doesn't make this kind of search very efficient. Also, you cannot have students enrolled on two courses without copying their data and placing it in two folders. There are, however, various indexes that can be created to improve the search within the hierarchical structures. For example, an index based on the name of each student could tell you in which course and year to find the student record – but the index structures themselves can become complex and difficult to manage.

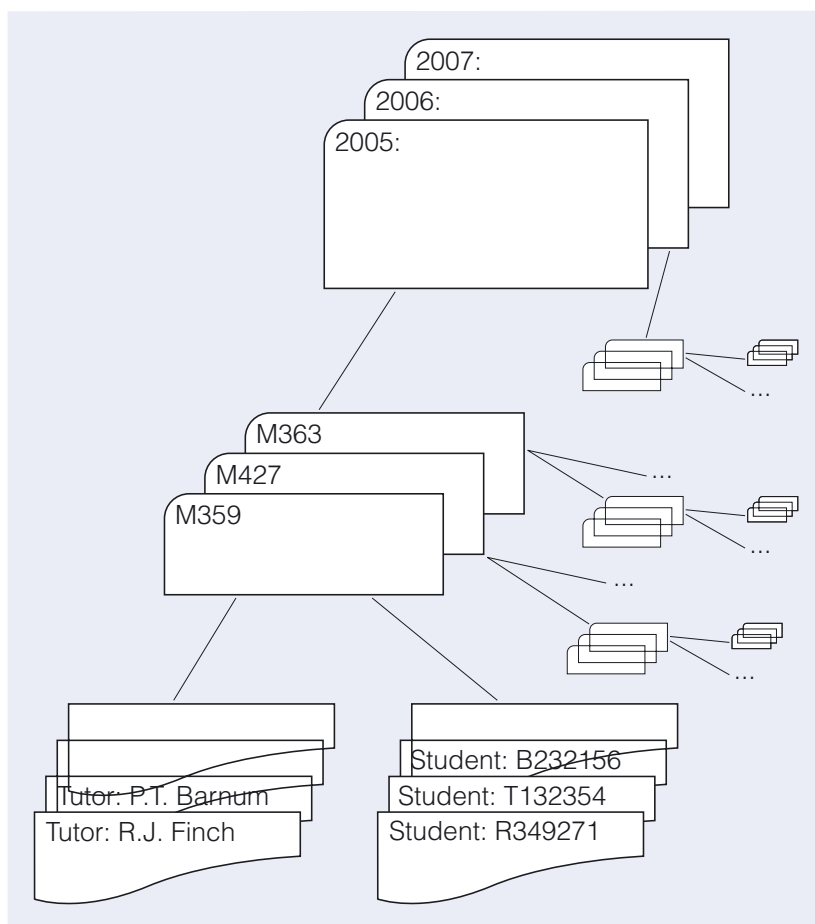


Figure 1.7 Hierarchical data structure showing students enrolled on courses in different years

In the hierarchical model a data item always has a link to a single data item in the level of the hierarchy above it – in data structure terminology each child data item has a single parent data item to which it is related. The network model extends this to permit multiple parent data items, so for example a student record could be linked to the presentation of the course and to the year of enrolment, and could be accessed using either of these links. The network model has increased flexibility but is even more complex to navigate than the hierarchical model as there can now be several different ways to access the same data. However, for large amounts of data that is queried in ways that fit with the physical data structuring, such systems are extremely efficient and are still used in large-scale data processing activities such as airline reservation systems and financial transaction systems.

Relational systems

The relational model for database systems was developed at the IBM laboratories in the 1970s. Relational database systems were based on the work of E.F. Codd and first presented in his paper 'A relational model of data for large shared data banks'. Codd proposed a logical data structure (in contrast to a physical data structure) in which all data appeared in a tabular form, known as a **relation**, and which could be manipulated mathematically by operations that produced results that were also in tabular (relational) form. (Don't worry – we will be reviewing what that last sentence means!) In addition to working on the underpinning mathematics, Codd was later to produce a list of objectives for relational database systems so that the formal mathematical model could be preserved when applied to implemented database systems. These objectives could be seen as a direct response to the problems of the hierarchical and network

Codd, E.F. (1970) 'A relational model of data for large shared data banks', *Communications of the ACM*, vol. 13, no. 6, pp. 377–387.

models in that they set a high degree of importance on data independence, consistency, redundancy management, flexibility within the database and the ability to express ad hoc requests against the defined data. In later blocks of the course we will go into a great deal of detail regarding the theoretical relational model and also the offshoot from the IBM work on producing a relational database implementation, the **Structured Query Language** (SQL). It should be noted that very shortly after it was developed, the SQL language was extended to include several non-relational features and this development has continued with each new version of SQL. Many people consider that SQL should no longer be considered a relational language, in much the same way that few people would consider a modern jumbo jet to be an example of the Wright brothers' technology. The relational theory and relational model were developed before the first relational database implementation rather than being retro-fitted to actual implementations in the way the hierarchical and network models had been. Note that we will be coming back to SQL in detail later in the course.

In the context of a relational database, the relation is a mathematical structure that, for the moment, we can think of as a table with named columns of data. (We will introduce a more formal definition of the relation in *Block 2*.) Figure 1.8(a) shows a relation named *Student*, representing student data. Each row in the relation represents data about a different student, and the data along one row is data about the same student.

Student

Identifier	Name	Address	Postcode
R234534768	Ron Weedle	8 Danube Way	MK76EH
R282738768	Advinda Emir	23 Smooth Lane	NP128RG
T273297423	Robert Ablay	11 Artifact Lane	SR917LR
R272982388	Martin McGavin	23 Smooth Lane	MK92RG
P218293428	Mellisa Traspon	24 Rough Drive	KE389FT
R882937392	Eva Stone	9 Pennsylvania Avenue	PR239RT
P293479234	Robert Ablay	10 Downing Lane	RR892TF

Figure 1.8(a) A representation of a *Student* relation showing several student records

Simple queries can be applied to the *Student* relation, for example, to select specific students according to some condition such as selecting those students whose postcode begins 'MK'. The result of this selection is itself a relation (Figure 1.8(b)), containing those rows that satisfy the condition and all the columns from the original table.

Identifier	Name	Address	Postcode
R234534768	Ron Weedle	8 Danube Way	MK76EH
R272982388	Martin McGavin	23 Smooth Lane	MK92RG

Figure 1.8(b) The relation that results from selecting those students in Figure 1.8(a) who have a postcode that begins 'MK'

It is also possible to request certain columns from the relation – for example, to get just the name and postcode for students in the *Student* relation.

Name	Postcode
Ron Weedle	MK76EH
Advinda Emir	NP128RG
Robert Ablay	SR917LR
Martin McGavin	MK92RG
Mellisa Traspon	KE389FT
Eva Stone	PR239RT
Robert Ablay	RR892TF

Figure 1.8(c) The relation that results from requesting the names and postcodes of the students in Figure 1.8(a)

This result is also a relation (Figure 1.8(c)), and we could if we wanted now apply the first query to this relation and again this would result in a new relation (Figure 1.8(d)).

Name	Postcode
Ron Weedle	MK76EH
Martin McGavin	MK92RG

Figure 1.8(d) The relation that results from selecting those students in Figure 1.8(c) who have a postcode that begins 'MK'

This property of applying operations to relations and producing relations is referred to as the **closure property** and is very important to the relational model. In the relational model, closure means that you can build up very complex expressions and always apply relational operations to the results of those expressions, the results of which are always relations themselves. Codd developed a sound mathematical language for representing complex queries against the relations (which will be the focus for *Block 2* of this course).

Exercise 1.8

- Starting with the data shown in Figure 1.8(a), what would be the relation that would result if you first selected those students whose first name begins with 'M' and then selected just those whose postcode begins with 'MK'?
- Would you get the same result if you did the two selections the other way around – postcode first, then first name?
- Suppose I asked for the final result of first selecting those students whose first name began with an 'M' and then selecting those students from the first result whose first name was 'Martin'. Is there a more efficient way of getting the same final result?



Codd's **relational model** only applied to the logical description and manipulation of the data. It said nothing about the physical implementation of the underlying data or how to achieve the manipulation of the actual data values in the database. Early attempts to produce a commercial implementation of the relational model failed when the expressions that could be written to manipulate the relations far outstripped the ability of the software and hardware to efficiently process the data. It took the development of the

automated **query optimiser** to enable the commercial development of the relational model. A query optimiser examines a relational query and, with knowledge of the physical data structures used to implement the database, works out how to turn the logical query into an efficient series of physical processing steps to achieve the desired result. This enables the programmer to completely ignore the physical data structure when expressing requests to the database management system – a major step in achieving effective data independence through mapping between the logical and physical representations.

The combination of simple logical structure (the relation), the ease with which requests can be expressed using a logical representation (exploiting relational theory and automated query optimisation), and the flexibility of the underlying data representation soon ensured that the relational database model was a commercial success.

Systems based on the relational model have been dominant in the database market for nearly three decades now and their influence in the commercial environment will continue for many years to come. However, they do not mark an end to database system development.

Post-relational systems

There are several criticisms that have been raised against the relational model and its SQL realisation in a DBMS. These focus on two main areas, the representation of the logical structure and the nature of the processing available within the model.

The relation is the only logical storage structure in the relational model and is used to represent both facts about things in the world (students, courses) and the relationships between things in the world (students enrolled on specific courses), so there can be some confusion regarding the meaning associated with the data held in the relational structures. Attempts to develop more semantically meaningful data models have been made. Furthermore, the relation is also a poor representation of things in the real world, often imposing an unusual break-up of the information representing a single real world 'thing'. An example might be the representation of an order form, which has customer, delivery, payment and product data on it but is clearly a single thing, an order form. In the relational model the representation of the data could involve separate relations for customer, address, payments, products and so on. There are other representations of the data, for example an object-oriented representation, that would allow the order form to be considered as a single data item into which is embedded additional data about customers, addresses and so on. A query might treat the entire order form as a single object, or process the embedded content of the order form as required.

The style of processing available within the relational model has also been subject to criticism. The relational model operates on data in sets (that is, a collection of unordered values), but most programming languages operate on a single value at a time. This distinction between set-at-a-time processing and value-at-a-time processing is sometimes referred to as the **impedance mismatch**.

Impedance mismatch means that if a programmer is developing a user process that interacts with a relational database, it is necessary for the programmer to represent the query to that database in a relational (set-at-a-time) manner, but the rest of the user process is coded using value-at-a-time processing. The database returns a set of values (for example, containing the values representing all the student records in the relation shown in Figure 1.8a) which the user process must then access one record at a time. The programmer, or the programming environment they are using, is required to convert between the two approaches whenever data is transferred from the set-based to the value-based environment.

Critics of the relational model often suggest that it fails to capture the meaning (**semantics**) of the data being stored, and that to accurately model a given data requirement often results in complex relational structures and the need for complex queries to express common data processing requirements. Further criticism suggests that programming languages used for user processes and the languages used to manipulate database content are too dissimilar. This makes the database designer and user process programmer's tasks more difficult.

There has been much research interest in extending the relational model to include new features, usually focused on increasing the semantic content of the database which leads to models such as the semantic hierarchy model. Other developments have moved away entirely from the relational theory into alternative data structuring and processing models and some have combined the relational model with other approaches to data representations.

Object-oriented database systems

Object-oriented technology emerged during the 1980s as a programming language concept and has become the dominant software paradigm in the 2000s. In object-oriented programming languages the data structuring is achieved using **objects**. Objects can be regarded as independent packages, with both values (data) and behaviours, which respond to requests (known as messages) from other objects; they can often have complex nested data structures within the object. The object structure (and naming) is said to give a better reflection of the real-world properties that a programmer might want to manipulate. For example, a `student` object might have within it an `address` object, a `course-enrolment` object and so on. The behaviours associated with each object will define how that object can behave. For example, a `course-enrolment` object might have behaviours for a `new enrolment`, the `cancellation` or `suspension` of an enrolment, as well as behaviours to report the values of the enrolment such as the date the enrolment was made. These behaviours, also known as methods, can be associated with entire collections (known as classes) of objects. So two instances of the `student` object might each have an `enrolment` object embedded within them, the `enrolment` objects would come from the `enrolment` class and have the same set of associated behaviours. When an object receives a message to execute a particular behaviour, the object will perform the actions defined for that behaviour; these can process the data belonging to that object, send messages to other objects and possibly return a result to the message sender. Objects are therefore descriptions of both structured data and the behaviours associated with that data. Objects are also **encapsulated**; that is, their internal state is hidden and only accessible through the methods. When an object is passed an appropriate message, its methods are executed; these methods can include so-called accessor methods, which are used to store values in an object and retrieve the stored values as required. The way in which the data is stored internally and the programming required to implement the accessor methods is not visible or of interest to the other objects or user processes that make use of the object.

Those involved in object-orientation have always had an interest in databases. Some of the earliest object-oriented programming developments were aimed at producing a **persistent programming language**. A persistent programming language is one that preserves the internal state of the user process, its variables and their values, between different executions. In addition, the persistent data created by a user process should be accessible to other user processes to form a pool of shared data.



EXERCISE 1.9

- Objects are said to be encapsulated. What database concept have you met that is similar to encapsulation?
- Why does the persistent programming language idea naturally lead to an interest in databases?

There have been two major ways in which object-orientation has developed in the database area. The first is in the development of a new type of database management system, the object-oriented DBMS (OODBMS), which focuses on the unique issues in object-orientation and embeds the appropriate storage and manipulation in a suitably constructed DBMS. The second development is a more evolutionary approach where relational database technology has been augmented with object-oriented features and mechanisms for internally representing objects within the relational environment. This hybrid approach has led to the development of the object-relational DBMS (ORDBMS).

Object-oriented database technology represents another technology developed without any formal theoretical model – objects are complex things to manipulate in programming languages and can be even more complex to handle in a database environment. With the widespread adoption of broadband the industry expects that object-oriented database systems will be key components in complex multimedia web environments where there are complex web pages incorporating graphics, audio, video and other structured complex data. An OODBMS can be built that will effectively manage the complex data and processing required to support such requirements. What isn't clear for such complex technology is how well it will deal with the less complex requirements of the basic text-based information that is currently at the heart of most business organisations.

XML database systems

XML (eXtensible Markup Language) uses **data tagging** to indicate the meaning of data within an otherwise free text document. XML originated as a **markup language** (like HTML) for use on document-based websites. However, it was quickly applied to structured message documents in business-to-business communications such as those used in ecommerce to automate the exchange of order forms, invoices and so on.

For example, an XML document may contain information about a person such as their first and last names and their date of birth, and could use 'tags' to delimit this information.

```
<firstname>Kevin</firstname> <lastname>Waugh</lastname>
<dateofbirth>20-07-1963</dateofbirth>
```

Tags can be embedded within each other to represent structured data items; for example, to indicate that an address is made up of a house number, street name, postcode and country we would embed tags within tags.

```
<address> <houenumber> 8 </houenumber> <streetname> High Street
</streetname> <postcode> MR245TD </postcode> <country> United Kingdom
</country> </address>
```

The tag names are used to indicate the intended meaning of the values they surround. So we can search any document that has used the same tags in the same way to locate, for example, the last names of people who were born on 20-07-1963. An XML system will not need to know in advance how the data is structured as it can determine the structure by looking for the start marker and end marker tags. Consider the example of the free text records of patients in hospital wards suggested in Exercise 1.5.

A single tag consists of two elements: a start marker `<x>` and an end marker `</x>`.

It should be clear that a database built around free text with XML data tags would permit a structured search. The use of appropriate tags could distinguish between the doctor's name, the patient's name and the ward name, for example.

EXERCISE 1.10

Suggest the XML tags that could be defined so that the free text patient records in Exercise 1.5 could be unambiguously tagged. (Don't extend the tagging beyond the simple cases needed for Exercise 1.5.)



The list of possible tags and the meanings for those tags can be defined for specific application areas, such as orders, invoicing, payments and shipping notes. Two businesses can then exchange meaningful data without necessarily having the detailed structure of their messages agreed in advance. Provided that the correct tags are used then they can be located and the data they delimit can be reformatted into the preferred structure of the receiving business. The standardisation of **XML schemas** (collections of tags and their descriptions) has been said by many to have enabled the ecommerce and ebusiness growth of the late 1990s and early 2000s.

The XML database systems grew out of the business needs to store and manipulate XML-tagged documents. Like the pre-relational database systems, they lack formal data models. However, because of the sheer volume of XML-based messages and the needs of business to store and manipulate these, there is a lot of interest in the development of XML databases. At present vendors are incorporating facilities for XML data management into existing DBMSs. The web community is developing standards for the representation, storage and manipulation of XML-based data. We will return to consider database developments related to XML in *Block 5* of the course.

1.7 Specialist data processing systems

We finished the previous subsection by considering some examples of post-relational systems where the logical representation of the data differed from the basic tabular structure of relations and the tree-like structures of the hierarchical databases. These examples represent developments in the core technologies for the representation, storage, manipulation and retrieval of facts and structured data. So far, we've mainly talked about the kind of processing which is known as **Online Transaction Processing (OLTP)**, also known as an organisation's operational processing (operational in the sense that the database supports the day-to-day operations of the organisation). In OLTP we present **interactive requests** to the DBMS to store, retrieve or update small amounts of data in simple operations (transactions). We've also focused on the basic form of the data that we record in our databases as being simple facts. For example, we may have a student element with a name and a student identifier, address, postcode, and so on. OLTP is involved in maintaining an accurate snapshot of the current state of a collection of facts about which an organisation is interested – for example, an accurate list of customers' names and addresses for a supermarket chain. This kind of record keeping activity makes up the vast majority of the processing done by today's database systems.

There are a number of different types of application of database systems that have become sufficiently important and widely used that they have become specialist systems considered distinct from the OLTP systems. We will introduce some of the specialist systems and describe the ways they are used and briefly how they differ from the database technologies we will be considering in the course.

The alternative to online processing is **batch processing**.

Online Analytical Processing systems (OLAP)

Analytical processing refers to processing systems where the data stored is analysed to produce new information rather than simply being retrieved. We use data collections to generate new knowledge by applying analysis techniques to our data looking for trends, relationships and classifications. OLAP is usually seen as the combination of two database technologies: data warehousing and data mining. A data warehouse is the repository for the data that data mining techniques analyse.

Data warehousing is concerned with the integration of large quantities of historical data from many different sources intended to support decision-making activities within an organisation. The data from the internal operational systems is periodically copied into the data warehouse to act as snapshots of the operational system. This has the advantage that the performance of the operational system (used for day-to-day record keeping) is not compromised by the needs of the analytical systems (used for strategic decision support). The data warehouse contains historical data (unlike OLTP where it is usually current data). For example, a data warehouse might store all the previous addresses for a supermarket customer and the dates on which they reported their change of address. Similarly, a data warehouse might record an entry for the daily stock level for every item held in all of a company's stores for every day since 10 May 2005. This kind of data, once stored and validated, does not change. If the report for 11 May 2005 says that 12 oak doors were in stock in the Liverpool store then this will not change after that date, unless it was found to be inaccurate. Data is added to a data warehouse regularly; the frequency and level of detail of data recorded in the data warehouse will reflect the importance placed on the specific data for decision support by the company's management. The data warehouse will incorporate many different sources of data because it is expected to form a single company-wide integrated repository of all historical data that can be used to support important strategic decision making. So, for instance, supermarkets might include weather data in their data warehouses to enable the identification of purchasing trends related to weather conditions. We all know that more people buy ice cream when it is hot and sunny, but a supermarket sales analyst would be able to tell you how much more for each degree rise in temperature and how those amounts vary between stores in different parts of the country or even within a single town!

The physical structure of the data warehouse can be optimised to process these large quantities of historical data. They can take advantage of the fact that the data is rarely updated once it is in the warehouse. For example, a data warehouse will often contain a large number of summary facts about the raw historic data. Returning to our previous example, the total value of the sales of all oak doors in Liverpool in May 2005 can be calculated once at the end of May 2005 and stored in the data warehouse for immediate reference whenever that summary fact is required. These summaries are usually structured, so that an analyst can get as much or as little detail as they require. The supermarket company might have sales summaries at the country, county, town and store levels, and over yearly, monthly, weekly and daily time periods (or even every hour and minute!). This ability to view the summaries at different levels of detail is referred to as drilling-up and drilling-down, and most data warehouse systems offer extensive facilities for querying data in this way.

Data mining is the name given to the specialist types of processing that generate new knowledge from the large number of facts and summaries held in data warehouses. Data mining extends the traditional statistical data analysis by utilising pattern recognition and artificial intelligence techniques to automate the search for new knowledge (sometimes known as machine learning). Data mining is widely used to enable large organisations to improve their services to customers. Knowing the buying patterns for typical customers permits targeted advertising, while knowing the pattern of

credit card use that typically represents fraudulent card use allows companies to block suspicious card transactions and check with the card owner.

In traditional data analysis a data analyst might ask specific questions: Did the sales of ice cream increase in June? By how much did the sales of ice cream increase when we reduced the price by 5%, and by 10%? What percentage of customers buy ice cream and strawberries on the same visit? In data mining the analyst will ask a more general question. For example, what factors influence ice cream sales? The system will then analyse the data on ice cream sales and anything else that might relate to ice cream sales. It will automatically generate many hypotheses about the impact of these related factors in the sales of ice cream. The data mining system will then test these hypotheses using the available data and report back to the analyst. For example, it might include in its report such things as the high likelihood that a customer who buys waffles will buy ice cream more frequently during cold weather. It might produce a classification for frequent and infrequent ice cream buyers based on records of purchases by customers over several years – a middle income couple with two children will frequently buy ice cream when their total purchase during a visit to the store is more than £80; a middle income single male buys ice cream infrequently unless there is a two-for-one offer. Such new knowledge is valuable to the supermarket as it can be used to determine stock levels or when to make price reductions, or more crucially when not to make price reductions.

Sometimes the interesting knowledge is not that there are trends, correlations or predictions that can be made about large amounts of data. Often it is the anomalies that do not fit the overall patterns that lead to important new knowledge. So, the ability to locate and report details about data which does not conform to otherwise accurate patterns and trends is another key characteristic of data mining systems. This ability to spot unusual occurrences is important to credit card companies as these often represent instances of fraudulent card use.

Data mining is not a 'naive user' technique; such tools are complex and should be used only by skilled decision makers. They need to understand the business (or science) being analysed, the problems being analysed and the techniques the system will employ, and they must be able to make a value judgement about the meaning and trust that can be given to the new knowledge generated by the data mining techniques.

Geographic Information Systems (GIS)

A GIS is an information system that deals with spatial data that is commonly found in maps. This can include land survey information, land usage information, street maps, demographic data and other forms of maps such as sea charts and aerial photographs. Typical GIS systems are capable of generating textual reports but their normal mechanism for results output is in the form of annotated maps and graphical visualisation tools.

The type of data maintained in a GIS is usually based around digital map representations, to which additional information about locations or regions can be related. Data can be related to the geographic location and to spatial data relative to other locations. The digital maps can then be augmented by overlaying additional information depending on the application requirements – land use, mineral deposits, housing density, rainfall patterns, household income, catchment areas for supermarkets, political party membership and so on. Most online mapping services available via the internet, for example for road journey planning, permit the user to make basic choices as to the type of information displayed. For instance, along with street and road names, you may wish to include sites of historic interest, hotels, Post Offices and so on.

Medicine, high-energy and particle physics, astronomy and molecular biology are other areas in which data mining is routinely used.

A GIS can be used to answer OLTP-type queries, such as: What is found at the location of the postcode MK7 6AA? However, the real value of a GIS lies in the ability to combine geographic information with decision support systems and utilise techniques from data mining to answer questions such as: Which counties have a high density of housing estates with an area of at least two square kilometres, within 30 kilometres of a main-line railway station and an average household income 20% higher than the county average? Complex trend analysis and pattern identification is also possible in some systems. This allows for requests such as to identify major roads where the rate of road accidents has reduced, or to identify the road features that occur where there are very high fatal accident rates. The most sophisticated GIS systems are in use for modelling purposes, allowing planners to study the effects of, for example, increasing housing density, road placement, re-routing of a waterway, allowing river levels to rise, or placing a supermarket at a specific location.

Multimedia databases

Multimedia is the term used to describe collections of recorded or generated graphics, audio, video and animation. Previously such data was in the domain of the games and entertainment industries, but it is now an expected part of most training, educational or sales presentations and most web experiences. Multimedia items are also being stored as historical artefacts in their own right. The ability of modern computer systems to capture, store and present multimedia data has resulted in specialised database systems optimised for the specific requirements of multimedia access.

Multimedia data structures are typically large and complex: a high-resolution photograph is typically around 2–4 MB, a music track needs about 4 MB, a ten-minute high-resolution full-screen video can require 10–40 MB, a DVD recording of a movie requires several gigabytes. In fact multimedia items can be arbitrarily large, and physical storage within a database must be capable of holding these items. Multimedia data values are also complex values: an image file for a digital photograph contains within it information about the camera used, the picture resolution, the date and time the photograph was taken, technical aspects of the camera settings such as lighting, shutter speed and so on. All of these can be embedded in a data item held by a database and may be the target for specific queries – such as retrieving all the photographs taken on 21 May where the camera flash was switched on but did not fire. Multimedia database systems are therefore constructed to permit the manipulation of data items with complex, searchable internal structures. Consequently, object-oriented database systems are frequently used for multimedia storage.

Much of the querying of multimedia is based on the **metadata** used to describe the multimedia data item rather than directly on the data item itself. The date the image was photographed and the use of flash are examples of metadata typically held about photographic items. A movie will have metadata that may include one or more titles, an actor list, director list, genre type and so on, that are associated with the actual movie data file. Researchers have been extending the ability to directly search the content of multimedia data by looking for specific patterns or features. Some success has been achieved but the technologies for this are still at a very early stage. Simple static image analysis has been quite successful and facial image recognition from photographs and video is a commercial reality. However, the technology is still a long way from being able to ask for the retrieval of a selection of still images from scenes in comedy movies in which actors who look like Robin Williams can be seen wearing bow-ties.

EXERCISE 1.11



Which of the specialist database technologies would you use in the following situations?

- (a) You want to find out the significant circumstances that lead to a student enrolling for a course after they have completed the one they are currently studying.
- (b) You have a large catalogue of photographs and video recordings that are related to your activities as a bird-watcher. You record several details about your bird-watching sightings and some of those have photographs and occasionally video included. You also hope to be able to include sound recordings of bird noises from future sightings – you've asked for a digital sound recorder for your birthday (and a bigger hard disk to store all this data on!).
- (c) You work for the Royal Society for the Protection of Birds. You are sent summaries of bird sightings and conditions (location, time, place, weather, bird activity) from bird-watchers throughout Britain. You want to be able to see if bird populations in different places are rising or falling and if there is evidence of birds changing their behaviour. You also want to be able to send 'bird-watching advice' maps back to the bird-watchers who want to know the expected locations of specific birds at different times of the year.

We've mentioned these types of specialist databases to give an idea of how operational databases used for core business activities differ from those intended for specialist applications or to support information systems and decision-making processes. The type of data and the way in which that data is manipulated requires specialist processing – a multimedia system has large data items linked to metadata, while data warehousing involves large quantities of historical facts whose values are never changed after being inserted (unless to correct errors). The fundamental issues of data representation, storage and manipulation still need to be addressed for these systems, but the specifics are tuned to the requirements of the application area.

In this course we will not be looking at further examples of these types of specialist system but will concentrate on the design and implementation of operational databases. You should be able to extend the knowledge you gain from this course into specialist database application areas.

1.8 Glimpsing the future of data management

We started this section with a look at the historical development of data management systems; we will finish with a brief consideration of the future for data management systems. Unfortunately computing isn't a predictable subject because of the impact of sometimes minor technology breakthroughs or refinements, or the combination of existing technologies in interesting ways. For example, the technology that enabled the development of the Web is attributed to the invention of the image data tag by Marc Andreessen in 1993 – a minor improvement to a core technology which had a major, and largely unforeseen, impact.

However, we can say that the short- and medium-term predictions for data management must take account of the current commercial markets – this would suggest that the predominant technology for the foreseeable future will be SQL-based database management systems. The commercial investment in pre-relational database systems has resulted in an enduring market for niche applications and products. This has ensured the continued existence of hierarchical and network database management systems and the continued need for staff to manage and control them. In the same way, the enormous investment in the development of SQL-based DBMSs over the past 30 years will not suddenly disappear. There will be incremental changes in the

capabilities and features of such systems; we're already starting to see databases that are distributed, replicated, web-enabled, object-enabled, XML-enabled, multimedia-enabled, mobile-enabled, and new features will continue to be added to the core database technologies. For a long time these systems will continue to be SQL databases with the capability of handling the data and data systems deployed in today's organisations. We have already mentioned that the early SQL DBMSs were developments of the relational model, rather than accurate implementations of the model. Since those early systems they have developed along pragmatic, customer- and technology-led routes rather than as refinements of the underlying relational theory. This trend will continue as customers press for specific capabilities and new technologies impact on the data management industry. However, in almost all vendor systems that claim to be SQL-based DBMS products you will find that the legacy capabilities remain and that in most cases it is still possible to implement a database that satisfies the rules and definitions of the relational theory and the legacy limitations of the earliest SQL DBMSs. The future of the SQL market is probably secure for the foreseeable future, but will be subject to slow development.

The development of specialist information systems and their supporting database technologies is much less predictable than general data management systems. GISs grew out of a specific need to manage particular kinds of information, and such developments are not always easy to identify. Some developments, however, arise through technology breakthroughs or developments in related fields. The low cost of digital cameras and digital sound recorders has fuelled the development of consumer-based multimedia data management products (photograph albums, digital jukeboxes and so on). The majority of complex websites are database-driven with basic user interfaces coupled with web programming to access web pages or textual data held in one or more databases. As this technology continues to mature there may be specialist tasks that lead to a new type of web-specific data management system (which will probably be XML-based). The extension of mobile phone technology with high-resolution screens and the use of wireless PDAs will give rise to mobile data management requirements focused on location-specific data – the systems to support such activities are still in the early stages of development.

In the last 5 to 10 years the development of object-oriented technologies has been expected to have a major effect on the data management market. However, their impact has largely been restricted to specialist markets where aspects of the object model (encapsulation and inheritance) have direct benefit. Most multimedia systems and **computer-aided design** (CAD) systems use object-oriented database management systems, for example. While entire OODBMSs are being developed, it is more common to find that existing SQL-based DBMSs are being extended to incorporate object-oriented capabilities. This may represent a transition phase between the technologies that will gradually see the emergence of the object-based databases.

At the time of writing, XML data representation is being suggested by some as the natural successor to SQL data representations. There are two kinds of XML database technologies entering the commercial markets. The first type is an XML-enabled system in which an SQL database system is used to store XML data. In such a system there needs to be a translation to and from the XML and SQL (or other) representations, which may be a complete or partial translation. The original XML document is usually also stored as a single item. This is a bit like extracting the metadata being held within a complex multimedia data item and storing it in a relational database alongside the data item itself. The second type of XML database is the native XML database. Here the entire XML document is stored and the system indexes the contents of the document directly. While the storage aspects of the XML database systems have been largely resolved there still remain many issues relating to how queries can be expressed to retrieve and manipulate the XML structures. Successful attempts have involved either

extending SQL-like capabilities or using the navigational approaches of hierarchical databases. While it is certain that the development of XML will lead to its use in the data management field, how quickly it will impact and how major that impact will be is still not predictable.

We will briefly consider aspects of both the object-oriented database technologies (from the perspective of Java technology) and the XML database technologies in *Block 5* of the course.

1.9 Enduring issues in data management

There are several key characteristics that have been considered in the early part of this block. These are applicable to data managements systems in general and are not specific to any particular era of technology, kind of database system, or application area. These characteristics underpin the general field of data management. As you study the remainder of this block you will revisit these topics, where the specific focus will be on their relationship to the relational model and relational database management systems.

EXERCISE 1.12

List the enduring principles of data management covered in the earlier part of this block.



The key principles summarised in Solution 1.12 can be considered as examples of general issues in data management. Before concluding this section we'll briefly cover these issues.

Meaning

Data requires context for interpretation and its use must be relevant to that interpretation. Often we take it for granted that we understand what is meant by someone using a particular term – say, something as straightforward as the word 'Customer'. However, if we are building a data management system that is required to store and manipulate data about Customers then we need to be exact in our understanding of what is intended when we use that term. Is a Customer someone who has bought something from the company, who has registered as a customer but not yet bought anything, who has browsed an online catalogue, who has ordered but not yet paid for an item, who has paid for but not yet received an item, and so on? We sometimes refer to the agreed domain of discourse for the database content, and we would expect to be able to find a description of the meaning of the terms used within that domain. The **domain of discourse** serves to define the scope and meaning of the data that the organisation has identified as being valuable to it. If data is to be meaningful to large groups of people then we require standards to define the agreed conventions for exchanging data, documentation requirements and manipulation rules. In such cases formal **standards bodies** becomes responsible for publishing and enforcing these standard descriptions.

Persistence and durability

A data management system must ensure that the data it manages is persistent and durable. Unless explicitly changed or deleted by a user process, the values entered into the database should be maintained without loss or change for as long as the data is required.

Correctness and consistency

As far as is possible, meaningful data must be accurate and correct. If we do not know that the data we hold is correct then we will not be confident in using the data. However, there are limits to determining the accuracy of our data. If a person's name is given to you as 'Kevan' it is not possible, without additional data being supplied, to determine if that is a misspelling of 'Kevin' or is an accurate representation of the person's name. If someone said they were 216 years old then you would know that the value given was incorrect. Although you could not say what their true age was, you would know it wasn't 216. Consistent data represents a form of accuracy in which the internal relationships between data are known to meet certain agreed rules. Suppose we agree that the meaning of Customer is that it represents someone who has purchased something from the company. If a customer's details are recorded in the database there should also be data about at least one purchase that the customer has made. If no purchase can be related to the customer name then by definition the appearance of this customer name is inconsistent within our agreed domain of discourse and should not have been stored.

Data accessibility

If data is being stored then it also needs to be accessible. We would expect to find that data management is concerned with ways to retrieve and manipulate data, as well as to store it. For maximum benefit, organisations would like a wide range of users to share the same data (subject, of course, to any security limitations imposed by the data owners and any legal limitations on its use). Different users will have their own processing requirements and data access methods. By ensuring that data is held in systems that can share and integrate data it becomes possible for users to access data held in a wide range of systems without needing to know the details of each specific system. Holding data in such **interoperable systems** maximises the availability of the data. This permits an organisation's data to better support the operational requirements of all its users. Finally, data held for one purpose should also be accessible for other purposes (subject again, to access limitations that the data owners may impose); this requires that data be accessible in flexible ways, rather than being rigidly tied to a specific access mechanism or to limited manipulation. The data management systems should also allow for the organisation's data requirements to evolve over time.

Separation of concerns (physical and logical independence)

Data can be represented in many different ways and if we can translate between the representations it won't matter which representation we choose to use because we should be able to access and manipulate that data. If we had a universal language translator it wouldn't matter if someone told us the time in French, German or Arabic; our ability to translate would ensure we would know the time. In data terms there are three key representations that are important: the physical representation of where and how the data is stored; the logical representation of what data can be stored, its semantics and relationships with other data; and the external representation, which is how it appears to the users. Assuming it is possible to translate between the different representations allows people to understand and use the data in their preferred way. So, at the physical level a database technician would be able to optimise the physical storage of data independently of the logical description or the external description provided that they adjusted the stored description and the translation between the representations. By separating these concerns it becomes possible to build general purpose data collections that can be used by different users with different requirements and who use different representations, without the need to duplicate the data.

We will be revisiting these characteristics throughout this course.

1.10 Summary

In this section we have:

Highlighted the importance of the general field of data management and the role of the database system within that field.

Considered the historical development of data representations and data processing systems, and identified desirable properties of data management systems.

Defined some of the terminology that will be used throughout the course.

Identified the basic characteristics of the database approach and highlighted aspects of the relational data model and relational database system that will be further developed in the course.

Considered the general benefits to be gained from the use of a database management system, and some of the risks such an approach can bring.

Briefly examined some of the specialist database systems that have evolved from basic data processing capabilities to meet novel processing needs.

Considered how future developments in data management may progress.

LEARNING OUTCOMES

Having completed this section of the course material you should:

- ▶ Understand the general context for the study of database management systems, and recognise the associated concepts.
- ▶ Be able to use specific terminology correctly and be able to communicate about the general topics of databases, database management systems and data management.
- ▶ Be able to make a reasoned argument in favour of the use of a database management system when implementing applications requiring persistent, shared data, highlighting the benefits to be gained and the risks inherent in that approach.

2

Data management – why it matters

2.1 Introduction

In much of this course you will be learning about good relational database design and good implementation techniques and strategies. You won't be surprised to read here that we think that good database design and implementation are important. However, you should take some time to think about what exactly it is that a company, organisation or individual risks if the database development and data management are performed poorly.

At this point in the course it is not our aim to teach you how to design and implement a database. We will be highlighting the problems that can arise from incorrect or poor data management and database design. We will start by considering what a large organisation, such as a major credit card company or airline, requires from its corporate information management systems and examine issues directly related to data management. Corporate information management has a much wider scope than the topics we cover in our course; however, it is important to gain some understanding of the wider issues involved. If we don't understand the context within which we design and implement our databases it will be hard to evaluate good or bad designs or evaluate the effectiveness of an implemented database.

As businesses moved towards an information and service based culture, they soon realised that the data within their organisations had to be treated as a corporate asset together with other tangible assets such as cash, buildings, equipment and so forth. Like all other assets, data must be well managed to gain maximum benefit from its use, and like any other asset, it should be protected from damage or loss. Organisations that manage their data well can provide enhanced services to customers; they can, for example, reduce or eliminate reworking costs, improve internal work flows and have better information on which to base their decisions. This can give an organisation a competitive advantage and lead to improved customer satisfaction.

An organisation will want to achieve its business goals and will use the data it holds to achieve them. To be able to benefit from the information held, the underlying data must be classified, structured, validated, valued, secured, monitored and measured. This can be achieved only if an organisation manages its data – we typically use database technologies to achieve this.

Corporate information strategy

Database development within a large organisation is usually part of a wider **corporate information strategy** (CIS). The CIS is developed to ensure that the technology provision is meeting the company's needs and is appropriate for those needs.



EXERCISE 2.1

Why are both the phrases 'meeting the company's needs' and 'appropriate for those needs' used in the above statement?

A CIS can be broken into the following three main areas:

- ▶ **Information systems strategy (IS)** This is a strategy to exploit IT either to support business plans or to create new business opportunities. This strategy will integrate the information requirements and the long-term overall business goals. It will form a framework for determining what is required to achieve a business objective, what information and management resources will be needed, and where they will be deployed.
- ▶ **Information management strategy (IM)** This strategy determines how the IS strategy developments are to be implemented within the organisation. It includes planning for change, organisation and reorganisation of the corporate environment, imposing appropriate auditing, and structuring of the technology and its use. The IM strategy will be responsible for the management structures that will control those departments that use and manage the technology and applications.
- ▶ **Information technology strategy (IT)** This is a technology-focused strategy that determines how best to achieve the IS requirements within the IM structures. The IT strategy will form a framework to support the analysis and design of the technology infrastructure. This will include (but not be limited to) computing and storage purchase, data acquisition and management, application authoring (or purchase and customisation of standard software packages), management and maintenance, networking and security implementation.

The design and implementation of databases is usually considered to be part of the IT strategy, but clearly the databases also need management and control and they need to be integrated into the wider IM strategies. The scope of the CIS areas are summarised in Table 2.1.

	IS	IM	IT
Subject	What is required?	How is it embedded in the organisation?	How is the requirement to be met?
Basis	Customer focus, management and strategic planning support.	Internal organisation.	Processes and activities.
Orientation	Demand for applications and infrastructure to meet requirements.	Relationships between people and information.	Supply of technology and its organisation, interaction and auditing.
Focus	Business improvement.	Management mechanisms.	Technology – design, implementation and improvement.
Legal and ethical concerns	Ensuring that the business operates within current legislation and ethical policies.	Ensuring specific legal and ethical obligations are met during the management and processing of information.	Ensuring technology provision enables the audit and control levels required by legislation and company policy.

Table 2.1 The Corporate Information Strategy: IS, IM and IT roles and responsibilities



EXERCISE 2.2

Not all database development takes place in organisations large enough to have management structures that parallel the corporate information strategy that we have outlined. What benefits would be gained by developing such strategies in smaller organisations?

The British Standards Institution has published a guide called *Principles of Good Practice for Information Management* (BSI-DISC PD 0010). This document provides a practical framework to guide the development and operation of new methods and technologies for information management. Its advice is structured around five core principles that are expected to act as guidelines for establishing procedures and controls in the management of data:

- 1 Recognise and understand all the types of information being managed
- 2 Understand the legal issues and execute 'duty of care' responsibilities
- 3 Identify and specify business processes and procedures
- 4 Identify enabling technologies to support business processes and procedures
- 5 Monitor and audit business processes and procedures

The detailed framework of principles includes the expected application of other best practices – such as those defined by the *Code of Practice for Information Security Management* (British Standard BS7799).

Having considered the CIS we are now going to look in more detail at why data management is important, by considering the costs and value that are associated with a company's data and by looking at data quality issues. We will also briefly look at the legal controls that affect the acquisition and use of data. Later we consider the importance of having a clear data management strategy and the benefits that this can bring.

2.2 Data as an asset

It doesn't take much analysis to realise that data is important to organisations – customer details, employee records, invoices, orders, payments, stock levels and so on are necessary for businesses to function. For most companies the data provision must be cost-effective, that is, the value gained by having data available must outweigh the costs of collecting it and providing it to the users who can realise that value.

The cost of data

Costs of a data resource are usually tangible; we can identify a direct monetary value that can be placed on the activities associated with acquiring, managing and using a data resource. However, in a complex organisation that figure can often be hard to measure with any degree of accuracy. The least accurate figures are usually the cost to individuals in the time, effort and frustration that are a consequence of poor-quality or missing data.



EXERCISE 2.3

How could missing data give rise to costs in an organisation?

There is a general issue of the cost of the time spent capturing, entering, cleaning, communicating and accessing the data, which is usually overlooked in larger

organisations but can be a major overhead to volunteer organisations and small companies.

We can consider the data management costs under four broad headings: system costs, origination costs, communication costs and execution costs.

System costs: Any new or changed systems will involve development costs, which include consultancy fees, and the time and effort spent on the analysis of any existing systems or in requirements gathering. A company with a large computer installation may incur costs involved with constructing or renting a new building or finding office space within an existing work environment.

The computing equipment required to operate computerised data management systems needs to be accounted for in the overall cost of data provision. In many organisations this will include specialist **back-end systems** as well as network and **front-end systems**. In a small company it might just be the cost of a stand-alone PC. However, what are often overlooked are the operating costs for things like consumables, maintenance, power supply and the need for backup and recovery systems for emergency cover.

A system requires people to operate it. This might include specialists whose sole responsibilities lie with the system, but it will also include those who need retraining to work with the new or changed system. And, of course, staff may be made redundant as a result of a new system replacing existing working practices. There can be a lot of overheads involved in staffing an information system.

Origination costs: Origination relates to the initial acquisition of data and ensuring it can be used within the company information systems, that is, finding and importing the raw data and turning it into useful data.

Data can be generated as part of an existing activity within the organisation, such as data logging of machinery, or generating till receipts. Data can be captured, such as when a customer completes an application form, and it can be purchased from an outside source, which can include mailing lists and specialist data such as maps or technical specifications. In whatever way the data is acquired it will involve some cost to the organisation.

Having acquired this data it needs to be cleaned. **Data cleaning** includes the removal of inaccurate values, ensuring that the collected data is relevant to the organisation and ensuring that it has a common and appropriate representation. The cleaned data may need to be turned into something more useful to the end users of that data, that is, as meaningful information. So, there will be additional costs involved in further filtering, organising, analysing and summarising the data to improve its value to the organisation.

Communication costs: Good information is useless unless it is properly communicated to those who can make use of it. There are two specific areas of communication costs, relating to technology management and delivery. The cost of the technology management to ensure that the data is adequately stored, catalogued, secured, archived and made accessible is usually an up-front cost when the data is first added to a system. The delivery cost, which includes the actual accessibility costs related to the delivery mechanisms, is an on-going cost during the life of the data.

Execution costs: Finally, once the data system has been constructed, the data acquired and provision made for communication, there is the cost of the effort involved for the end users in utilising the data. If the data user requires complex processing requirements in order to use the data then this will require appropriate end user systems. For example, this can include end user training, employee costs and possible additional equipment and software costs. Execution costs can be reduced by ensuring that the user receives appropriate data that is suitably summarised and structured.

Some very large organisations, such as credit card companies, will transfer large archival data sets by road transport as this is cheaper and faster than utilising a computer network.



EXERCISE 2.4

A home user is considering buying a computer system to support the activities of a small sports club (accounts, membership lists, etc.).

- (a) Where would the origination costs arise for the data managed by this user?
- (b) The sports club supplies regular reports to the club committee and annual reports to the club members. What are the communication costs involved in this activity?

The value of data

Data has value to an organisation only if that data can be used to make decisions, to plan strategy and to take actions that result in a net gain to the organisation. Any valuation of data needs to include both the operational value (the ability to undertake business activities with the data available) and the strategic value (the value of decision making in the presence of data, be it accurate or inaccurate/incomplete data).

The major problem in putting a value on data use is that the data can often lead to intangible benefits. It is difficult to put a value on things like customer satisfaction, employee morale and the effectiveness of decision making that can be gained from having particular data available or acting on specific information derived from that data.

In some cases we can try to use crude estimates for the value of data. Consider a mail-order company that suddenly loses access to all its orders and payment records. It would be unable to function and would probably close very quickly. In this case an extreme figure for the value of this data would be the current value of the entire company! However, this isn't a particularly useful figure.

One approach to quantifying the value of the data is to evaluate the consequences of having only partial or inaccurate data available. If some of the data a company has is inaccurate then the company can continue to function. However, some of its business activities using that data will be incorrect. The cost of using the incorrect data can include direct costs, such as paying postage for letters to customers that do not exist, as well as indirect costs, such as employee time spent handling customer complaints. We could try to work out the value of data in terms of the additional costs that inaccurate data or partial data would cause; that is, we could quantify how many orders would be lost, how many order errors would occur, and so on. Such estimates can be complex to achieve and may be quite inaccurate, but there will be situations where they will be better than having no estimate at all.

Not being able to effectively measure the value gained by having and using data can give management a problem. A simplistic, and often incorrect, view is that increasing the use of the data will increase the value gained by having that data; this leads to development of policies that focus on maximum data sharing and data accessibility. In fact, in a later part of this section we will look at the issue of fitness for purpose and we will see that the highest value to be obtained from data use comes from using data that is fit for purpose. It is important that the right data is presented to the right users and this can mean imposing controls on the sharing and accessibility of data. Some data users would be swamped by the presentation of raw data when a summary would be more appropriate, which can lead to increased costs as the user needs to spend time finding the information they really require.

We can maximise the value of our data by ensuring that we collect relevant data, that we filter, organise, analyse and summarise data to satisfy the requirements of the data users, and that the data and information summaries are properly communicated in a meaningful form to those who can make use of them.

2.3 Data quality

Quality management involves establishing standards of quality for a product or service, establishing procedures or methods which ensure that these standards of quality are met, monitoring the actual quality, and taking control when quality falls below acceptable thresholds. Quality management can apply to many things; in this subsection we consider its application to data.

Data can be said to have high quality if it satisfies the requirements of its intended use, and it lacks quality if it fails to satisfy the requirements. In other words, the definition of the quality of a piece of data depends as much on the intended use for that data as it does on the data itself. Information systems professionals need to know that their data resources have 'fitness for purpose'. The move to centralised, shared, reusable data (i.e. data held in databases) makes the issue of fitness for purpose a key one. While current technology capabilities make the creation of shared access and **web services** simple, there is no point in creating these services if the information that is being shared is incorrect or inappropriate. In fact, you may create a liability for your organisation if you disseminate incorrect data.

To satisfy its intended use, data must encompass the following qualities.

- ▶ **Accuracy:** how well data within a database reflects what was supposed to be collected. This has two aspects: how accurate is the model of the world that the data represents and how accurate are the values of the data actually recorded?
- ▶ **Completeness:** having sufficient data values on which to base decisions. This relates to the purpose to which the data is being put to meet specific requirements. For example, if a trend in data values is being identified in a data set then is that data set big enough to show the trend accurately?
- ▶ **Timeliness:** whether the data is available for user needs within a reasonable time period. For example, last year's bank statement is not fit to determine my spending for the coming month.
- ▶ **Relevance:** the appropriateness of the data for the use to which it is being put. Has the right data been collected and prepared? Does the data sample represent an appropriate level of abstraction and summary? Is the data excessive in volume? Is the data communicated to the right person and appropriately represented in that communication?
- ▶ **Understandable:** the meaning and interpretation of the data must be clear to the users of the data. Data gatherers, modellers and users must have an agreed understanding of the data if it is to be used appropriately. The data must be accurately and meaningfully documented to allow a potential user to understand the data context and quality attributes and decide if it is fit for their purposes.
- ▶ **Trusted:** management of the data must include an audit trail of responsibility, ownership and management which can be used to verify the integrity of the data. Without adequate guarantees of security, privacy and ownership, the data users will not have confidence in it.

You will recognise many of these qualities from your study of Section 1.

Usability of data is the term commonly used to describe how easily the storage and documentation of data allows a potential user to make intelligent use of supplied data.

EXERCISE 2.5

In the following statements decide in what ways the data may be unfit for purpose according to the qualities discussed above:

- (a) My phone number is held incorrectly in a contact database.
- (b) A credit card bill is posted from the credit card company on the 8th of the month; the payment due date is the 10th of the same month.



- (c) An appointment system allows half-hourly appointments between 9 am and 5 pm, Monday to Friday.
- (d) After asking to see my medical records at my doctor's surgery I am given a copy of the medical notes for a person with the same surname who lives in the same part of town.
- (e) An investment bank, where your holdings cover three fund portfolios, each identified by a five-digit reference code and with different dates for the end of the distribution periods, sends you a cheque and a covering note that identifies the cheque as a distribution payment related to 'distribution period 25 of your holding'.
- (f) An online catalogue reports that a floor rug is 'large'.

The quality of data available to an organisation can be judged by the way in which it supports effective decision making and motivates action. Data has positive value if it results in a beneficial action taken as a result of a decision that uses the data. However, if the data is invalid or untrustworthy then it can have a negative impact on the value of the action – either the wrong action is performed or a required action is not taken.

The impact of poor data quality on the management of a typical business can affect most aspects of that business:

- ▶ Operational effect: lowering customer satisfaction, increasing costs, lowering employee satisfaction.
- ▶ Management effect: poor or invalid decision making, taking longer to make decisions, resulting in internal organisational mistrust.
- ▶ Strategic effect: creating difficulties in setting strategy, raising issues of data ownership and control, diverting management attention.

In small-scale and heavily departmentalised organisations the impact of poor data quality can often be overlooked as the data sources, data managers and data users are in close proximity. In such close working environments the weaknesses, 'work-arounds' and 'fixes' are easily communicated which can hide data quality issues. However, in large-scale, corporate-wide data collections and large data management and control systems the data sources can be anonymous and the data managers far removed from the data use and data users. In shared data environments there is an increasing risk that data users are making use of data not originally intended for that purpose. Recognising and maintaining data quality in such environments can become problematic. It is important therefore to know what data is being used, who is using it and how fit for purpose the users feel it is. If problems occur, the system can be refined and the data's fitness for purpose increased by amending and remodelling the data appropriately.

Data quality is dependent on appropriate acquisition and data cleaning checks and on a feedback loop from data users so that reports of poor-quality data lead to reviews of the data quality management processes.

2.4 Securing data assets

As soon as an organisation has an asset that has a recognisable cost and a value to the organisation then it becomes advisable to protect that asset, and any legal and ethical requirements for securing assets must be addressed. This applies to data too – it must be secured and protected.

All good security management starts by looking at the risks and threats that could compromise assets. For data assets these cover both physical and intellectual property risks and risks to the systems that manage the physical data.

The risks to a company's data assets include accidental or deliberate threats which might cause unauthorised modification, disclosure or destruction of data and loss or damage of the information systems caused by the degradation or non-availability of services. The consequences of a security breach can include the loss of integrity, loss of access, loss of confidentiality and loss of confidence in the accuracy of the data and the decisions made using that data.

The threats to data security come from many different areas which must all be considered for protection. However, the highest incidence of problems arises from simple human error. A mistake while typing during data entry, data processing or data deletion can lead to major problems with corrupt data. Users avoiding security procedures or reducing security in the interests of making their work easier can make data systems vulnerable. Also, the impact of poor user training and following incorrect data handling procedures can be significant. Other threats can come from technical error (hardware malfunction, software crash, power supply failures), natural disasters (flood, fire, explosion, lightning, earthquake), deliberate actions (fraud, commercial espionage), malicious damage (such as from unhappy employees or political and environmental activists) and industrial action (strikes, work-to-rules).

Security in an organisation is intended to minimise damage caused by a security breach. The security functions include prevention, detection, recovery and threat avoidance. While a lot of these security issues fall outside the scope of this course it is worth noting that things like user access controls, constraints, backup and recovery mechanisms are all tools that can be used to protect data assets.

A computer program or data file is one of a few assets that can be stolen yet still remain in a company's possession – copying a data file will leave the original intact and usable.

2.5 Legal and ethical issues in data management

The aim of this subsection is to provide an overview of the legal issues surrounding data management, rather than to legally define or interpret the application of the current legislation. Anyone who requires specific information covering the interpretation and application of the legislation should seek further information and advice from appropriate, qualified sources. Specific details on the regulation and enforcement of the UK Data Protection Act (1998) can be found at www.informationcommissioner.gov.uk.

This course cannot cover the detail of the various legal requirements for data management. We consider here the underpinning issues in the current Data Protection Act, and, briefly, the Freedom of Information Act, in the United Kingdom. This overview intends to highlight how these requirements can impact on the data management strategy of an organisation and to highlight why individuals who store and process data must also ensure good practice is followed in their data management processes.

Some sectors of industry, such as pharmaceuticals and financial services, and some common business functions within organisations, such as health and safety and employment practices, are heavily regulated (with regulations covering data management issues). This leads to enforceable legislation that is intended to encompass good practice. In addition, organisations and various industrial sectors are developing guidelines related to ethical practices and good conduct that, while not carrying the full weight of the law, can give an indication that an organisation is paying

due attention to sensitive data management issues. Ethical issues are often presented as part of the codes of conduct of professional bodies such as the British Computer Society (BCS) and the Association of Computing Machinery (ACM).

Legislation on data management covers four broad areas (often interlinked):

- ▶ The types of records that must be created for record keeping, audit trails, financial reporting and so on. These are often industry specific or functionally specific (such as employment or health and safety legislation).
- ▶ The types of records that cannot be created without oversight and registration with the appropriate government or controlling bodies. Generally these apply to the handling and management of personal data and data of a sensitive nature (such as security and health records).
- ▶ The purpose for which the data can be held. This ensures that data for specific functions, such as health and safety, is not used for inappropriate purposes.
- ▶ The length of time the data must be kept. This can work in two ways: to ensure the long-term availability of data, or to ensure that outdated data is not held. For example, some financial records must continue to be accessible for several years after the events to which they relate, but personal data should not be held for excessive periods.

In the UK there are two major pieces of legislation that have an impact on data management activities: the Data Protection Act (DPA) (1998) and the Freedom of Information Act (FOIA) (2000).

Timeliness

Note: the outline of the DPA and the FOIA given below was accurate at the time this course was published (2007). From time to time legislation does get amended and the guidelines from government do change. You should confirm that the DPA and FOIA descriptions given here are still current.

The DPA gives individuals certain rights regarding information held about them. It requires organisations and individuals who process personal information to register with the **Information Commissioner** details about the data and the purpose for which it is being processed. It places obligations on those who store and process information (known as the **data controllers**) while giving rights to those who are the subject of that data (known as **data subjects**). Personal information is considered to cover both facts and opinions about the data subject. The Act seeks to strike a balance between the rights of individuals to reasonable levels of **privacy**, security and control of their personal data and the occasionally contradictory interests of those with legitimate reasons for holding and/or using personal information.

Under the DPA all data controllers are required to comply with the eight enforceable principles of good information handling practice, which state that data should be:

- 1 fairly and lawfully processed;
- 2 processed for limited purposes;
- 3 adequate, relevant and not excessive for the purpose to which it is put;
- 4 accurate and up to date;
- 5 timely and not kept longer than necessary;
- 6 processed in accordance with the individual's rights;
- 7 held securely;
- 8 not transferred to countries outside the European Economic Area unless the country has adequate protection for the individual.

The DPA considers that personal information is being fairly processed if:

- 1 the individual has consented to the processing;
- 2 it is necessary for the performance of a contract with the individual;
- 3 it is required under legal obligation (e.g. health and safety);
- 4 it is necessary to protect the vital interests of the individual;
- 5 it is needed to carry out public functions (e.g. the administration of justice);
- 6 it is necessary to pursue the legitimate interests of the data controller or other third parties (unless it could unjustifiably prejudice the interests of the individual).

Specific provision is made under the DPA for processing sensitive information. Sensitive information is considered to include information on racial and ethnic origin, political opinions, religious or other beliefs, trade union membership, physical or mental health condition, sexual life, and criminal proceedings or convictions. Sensitive information has an additional set of conditions which must be considered to determine if it is appropriate to be stored and processed. It must be:

- 1 being stored and processed with the explicit consent of the individual and
- 2 required by law to support employment legislation, or
- 3 used to protect the vital interests of the individual or some other person, or
- 4 used in dealing with legal proceedings or the administration of justice.

A data subject is given certain rights by the DPA to ensure they can access the data held about them, to prevent processing and use for direct marketing if they so wish, and have rights to object to automated decision-making processes. In addition, if there exists data that is incorrect or data that is not appropriately held within the principles of the Act then a user can require rectification, blocking, erasure or destruction of the data.

The FOIA applies to any **public authority** or publicly-owned company and any data (except data explicitly excluded for reasons such as national security, etc.) held by that authority or company. It does not apply to private companies or individuals. The FOIA widens the DPA legislation on personal data to include all data held by a public authority. It extends the scope of personal data, defined in the DPA, that must be released when a data subject makes an access request.

As a consequence of the above Acts, when planning new systems it is important to follow these guidelines:

- ▶ Explicitly identify the types of data collected, the purposes for which it is processed and the criteria that legitimise the processing of that data.
- ▶ Clarify and document the roles of those involved in the processing, including the data subject, and consider their rights and obligations.
- ▶ Ensure that individuals are adequately informed why their personal data is being collected, the purpose for which it will be used and the types and reasons for third-party disclosure or transfer.
- ▶ Ensure that implicit or explicit consent, preferably explicit consent, has been obtained and recorded, and that if individuals withdraw consent this can be achieved quickly and effectively. Preferably ensure that an auditable record of that consent is maintained.
- ▶ Ensure that the system has adequate mechanisms to satisfy data access requests under the DPA (and the FOIA if the organisation is a public body), and that instructions to rectify, block, erase or destroy identified data can be acted upon to achieve the desired effect.

Further reading

If you are considering a career in database development and data management, or are already pursuing such a career, we would recommend that you find the government summary of the DPA and FOIA and at least read the overview to confirm that the above descriptions are still accurate. Both these Acts and other computer-related legislation can found on appropriate UK government websites (starting your web search at the home page for the Information Commissioner's office, www.ico.gov.uk, should yield some positive results).

The British Computer Society has a web page at www.bcs.org, and the Association of Computing Machinery can be found on the web at www.acm.org.

There will be no assessment of this additional reading; however, it is essential that you have an understanding of the legal obligations that data users must satisfy and the ethical issues that relate to data management.

2.6 What can happen when it goes wrong?

At the start of this section we looked at the need for appropriate data management which led us to consider the issues of data cost and value, data quality, data security and legal requirements. In the following material we will consider examples of some of the problems that can arise if adequate attention to detail has not been given to aspects of an information system. We will do this through a series of short activities based on a supermarket administration scenario.

To support our activities we will use a pair of fictitious supermarkets. Obviously the course team don't have access to the data and applications of a real supermarket so the examples will not be complete. The data and applications we use will be examples and fragments of the kinds of data sets and the type of applications that you would find in a supermarket administration system. We chose a supermarket because its business functions, and customer and product data will be understood by a wide audience. However, we haven't tried to achieve super-realism in our examples (for instance, we've had to be creative in the use of dates within the applications so that we can complete some activities in one day!). We want to highlight some key issues that can arise when an information system is used in practice. There is a potential problem in using incomplete examples: you may overlook what is being explored because of the contrived appearance of the example. However, all these activities will illustrate real issues that can arise in information and data management, sometimes on a daily basis.

We will consider applications and data sets for two instances of a supermarket. The first supermarket, the Notlaw Supermarket, will be used to underpin examples of what can happen in an information system if the underlying database is poorly designed or poorly implemented. The Walton Supermarket, on the other hand, will attempt to show that the problems identified can be avoided by careful design and implementation of the database.

In the activities we will be considering examples of what has been done correctly and incorrectly and the potential impact these actions might have. We will not be teaching about the techniques to achieve a good database design – these techniques will be considered in *Blocks 2, 3 and 4* of the course. We should also emphasise that the course team are aware that the examples used and the code fragments we show you may not be immediately understandable to everyone at this stage of the course. However, by the end of the course we expect you will be able to appreciate the database issues demonstrated by the Notlaw and Walton applications. Accordingly, in

the course folders we have supplied the underlying SQL scripts that query the databases and supply data to the Java applications.

At this point in the course we aim to give an overview of why the weaknesses appear. Where we draw your attention to specific code or database content, we will endeavour to explain what is being shown. In all cases the underlying problems we are highlighting will relate to data modelling, representation and storage or retrieval issues – the application code we show as using the data is not the cause of the weaknesses we examine. In fact, in most cases the Walton and Notlaw application code will be identical; what differs is the design and implementation of the database underpinning the applications.

The following activities will involve you executing applications and interacting with these applications. You may be asked to print out some results for comparison or to consider the data that the application presents. Just getting a result from the application isn't sufficient here; you will need to think about how and what the problems being highlighted might be, and think about how those problems might affect the cost or value that the supermarket realises from the data being considered, or how it may be failing to meet its legal or ethical responsibilities as a result of the problem.

To simplify access to the various activities the required functionality has been built into two application user interfaces: one for Walton and one for Notlaw. In the activities that follow when you are asked to execute an application, there should be a button visible on the application front-end window. Clicking on that button will execute the required application. The activities will describe what data you are required to enter and the sequence of interactions; some may be quite complex. There will be a close button at the foot of each application window which will shut down that application; you will need to close the application to switch to another application within the same supermarket.

Some activities do not involve executing applications; these will involve additional data or document files that can be found in the installed files from the course CD-ROM. You should look in your **M359** installation folder, within the **..\M359\Activities\Block 1** **walton** and **notlaw** subfolders to locate the named files.

The *Software Guide* includes instructions to allow you to check that the applications have been installed correctly on your computer. If you have followed the *Software Guide* installation procedures you should be ready to start the activities; otherwise go back to the *Software Guide* and follow the instructions for *Block 1*.

We would like you to attempt each activity, consider the questions raised by the activity and then go on to read the discussion points that follow. Remember there are *two* supermarkets, Walton (demonstrating good practice) and Notlaw (bad practice), and two application front-ends. You must be sure to follow the activity steps with the right application front-end otherwise the explanations and discussion points may not make much sense.

Note that your PC may permit you to use both Notlaw and Walton applications at the same time; this will depend on the available memory and processing power of your PC. If you can execute the two applications simultaneously then you will be able to do a side-by-side comparison of the activities.

You might want to discuss some of the questions raised in the activity discussions with other students in the course conference.

ACTIVITY 2.1

Messy mailing lists

Open the Notlaw Supermarket front-end application. You should see a **Mailing List Generation** application (along with other applications). Its purpose is to produce a list of names and addresses of supermarket customers using various different selection criteria. We've only implemented a few of the search criteria that a full application might offer.

Choose the **Mailing List Generation** application and use the **By Postcode** selection option at the top of the window. When prompted use the partial postcode **MK**. This should list customers with a postcode that starts with the letters **MK**, who should all live in or around Milton Keynes.



Examine the list of customer details that the application displays. Identify any problems with the data displayed.

Now examine the list that the application generates for the entire customer list (use the A11 option at the top of the mailing list application window).

What data quality problems can you identify that could have been avoided by improved data cleaning activities or better constraints defining valid data?

What data quality problems cannot be avoided with either data cleansing or better constraints?

In what ways will the poor quality of this data impact on the Notlaw Supermarket?

DISCUSSION

Clearly there are some invalid and meaningless data items in the Notlaw list of customers. Not all the MK postcodes have Milton Keynes listed as their town. Consequently there is some inconsistency between the two related data items, which could have been checked during data entry. When you looked at the A11 customers listing did you notice an additional Milton Keynes address? It was missed because an entry of 'unknown' has been stored in place of a postcode.

Notlaw appears to have a customer 'The one and only Mickey Mouse' who lives in Disneyland with a Milton Keynes postcode – which is highly unlikely. This was possibly entered by someone who needed to fill out an online registration form before using some service the supermarket website makes available.

William Shatner also appears to be a customer, which is a possibility as names are not unique, and there is a customer Michael Mouse, with a very suspicious address. However, we cannot just delete these entries without undertaking further checking as they might be valid customers – checking these customer records will be a more difficult task. I know a David Bailey whose graduation photograph appointment was cancelled because the photographer's receptionist thought it represented a joke booking.

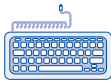
Now look at the Walton Supermarket **Mailing List Generation** application and do the same selections that you did for Notlaw; does the generated address list look any better? Are there any remaining issues in the Walton data? The data all looks reasonably sensible, except for the presence of Michael Mouse and William Shatner. In fact the name of Mrs Alberten has been misspelled, but unless you know the correct spelling there is no way to know that this mistake has been made. So, there are limits to how far data can be cleansed without hand checking every fact recorded in the database.

Basic data cleansing techniques are getting better at rejecting or correcting obvious errors; a combination of more complex algorithms and better access to validation data such as postcode checking, electoral rolls and so on. However, the data management team must apply those techniques routinely to protect their data assets. In the Notlaw case about 30% of the mailing list entries contain some form of error. In the Walton case there may still be erroneous entries (or even suspicious entries that might represent invalid data), but the obvious errors have been eliminated.

Recall that one of the benefits in using a database is data sharing. So within the supermarket the customer mailing data could be used to support order delivery, marketing mailings, debt recovery and so on. In fact, this list will be used every time customer address data is required. Accordingly, all the invalid, meaningless data is going to be shared across those applications. The cost in wasted postage alone will be high and there will be further costs associated with misdelivered orders, customer complaints about repetitive mailings, and so on.

Some online registration databases have been found to contain as high as 70% invalid entries, where people have registered to use a service, but have chosen not to enter their actual personal data.

ACTIVITY 2.2
Refund problems



In this activity you will be acting as a supermarket salesperson using the software to record and manage sales and refunds.

Find and run the Notlaw Till Management application using the Notlaw Supermarket front-end. You are going to record the sale of some milk.

- When the application opens give the date as some date last week. (In reality the date would be today's date and this date would be automatically available to the system – but to avoid spreading this activity over several days we've added the Enter Today's Date prompt to allow us to simulate the passage of time.) The application will supply the till number, operator number and receipt number as if you had logged on at a particular till.
- Using the till management application, record the purchase of at least one four-pint carton of milk (and anything else that you fancy, but be sure to include a four-pint milk carton!). Highlight the row for the item you wish to process the purchase of, enter a quantity and click on the Add Item button.
- Make a note of the till receipt number, the number of the line on which the milk appears on the receipt and the price of the milk (you'll need these later). When you have completed your purchases click on the Accept button.

Now run the Notlaw Price Management application. You are going to change the price of the milk in the supermarket.

- Give the date as yesterday.
- The four-pint milk cartons have the item code A10102; increase the price by 3 pence. Then close the price management application.

Run the Notlaw Refund Management application. You are going to give a refund on the milk you sold earlier.

- Give today's date.
- When prompted use the till receipt number and receipt line number you noted earlier.

How much profit would the customer have made in this situation? Can you do the same at the Walton Supermarket and so double the customer's profit? (Try the above actions with the Walton versions of the Till Management, Price Management and Refund Management applications.)

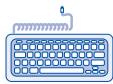
DISCUSSION.....

This is a problem related to poor data analysis and inadequate checking of the implemented database. The Notlaw Supermarket is basing its refunds on the current price of the item being refunded, not the price at the time the sale was made.

The Notlaw data analyst was told by a supermarket data manager that every item in the store has a single price (assuming no discounts, promotions, etc.). This is true, but only up to a point – every item in the store has a single price at any instant in time, but over a period of time the price may change and an item can have different prices at different times. The Notlaw database contains an association between the till receipt for the milk and the current price for that item – the current item price is accessed each time the till receipt needs to be reconstructed, in particular by the refund application. So whenever a refund is calculated the current price of the stock item is used for the refund, not the price at which the item was sold and recorded on the receipt.

The Walton analyst has spotted this potential problem and stored the price that each item was sold at with each till receipt record. At Walton, if the price of the item changes the till receipt still retains the actual selling price of the item at the time it was sold.

Confusion about the associations between data items and their details can be quite common. A combination of careful analysis and a good understanding of the data meaning and interpretation are vital to accurately underpin a company’s business applications.



ACTIVITY 2.3
Moving problems

A customer informs the supermarket of a change of address.

Using the Notlaw Customer Maintenance application:

- Select the data for Geraldine Taylor, by entering her customer number C12345.
- Change this customer’s address by updating the displayed data; you can choose her new address.
- Save this change to the database by clicking the Update Data button and then close the application.

Using the Notlaw Orders Pending application:

- Look for outstanding orders still to be delivered to Geraldine (there is only one).

What problem is evident in the data the applications display?

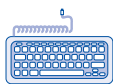
Is this problem present in the Walton Supermarket?

DISCUSSION.....

The delivery address (assumed to be the same as the customer’s address) has not been updated. This kind of problem is evidence that data sharing is *not* taking place between different application processes – these applications are using different, presumably duplicated, instances of the customer’s address. However, the individual applications, **Customer Maintenance** and **Orders Pending**, are not attempting to maintain consistency between the duplicated data held in different parts of the database. Consequently the data has become inconsistent and this will cause problems for the customer and the supermarket.

In contrast, the Walton Supermarket database developer has used a single instance of the customer address which is shared by both these applications.

The effects from this kind of poorly developed data sharing can turn up in some strange places. A tutor reported standing at Coventry railway station where the information display for two platforms can be seen from the same position on a platform. The two displays simultaneously reported that the same train was due to arrive at both platforms at the same time. This was almost certainly a consequence of the existence of duplicate data being inconsistently updated within the display system.



ACTIVITY 2.4
Confusion reigns

Let’s look at something that doesn’t involve executing applications. Suppose we take on the role of an application developer and we’ve been asked to develop a new application program that will use data from the supermarket database.

The application we are to develop will report the details of orders to suppliers that have not been completely fulfilled or that are overdue (i.e. orders that are somehow incomplete). We are in the early stages of our development and need to choose the data that is going to be useful to the application. We start by considering the descriptions of all the data held in the existing database.

Look at the document `notlaworderdatadescription.doc` in the `\Activities\Block 1\Notlaw\OtherDocs` subfolder within your M359 installation folder. This is supposed to be part of the descriptive documentation that supports the existing Notlaw database (it contains names and descriptions of data items that can be retrieved from the database – don't worry about exactly how we would do the retrieval).

Using this description identify those data items you would need to use to decide if an order was incomplete. What is your understanding of the meaning of the data that Notlaw has in its database?

Now look at the file `waltonorderdatadescription.doc` in the `\Activities\Block 1\Walton\OtherDocs` subfolder within your M359 installation folder – why is this easier to understand and make use of?

DISCUSSION

The database designer and implementer have a responsibility to make sure that the data is adequately described in supporting documentation so that it can be understood and used appropriately. Remember that two of the enduring issues in data management, considered in Section 1 of this block, are the meaning and accessibility of data. This includes having meaningful naming and appropriate descriptions for business and data management purposes, both in the database and in any related documentation.

Obviously we've used examples of extremely poor data descriptions for the Notlaw Supermarket. However, it is easy to fall into the trap of giving a loose description of the data because the current development team and end users understand what is being implemented. Often it is only when a new user of the data makes use of the documentation that weaknesses will be identified.

Documentation needs to be subject to the same level of review, quality inspection and validation as the rest of a database design and implementation.

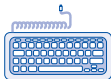
ACTIVITY 2.5
Using non-standard data representations

Notlaw Supermarket wants to sell books and has agreed an exclusive deal with a book wholesaler, Books-4-Booksellers. The wholesaler has ebusiness facilities and has supplied Notlaw with an online order form.

You are going to try to order some books from this wholesaler. Open the Notlaw Book Catalogue application.

- Examine the data held about the latest Airy Porter book (currently Volume 216). Make a note of the main details for this book.
- The Books-4-Booksellers electronic order form is accessible from the main Notlaw user interface (Books-4-Booksellers Order). Use it to order another six copies of this book (that is, if you can – don't spend too long puzzling over this one!). Use 1234 for the Customer Reference, and the supermarket name for the Customer Name.

Now try to follow the above instructions for the Walton Supermarket application.



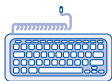
Incidentally, the original choice of the 10-digit ISBN has proved to be insufficient to cope with the volume of books being published; prompting the recent introduction of 13-digit ISBNs – so expect a mini-millennium bug type flurry of activity in the field of book databases.

DISCUSSION

The publishing industry long ago defined the standard reference number for a book edition: the International Standard Book Number (ISBN). The ISBN is used universally throughout the book world and is a unique number for each edition of every book. Notlaw, for whatever reason – or more accurately a lack of reasoning – is not recording this standard book number; instead it is using an internal reference code of its own construction. Consequently it will have problems accurately stating which books it wants when dealing with its suppliers.

This may seem like an extreme form of ignoring a well-known standard data representation in favour of an in-house solution. However, this is quite a common problem in some less obvious data sets. Do you happen to know the standard format for representing postal addresses (UPU S44) or international telephone numbers? Do you know the standard three-letter abbreviation for country names (ISO-3166), international airport names or calendar dates? If you want your data to be accessible to other users and to exchange data, for example in supply chain management environments, then the use of such standards might be important to you or your company.

If standard representations exist and are in common use, then developers need to build systems that either use these standards or store data in a way that can be communicated using the standard formats. As discussed in the introduction to this section, the first of the ‘Principles of Good Practice for Information Management’ is to recognise and understand all the types of information being managed. From a database implementer’s perspective this includes the need to know how the data underpinning that information is commonly represented in the relevant user communities.



ACTIVITY 2.6
Satisfying a Data Protection Act request

A customer who previously requested a copy of their data held by the supermarket has disputed the alleged non-payment of a bill. This customer says that on the occasion in question they did not make an order with Notlaw Supermarket so there should be no outstanding bill related to their account. The supermarket admits it has made a mistake by confusing this customer’s records with that of another customer. The supermarket has agreed to correct the fact that the customer is recorded as having a poor payment history.

Using the Notlaw Customer Maintenance application (you used it in Activity 2.3):

- Look at the entry for Gerald Tailor, who has the customer number C19342.
- You should see that the customer details show a Poor Payment flag on screen; remove that flag by changing the yes to no.
- Update the changed data in the database.
- Now, retrieve Gerald’s data a second time and confirm that the Poor Payment flag shows no.

Now execute the End-Of-Month Reconciliation application.

This application would do some automated reconciliation checks that the supermarket might make to maintain the internal consistency of data; it might also result in the payment of bills, sending of overdue reminders and other such activities. We’ve coded this so that it checks if there are any unpaid bills and sets the poor payment flag for those customers who have unpaid bills.

Using the Customer Maintenance application, check the state of the poor payment flag for Gerald Tailor. You should see that the poor payment flag has been reset by the reconciliation application.

Now use the Credit-Check application:

- Enter Gerald's customer number C19342 to bring up the details of any outstanding payments.

As you can see, changing the poor payment flag has not changed the data which shows that Gerald has an unpaid bill. In fact it should be inconsistent to have unpaid bills and a good payment record, but this is permitted by the Notlaw database. A change to the poor payment flag should require a change to the related data so that the reconciliation application cannot reset the poor payment flag at some later date.

Use the Walton Supermarket applications to make the above alterations to Gerald's data (he has the same customer number). Remember that the Walton data is separate from the Notlaw data, so Gerald's record hasn't been corrected yet. When you attempt to update the poor payment flag, the Walton application will behave differently compared to the Notlaw application. Follow the on-screen instructions to reallocate the disputed order to the correct customer Gerald Tailor C19324, and run the reconciliation application and check Gerald's customer records.

DISCUSSION

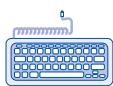
If the poor payment flag is simply reset (in the way that the Notlaw Supermarket permits) then there is discrepancy between the flag and the underlying payment records on which the flag's value depends. There should be a constraint imposed in the database that ensures this inconsistency cannot arise.

The Notlaw database doesn't take account of this dependency and has lost the association between the original data (the unpaid bill) and the summary report based on that data (the poor payment flag). Superficially, by resetting the poor payment flag the customer's data has been corrected – however, business processes that can lead to the flag being set could at some future time restore the poor payment flag, through either an automated or manual activity, because the disputed bill is still associated with Gerald's account.

The Walton application is prevented from simply resetting the payment flag as, according to the Walton business rules, it is inconsistent to have an unpaid bill and not have the poor payment flag set. The database designer has implemented a constraint to ensure this inconsistency is not permitted within the database. Consequently the company's business rule is enforced by the database not by an individual application. When the application attempts to change the poor payment flag the change is rejected by the database; the Walton application programmer has then offered a range of actions that would permit the flag to be reset. For simplicity, we've limited those options in this case, suggesting that the unpaid bill be identified (remember that the supermarket has agreed it was an error) and allocated to the correct customer (it should have been charged to customer C19324 Gerald Tailor). After the remedial action is taken it is possible to reset Gerald's poor payment flag.

Failing to identify dependencies between data items and protect that dependency with appropriate constraints can lead to the entry of data that is inconsistent, or can allow changes to data that should not be permitted. It is preferable to have the database enforce the constraints consistently for all the user applications rather than depend on the individual applications to be programmed to check the constraints.

We should stress at this point that it is the presence of the constraint in the Walton database that prevents the data being inconsistently updated, not anything in the Walton application code, which is identical for both the Walton and Notlaw applications.



ACTIVITY 2.7
Maintenance overheads

Run the Notlaw Customer Profiling application.

- When it asks for the date of the report use today’s date and look at the data shown.
- Look at the customer details listed.
- Now request the report for the same date two years ago by changing the report date.
- Compare the detail of a few customers shown on the two reports; some from today’s dated report and some from two years ago. (We’ve simplified the data by not having the actual customers change in this time, so the same customers will be in both reports.)

What is unusual about the data for each Notlaw customer on the dates you specified?

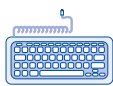
For comparison, look at the customer list produced by the Walton Customer Profiling application for a similar span of dates. (The customer details will be different from the Notlaw data.)

DISCUSSION

This is quite a common problem and relates to the maintenance needed to keep the data in a database accurate. The requirement for customer information in this database includes tracking the customer’s age, possibly because the supermarket specifically targets special offers for customers on an age basis.

The Notlaw developer has included the customer age in the database as a fixed value and appears to have forgotten that age changes each time the customer has a birthday! Even if you assume that the recorded age needs to be accurate only to within a year then every 1 January (or whatever day the supermarket decides is the collective customer birthday) the company must increase the age of each customer by one. This represents a messy and unnecessary maintenance overhead.

The Walton developer has taken a better approach; the base data for each customer includes their date of birth rather than their age – a value that never changes (except if there is a need to correct errors). When the profiling application requests the customer’s age it receives a value calculated using the date of birth value and the current date. This requires no additional annual maintenance to remain accurate.



ACTIVITY 2.8
Accessing archived records

Your task in this activity is to extract the contents of an archive file and print out the customer data.

- Locate the data file Notlaw_May_1_1977.nrc in the \Activities\Block 1 \Notlaw\OtherDocs folder of your M359 installation folder.
- This data file is an archival record of the content of the Notlaw customer database on 1 May 1977. Try to extract some meaningful content from this file.

Don’t spend too long trying to interpret the content of the file; you can use a simple text editor like Microsoft Notepad to open the file, then skip straight on to the discussion.

DISCUSSION

We hope you didn’t spend too much time trying to read the content! The DBMS (and even the hardware) for the 1977 system that created this file almost certainly does

not exist anymore (and is very unlikely to execute on a personal computer). In fact, even the description of the internal data format for this data file is very hard to find. In effect, this archived data set is lost and its content cannot be extracted.

Domesday ebook doomed

As part of the 900th anniversary of the Domesday Book, the BBC compiled a computerised collection of photographs, writings and other snapshots of life in 1986. The original book from 1086 is still readable while the digital version is no longer easily accessible. The software required to read the files is no longer readily available and the hardware capable of reading the files is breaking down due to its age. The University of Leeds has a project (CAMiLEON) designed to transfer the data into a more current, and hopefully forward-compatible, format before it is lost completely.

NASA data loss

NASA's early space records are suffering a similar fate. A researcher at the University of Southern California was unable to read magnetic tapes from the 1976 Viking landings on Mars. With the data in an unknown format, he had to track down printouts and hire students to retype data. 'All the programmers had died or left NASA,' the researcher said. 'It was hopeless to try to go back to the original tapes.'

Any organisation that needs to store and manipulate data for a long period of time must include archive maintenance among their data management activities.

Look in the folder titled **walton_may_1_1977** in the **\Activities\Block 1\walton\OtherDocs** folder of your M359 installation folder and at the titles of the files you find there. Again using a text editor like Microsoft Notepad, open the file **walton_may_1_1977.csv**. This is a file containing the contents of a Walton archive that has been stored in a very common and long-lived data format, the Comma Separated Values text file. No complex internal representation has been used, so the descriptions of that format can never be lost and the file is of a basic type that is highly unlikely to depend on specific software to read its content. Now look at **walton_1977_format_description.txt**. This file contains a description of the data held in the **.csv** file. In this way Walton has ensured that the original data is accessible and the meaning of that data is not lost.

Note that if you have Microsoft Excel installed on your PC the **.CSV** file may open directly into Excel when you double-click on the filename.

Additional documentation stored with these archive files can also indicate any other dependencies that might exist – for example, a CD-ROM disc has a physical life expectancy of only around 10 years and will soon be a minority format as DVD, blue-laser disk technologies and holographic storage become more popular. This is relevant to the documentation indicating how often the archive and its associated documentation need to be reviewed.

Activities afterthought

The above activities, and the associated software examples, have illustrated some of the more obvious problems that database and application developers may face. So the next time you see a news report about a 'computer error' causing a final demand notice to be sent for £0.00, you should think about the failings in the organisation's information strategy that could have led them to issuing the bill and the cost to the organisation involved in correcting their error.

2.7 Corporate data management strategy

Earlier in this section we saw some of the things that large organisations need to consider about their information strategy and the importance of data management. We then considered some of the practical pitfalls that can occur if we fail to develop and manage our data systems appropriately. To complete this section we will look at how to ensure that appropriate attention is paid to data management and how the development of a new database, and the management of any existing databases, is properly controlled. We will consider the development and application of data management strategies within the CIS.

Data management strategies should apply to data, metadata and documentation regardless of the media in which they are held.

Data management strategies should take account of the organisation's information strategy. Data management strategies cover a range of issues including:

- ▶ Development and implementation of a data management policy
- ▶ Arrangements for data ownership and stewardship
- ▶ Allocation of responsibilities for legal compliance
- ▶ Application of industry codes of practice (including keeping up to date with industry best practices)
- ▶ Management of the data life cycle from acquisition to destruction (or long-term archiving)
- ▶ Data documentation and metadata compilation
- ▶ Data quality assurance
- ▶ Data standardisation and harmonisation
- ▶ Data integrity and security
- ▶ Data access and dissemination arrangements
- ▶ Data storage, retention and archival preservation

A good data management strategy will allow the sharing and reuse of existing data and help to simplify the technology needed to support the business processes. It will ensure consistency in data management across an organisation and simplify the exchange of data between users. A good data management strategy properly implemented will go a long way to ensuring that the costs of the data are properly managed and that the company obtains maximum value from its data. It will ensure that data security and quality aspects are adequately addressed and legal and ethical concerns are not overlooked.

Large organisations often operate over very long periods of time. So, their data collections and applications can frequently span time periods longer than the personnel, technologies and data storage media originally used to build them. Embedding their data management strategy into the framework of a corporate information system attempts to ensure that the knowledge and awareness of the data is not overlooked when the organisation undergoes change or simply over the passage of time.

In whichever way an organisation chooses to control its data processing, it is important to avoid what has become known as the '**data island syndrome**'. The development of the desktop personal computer along with the spreadsheet and simple data processing programs resulted in department managers building their own data collections and data processing activities. These could be independent of the company's policies and outside the company's direct data management lines – creating isolated islands of data within the organisation. Data held in data islands is rarely shared, as it is being used for a specific limited purpose within the department that has built the data island, and in extreme cases is for the sole use of a single person within the organisation. If the company considers that there are benefits to allowing data islands then they must ensure that the data present on the islands does not get

overlooked in the context of the corporate data management strategy. The data islands themselves will need to be specifically managed and incorporated within the overall data management strategy.

The author saw an extreme version of the data island problem when working for a small accounting firm in the late 1980s. In the corner of a manager's office was a (pre-PC) home computer that still used an audio cassette player for data storage (it was around 8 years out of date at that time!). This computer was used to handle the employee records and payroll for a small printing firm – each week data updates were made from paper records forwarded from the printing firm (clock cards, new employment contracts, termination of contract notices, etc.). The system had been created by a former manager who wanted to see what could be achieved with the new home computers – he implemented the payroll contract in a hand-coded program. This system was never included in any management review of contracts or IT provision and it could not be integrated into the accounting firm's growing data network, so could not exchange data with any other office systems. Furthermore, there was no longer any expertise to amend or maintain the program – the original programmer had left the company two years previously. It wasn't even possible to copy the payroll data to a new audio cassette tape because the system had only one tape and insufficient main memory to store the entire content of the tape.

The accounting firm needed to decide how to transfer the existing system, applications and data sets into their office systems. Eventually a costly process of developing a new system, hand transferring and checking data and validating the new system kept the author busy for around four months.

EXERCISE 2.6

The introduction of the Data Protection Act has increased corporate management awareness of the extent to which data islands were present in their organisations. Why would it have this effect?



Within the CIS that we outlined earlier we must address corporate data management responsibilities directly. When the strategic business importance of data assets became recognised it was no longer considered acceptable to place the data management responsibilities solely within the IT area. It is now common practice to include corporate data management personnel in all aspects of the CIS decision making within an organisation. Three distinct roles of corporate data management within a CIS can be defined as follows.

- ▶ **Data management (within the IS strategy)** Corporate planning and analysis – preparing strategic and technical plans; identifying data sharing opportunities; defining policies for data retention; ensuring legal requirements are known and met; setting and enforcing operational procedures to support the business activities.
- ▶ **Data administration (within the IM strategy)** Developing and enforcing policies on data ownership and access control; planning for organisational data resource requirements; developing the organisational data model and data architecture with the aim of standardisation of the data resource across the organisation and ensuring its availability to users.
- ▶ **Database administration (within the IT strategy)** Responsible for the proper use of database technologies – logical and physical schema designs; implementation, tuning and capacity planning; metadata maintenance; evaluating and selecting hardware and software.

An organisation will gain many benefits from the employment of good data management practices – some examples are listed below.

Benefits to data suppliers:

Confidence and trust that supplied data will be used in accordance with agreements and confidentiality, copyright, non-disclosure and intellectual property rights (IPR) agreements, and other obligations.

Clear understanding of where and how the data will be used, protected and so on.

Benefits to the organisation:

Better quality, coherent data using standard definitions, references, formats, validation and so on.

Better control over the data through document procedures and practices related to data care.

Better knowledge of the data being held, its availability, interpretation and use, which leads to better use of metadata and improved indexing and cataloguing.

Improved business processes with use and reuse of data and the eventual standardisation of shared data sets within the organisation.

Confidence that statutory requirements are being met.

Control over access to the data (internal and external from defined policies on data release), disclosure controls, security.

Ability to manage clear and consistent data pricing and sharing policies for free access and commercial use customers.

Increased trust and confidence of the customer in the quality and reliability of the outputs.

Benefits to internal and external users:

Better awareness and understanding of the available data, both now and in the future, based on archiving and catalogue policies.

Improved access to data, safeguarded from disclosure of personal or other legally controlled data.

Better quality and timely information avoiding conflicting information.

Simplified data exchange and integration with other harmonised data.

Efficiency gains from the ease of use of high-quality documented data and metadata.



EXERCISE 2.7

We mentioned the NASA data loss during the discussion of Activity 2.8. What aspects of the data management strategy might have prevented the problems faced by the researcher?

2.8 Summary

In this section we have addressed the question of why it is important to develop strategies and policies for data and information management. In this course we will be looking at database theory and practice which forms only a small part of the information management that takes place in large organisations. It is, however, a vital role because virtually every business activity, every business decision and every business strategy is underpinned by data controlled by a database management system. We have seen how

the cost and value that data has to an organisation is dependent on several factors and that data quality is an important consideration.

By understanding the wider context in which databases will be embedded we are in a better position to know if the design and implementation are 'good'. This course won't be considering data management strategies in any more detail – this is a technically oriented course and these wider issues fall more properly into the scope of management. However, the theory and practice that we develop in the remainder of the course are intended to support whichever data management strategies and information system strategies are being employed by an organisation.

The **development life cycle** and the documentation and development artefacts that we discuss in Section 4 of this block will form the basis of the record keeping and quality assurance processes that the management activities will require. The decisions that we make during the development processes and the justification for the choices we make will also be open to scrutiny. In Section 4 we will present an overview of the activities required to develop and implement a database to meet user requirements, which will be refined in *Block 4*.

LEARNING OUTCOMES

Having completed this section of the course material you will:

- ▶ Understand the need for quality data to meet user requirements.
- ▶ Understand the importance of managing data in a manner that will support business requirements and satisfy legal obligations.
- ▶ Understand some of the problems that can arise in an improperly developed database or in an information system that is not adequately defined or supported.
- ▶ Be able to discuss the need for appropriate data management and information management strategies and the benefits these will bring.
- ▶ Be able to communicate about the general topics of databases, database management systems and data management using appropriate terminology.

3

Database management system concepts

3.1 Introduction

In the earlier sections of this block we covered the context and some history for the development of databases. We've also seen how important it is to get the right data to the right users and to control and maintain that data. We can now start to look at the more formal aspects of databases. In *Blocks 2, 3 and 4* of the course we will work through the underlying theory and practical application of relational techniques and the development and implementation of relational database systems.

It should be becoming clear that an information system can be very complex in terms of the number of users, their requirements and the amount of data managed. At the same time, the requirements of each individual user are likely to be limited, perhaps to just simple queries relating to a small part of the available data and accessed through **custom-built applications** that hide the underlying DBMS. In general, end users have neither interest in, nor any understanding of, the complete system and all its complexity. However, all users depend on many of the characteristics of the system that we consider the responsibility of the DBMS – these cover aspects of data access, and the administration and control of the system. So far in the course we have taken a high-level view of a DBMS – that it is a software system that manages all access to one or more databases.

In the following sections we look in more detail at the DBMS components and also the context in which the DBMS is found within the overall data system. The role of the DBMS in relation to the data and the users is shown diagrammatically in Figure 3.1.

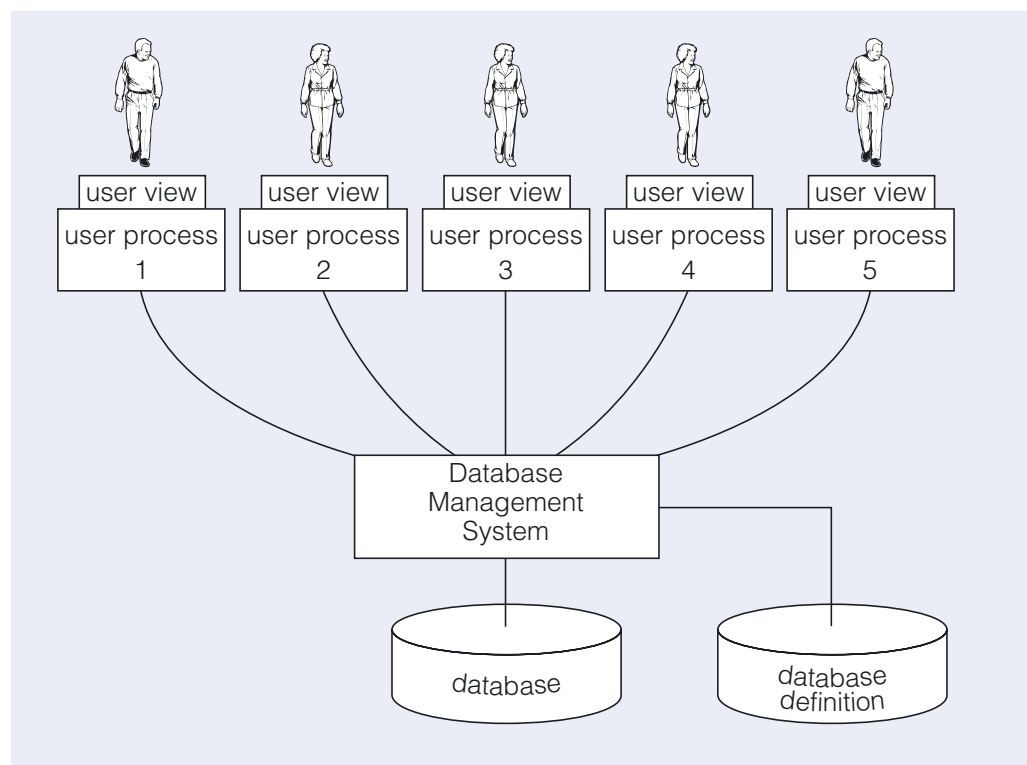


Figure 3.1 Model of a database system showing the position of the DBMS

Notice that the actual database content and the database definition are shown as being outside the DBMS. In fact they will reside on hardware directly managed by the DBMS host computer's operating system – however, we would not expect that the content of the databases could be accessed other than through the DBMS interfaces.

3.2 The three-schema architecture

Before we look at the detail of a DBMS's functionality we will consider a common abstraction that gives an architectural structure to the components of a typical DBMS. We've mentioned the ANSI/SPARC three-schema architecture earlier in the course so, as promised, we'll now take a more formal look at this architecture.

The ANSI/SPARC 1975 report proposed a standard set of terminology and a three-level architecture for database systems. The work in this area developed from earlier CODASYL Data Base Task Group work and from the work of various database user groups, most notably that of IBM. It never became a standard, and is rarely the basis for vendor implementations of database systems, but it does offer a useful mechanism for considering database functionality.

The architecture attempts to separate user views of the data from the system's physical representation of the data. If this can be achieved then the user and their applications can be independent of the physical data descriptions and storage.

The term **schema** is a commonly used term in the database context – it means a data definition that describes a database. There are various kinds of schema that we will discuss in the following section; however, in all cases they will refer to a specification of the properties of data in the associated database. Each schema may have a distinct description because of the different ways in which the data is represented to the system making use of that schema.

The motivation for this examination is one of the original aims of the database approach – to avoid data dependencies between user processes and their stored data. To avoid such data dependencies, a user process should not include the low-level definition of how data is stored in a file. Each user process will still need some description of the data it uses, but the definition of how data is stored in a file will be used only by the DBMS, rather than by any user process.

To satisfy these needs for definitions of data in a way which provides data independence, we introduce what is called the **three-schema architecture**. This is a general architecture applicable to any class of DBMS, not just relational DBMSs. A diagram of its components is given in Figure 3.2.

A **logical schema** is the central component in this architecture. It defines the logical properties of data in a database, being concerned with a representation of the data and associated constraints that are independent of how it is stored in files. A database has just one logical schema, used by a DBMS to manage all the data in the database. For a relational database, the SQL data definition language (DDL) can be used to define this logical representation of data.

A **storage schema** defines how a database is stored in files and accessed, say by use of **indexes** to help in searching particular columns of data. However, a storage schema can specify how data is to be stored only if the data is already defined by an associated logical schema. Thus, storage schema definitions must relate to the definitions in a logical schema, and the connection between the two schemas is known as a mapping, shown separately in Figure 3.2. Note that the mapping is

ANSI/SPARC refers to the American National Standards Institute/ Standards Planning and Requirements Committee.

CODASYL is an acronym for Conference on Data Systems Languages.

A storage schema is concerned with a physical representation of data on a storage device, so it is also known as a physical schema.

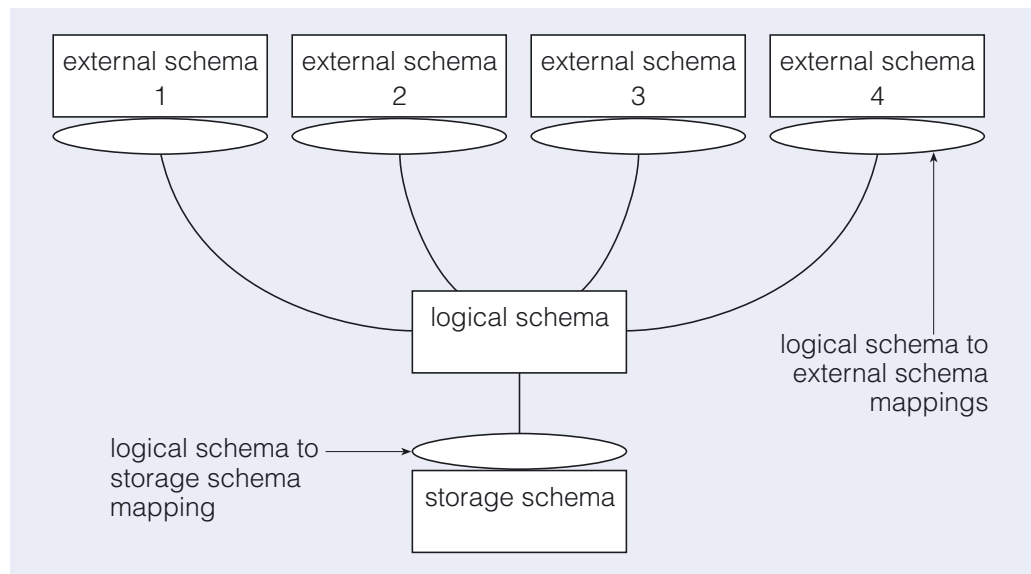


Figure 3.2 The three-schema architecture

associated with a storage schema rather than a logical schema – a logical schema can have meaning without the mapping but a storage schema cannot.

A database has just one storage schema, associated with its one logical schema. As a simple distinction, a logical schema determines *what* data is available in a database, while a storage schema determines *where* and *how* data is stored in files.

An **external schema** defines data for a user process or, more specifically, defines data from a database, which the user process retrieves or updates by interaction with a DBMS. Thus an external schema is concerned with data as it is presented outside the DBMS to user processes.

Again, an external schema can specify how data is available to a user process only if that data has already been defined by a logical schema. External schema definitions must relate to those in a logical schema, and each external schema must have a mapping from its associated logical schema, as shown in Figure 3.2. For a relational DBMS, an external schema is defined as tables which contain the data required by a user process. The external schema tables may differ from the tables in a logical schema by including only some of the columns and some of the rows of the logical table or by including data derived from the data described by the logical schema. The external schema is independent of the storage schema as it is defined by reference to the logical schema alone.

While Figure 3.2 happens to show four external schemas, they are intended to be representative of any number of external schemas, corresponding to the many user processes that may exist in a database system. To further place Figure 3.2 in context, we can relate the different kinds of schema to the components shown in Figure 3.1. External schemas are associated with both a user process and the DBMS, while both logical and storage schemas are associated with just the DBMS. Also, what we previously referred to as a database schema should now be considered as encompassing all three kinds of schema. However, you should distinguish the external schema for a user process from the user view as displayed on a screen, which is likely to involve the same data, but the data will be represented in a way that is determined by the user process not by the DBMS.

EXERCISE 3.1



- (a) Explain why an external schema should be considered as defining a logical representation of data.
 - (b) Give one property of data defined as part of a logical schema that is not included in an external schema.
-

Let us now consider the question of how this three-schema architecture provides data independence. Remember that data dependence arose in file-based systems whenever a change to one part of a system had consequential impact on other parts. Data independence is concerned with controlling the impact of database change on user processes.

Logical data independence exists when a change to a logical schema has no impact on a user process. This is possible because each user process is dependent on an external schema and not directly on a logical schema. A change to a logical schema does not affect the data defined by an external schema if either the latter does not include the changed data or the change can be taken into account by a change to the mapping. Thus, many changes to a logical schema do not require the data defined by an external schema to change, so that user processes do not have to change. However, it is not possible to guarantee logical data independence in all situations because it is easy to imagine drastic changes to a logical schema, such as deleting a complete table (its definition and all its data), which must have an impact on some user processes. Logical data independence is important in enabling a database system to meet new requirements or extending the range of data in a database.

Physical data independence exists when a change to a storage schema has no impact on a user process. This is possible because such a change does not affect a logical schema, since the change can be taken into account by a change to the mapping for the storage schema.

Physical data independence is important in enabling a database system to allow for increases in the amount of data being stored and improvements in the efficiency of its processing. Thus, the characteristics of files can be changed or indexes to data can be added without affecting how user processes work.

EXERCISE 3.2



When defining an external schema, you can choose which columns of a table in a logical schema are to be included. Describe how this provides data independence when a column is added to an existing table described by the logical schema in a database.

The three-schema architecture we have described is a general model, developed before relational DBMSs were in widespread use. Accordingly, we need to assess the extent to which the concepts of this schema architecture are supported in practice, specifically for DBMSs using SQL.

To begin with, many relational DBMSs have a fixed way of storing tables and so a storage schema is not required – this is a perceived weakness of such implementations of relational technology. However, most DBMSs permit indexes to be created and added to table descriptions when required; just like a book index, an index in a DBMS can be used to speed searches for indexed values. Indexes are not directly concerned with how data is stored, only with how it is accessed. In effect an index adds an extra description of the way in which data content is stored, but does not affect the way in which the data is physically stored. As a result, indexes are sometimes considered not to be part of a storage schema. On the other hand, they are definitely not part of a

logical schema, not being concerned with any logical property of the data itself. Consequently, indexes do not appear to fit anywhere in the three-schema architecture.

A DBMS without a storage schema may appear to keep definitions simple, but as a consequence the choice of tables and columns in defining a logical schema has to take account of requirements for efficient processing, which makes the design of the logical schema more difficult. We shall return to issues related to efficient design in *Block 4*.

The storage schema and large-scale database products

Some large-scale commercial relational database products offer a limited choice of different internal file structures to permit optimisation of the data storage and data processing. Different data structures used for the physical file storage have different strengths and weaknesses, depending on the patterns of data use in the files – for example, large files, with little updating and frequent searches, have different requirements from small but frequently updated files. Since this course does not consider the internal data structures of file systems we won't pursue this further.

Let us now consider a logical schema and its relationship with external schemas. For a relational database, a logical schema is concerned with tables whose data is stored in a database, which are also known as **base tables**. We can thus be clear as to what constitutes a logical schema for a relational database – the collection of the descriptions of all the properties of the base tables.

In the relational model you can also have a derived table, known as a **view**. A view is a table without stored data whose values are defined by evaluating a defining query involving one or more base tables or other views. The defining query is evaluated when the data for the view is required. In the context of the three-schema architecture, the view could be seen as part of an external schema because it is describing data to be used by a user process (the view contains a description in the form of a defining query). The mapping for the view from the logical schema is provided by the query statements in the view's definition, which specifies how data for the view is to be derived from other tables.

While a view may be considered as part of an external schema, a user process is not limited to using views – a user process can also access base tables directly. Thus SQL data definitions do not require you to define a proper external schema for user processes, but they do allow you to define views that are mappings from the base tables in the logical schema. We shall examine the detailed properties of views in later blocks and show that they have capabilities for providing limited data independence.

Despite the fact that few products clearly distinguish between the various schema definitions, the three-schema architecture remains a valuable model of the database and its context. It allows for the separation of concerns related to the data requirements and definition, storage efficiency and user process requirements. We return to this separation of concerns later in this block.

3.3 The functions of a DBMS

A DBMS is a complex software system with many components each responsible for different aspects of data and user management. These components have specific functions, many of which we have already considered in general terms (see Figure 1.5). We can now examine the role of a DBMS (whether or not it is relational) more carefully. We are interested in all the tasks associated with both accessing and managing database content and the database definition (schema).

We are using the term 'function' in the sense of what a DBMS is capable of doing.

We will consider the functionality of the DBMS by examining four main management task areas: data definition and manipulation, controlled sharing, security management and the management of requirements changes. These tasks involve a number of functions that could be expected of any class of DBMS, not just relational ones, but the examples we use will relate to a relational DBMS.

Data definition and manipulation

A DBMS must offer some mechanism for the definition of the database and the storage and retrieval of data.

Data definition

This DBMS function is concerned with the definition of properties of data and, more generally, with the creation and modification of a schema in which **data definitions** are kept. We describe this function first because it has to be used first – it is not possible to have any data in a database until it has been defined in the associated schema. A DBMS performs this function in response to a request from a user process that gives users the capability to define data. Such a request includes a specification of some properties of data that a DBMS then incorporates within a schema. For example, for a relational DBMS, these properties include the name and data type of each column for all tables (and other properties that we shall consider in later blocks).

Typically the logical and storage schemas are combined into a single schema holding the database definition.

Note that this function includes modification of a schema, which is not a capability of all classes of DBMS but is an important capability of a relational DBMS. For example, this allows new tables to be added at any time, or existing tables to be altered by adding new columns.

Data manipulation

This DBMS function provides the means to manipulate specific data in a database, which includes retrieving and updating data – whether this involves adding, deleting or changing data. A DBMS performs this function in response to a request from a user process, which provides a specification of what data is to be retrieved or updated.

The term **data manipulation** is used specifically because this function does actually involve *manipulation* of the data: it is not just a matter of passing rows of data to and from between a user process and a database. As an example of one kind of manipulation, derived data such as counts, totals and averages can be calculated by a DBMS for display by a user process.

Controlled sharing

Single-user versus multi-user information systems

In this course we generally assume that an information system has many users. This assumption does not exclude the possibility that a particular information system may be used by just one person. Single-user information systems have essentially the same characteristics as more general multi-user systems. For example, even in the case of a single user there can be a number of requirements, because of the varying needs of that one person at different times. Consequently, we will not consider any special characteristics that only apply to single-user systems. We will, however, use examples of single-user systems, those that we provide on your computer, to demonstrate characteristics of more general multi-user systems.

Note that we also consider access control as related to controlled sharing; access control is discussed below within the subsection on security management.

There is one particular way in which **multi-user systems** are directly comparable with **single-user systems**, and that is when considering how a user interacts with an information system. The important point is that the users do not interact with each other,

but only with the information system. That is, each user of an information system works as if the system is a single-user system, with no interference from other users. In order to achieve this, the DBMS must control the way the various simultaneous activities in the multi-user system can interact.



EXERCISE 3.3

If you were one of the users of a multi-user information system, explain how you might become aware of other users updating the database.

Transaction support

A DBMS performs the function of defining a **transaction** in response to a request that specifies a sequence of updates to be executed as a single unit of work (that is, not as individual updates), which will succeed or fail as a complete unit.

Two standard SQL commands **COMMIT** and **ROLLBACK** are available and can be used to mark the end of one transaction and the start of the next. A transaction is started when a user process initially connects to a database. Any changes that the user process makes to the database content are considered to be within the current transaction. The current transaction can be committed, that is, all the data changes within it are made permanent, by issuing a **COMMIT** instruction at the end of the transaction. Alternatively, the changes can be discarded by issuing a **ROLLBACK** instruction, which has the effect of taking the database back to the state it was in before that transaction began.

The classic example of a database transaction quoted in most textbooks is the transfer of funds from one bank account to another. The transaction consists of two steps – the first step removes the money from the source bank account; the second step places the money into the destination account. If the transaction did not complete fully – if, for example, only the first step was executed – then the money would seem to have disappeared; it is in neither the first nor the second account. The transaction either succeeds or fails completely such that both bank accounts remain in a valid state.

A DBMS automatically prevents a situation in which software or hardware failure causes only some of the updates of a transaction to be recorded in a database. If there is any reason why a transaction cannot be completed, the effect of any updates that have been carried out must be removed so that it is as if the transaction had not even started.

Transaction support includes the functionality both to define a transaction and to ensure that its properties, all-or-nothing completeness for example, are enforced.



EXERCISE 3.4

What are the two possible outcomes if a bank transfer is attempted as a transaction in a DBMS that properly implements transaction support?

A transaction is used to control sharing of data because the result of a transaction execution is visible to other users of the system only at the end of the transaction (whether it succeeds or fails). In the example of the fund transfer above, if a user could view both accounts half-way through the transaction, when the money had been taken from one account but not yet placed in the other, it would appear as though the money had been lost. This is impossible when the DBMS properly implements transaction support.

Concurrency support

Remember that we have described how each user should be able to access a database independent of other users. **Concurrency** needs to provide users with the perception that they can access any data value whenever they want, without being affected by other users who want to access the same data at the same time. If all users just want to retrieve that value, there is no problem – they all see the same value. For updating, there are potential problems when many users want to change the same data. We shall examine this in detail later in the course. Concurrency control enables many user processes to access a database at the same time in a way that prevents them interfering with each other.

EXERCISE 3.5

Explain why there is a need to support transactions even when there is no concurrency – that is, there is only one user process on a single-user computer accessing the database.



Security management

A key responsibility for the DBMS is ensuring that the data is both available and reliable. Security management covers a range of key functions including ensuring the integrity of the data and the correctness of the processing of the data held in the database, ensuring that only authorised users access the data and ensuring that it is not possible to lose data when a system crashes.

Note that sometimes security is used to refer only to the management and control of users.

Constraint management

This DBMS function is concerned with the definition of constraints and, since constraints are properties of data, they are included in a schema in the same way as described for data definition. Indeed, constraint definition could be considered as just one aspect of data definition, but we have described it separately to emphasise the importance of constraints as a special characteristic of data in a database.

After a constraint has been defined, a DBMS must ensure that data in a database never violates the constraint – that is, a constraint is enforced automatically. Constraints are checked whenever a request to the DBMS would result in changes to the data that might affect the constraint. Note that many DBMSs allow constraint enforcement to be switched off either temporarily or permanently – this control should be exercised with great caution.

Access control

Another security management concern is controlling what users can do and the data they can access. This reflects the need for privacy of personal information, such as a patient's medical records, and preventing disclosure of confidential data, such as an organisation's financial details, as well as restricting updating to those users who are explicitly authorised to do it.

Access control involves both the definition of users' rights to access specific data and the prevention of access to users without rights. A DBMS performs the function of defining users' rights in response to a request that specifies in detail how data may be accessed by users. A DBMS automatically controls access according to defined rights. For example, for a specific table, a right could be defined for a particular user to retrieve data; that user would then be able to examine rows of that table but nothing else (unless further rights were granted). We'll see how access control rights are managed in an SQL database when we reach Section 6 of *Block 3*.

The rights to access data are included in a schema as part of the definition of a database. Of course, the DBMS must include mechanisms to create usernames that the access control mechanism can use. We'll look at an example of how this is done when we look briefly at creating a database using a Sybase Central 'wizard' in Activity 3.2 later in this block.

Backup and Recovery

If a hardware or software failure occurs, there needs to be a way of reconstructing the database so that it is the same as it was just before the failure. This is known as the **recovery** of a database. Note that recovery is more than just using a **backup copy** of data from the night before (which, of course, you will all be doing with your TMA documents!); all data up to the time of the failure has to be recovered. For example, a bank will want to recover the effect of every transaction right up until the moment of the system failure. (An exception to this is identified in the following exercise.)



EXERCISE 3.6

We have described a transaction as a series of changes to a database that must be completed all together. What data should be recovered if only part of a transaction has been completed when a failure occurs?

An implicit part of this function is the continuous **logging** of all changes to a database as they happen, providing a copy of the updated data so that it can be used by the recovery process if needed. Logging can often be switched on and off, so in some situations a database administrator can decide that logging all changes is not required. However, any updates since the time of the last backup would be lost if logging had been disabled.

Change management

In this context change management refers to changes in the requirements that an information system is established to support – that is, the information needs of its users. We are *not* referring to changes in the data content of a database because such changes result from the data manipulation activities you would normally expect to take place.

Change must be *expected* in the environments within which information systems are used. Changes may arise either from external influences imposed on an organisation, for example, as a consequence of continuing changes to government legislation, with new laws, taxes and so on, or from internal organisation choice, such as improving the way a business operates or the amount of data being recorded about the business activities.

Restructuring

Restructuring of the database occurs to meet changes in requirements. For example, if the definition of a table is changed to have an additional column, the storage of existing rows of that table must also be changed to allow for the new column. This is performed by the restructure function of the DBMS, with the new database structure being based on the original. Restructuring will involve changes to the logical schema and so will lead to changes in the storage schema, and may involve changes to the external schemas to make parts of the restructured database available to user processes.

Reorganisation

Reorganisation applies to the storage schema and involves changes to the way in which the database is organised in files. There are two main reasons for performing reorganisation. The first is operational maintenance – if the performance of the database, and the user processes using the database, is important and falls below some acceptable standard, then adjustments to the storage schema are applied to ensure that performance is maintained at an acceptable level. The second reason is for porting and implementation maintenance, in which the DBMS or the underlying computer system undergoes changes that require the storage implementation to be revised.

Porting involves moving a computer system from one hardware and/or software environment to another.



EXERCISE 3.7

- Describe the different kinds of definitions in a schema.
- Identify those DBMS functions that a user process could explicitly request (if not constrained by any access control rights).
- Identify those DBMS functions that are performed automatically without being requested.

3.4 Inside the DBMS

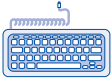
In an ideal DBMS the users, designers and application developers should not need to be aware of the internal components of the system; they should be concerned only with the functionality that the entire system offers. However, knowledgeable users, for example those tasked with achieving high performance or specific security requirements, will need to know how to control the individual components. There is usually a variety of tools available to database administrators (DBAs) to enable them to interact effectively with the DBMS internal systems. In the Sybase environment that you have access to, the Sybase Central application is the framework in which these administrative tools have been collected. If you execute the Sybase Central application, and examine the various tools and options given, you should see that it is possible to access most functions of the DBMS using this interface.

We will say more about the DBA in Subsection 3.6. For now you should note that there is usually a default username defined within a DBMS that has administration privileges – in Sybase Central this username is **dba**.

Internal DBMS data – viewing the system schema

In the description of the three-schema architecture there was a lot of mention of database descriptions, mapping descriptions, physical storage descriptions and so on, all of which require the DBMS to store internal descriptions related to its operation. Not surprisingly, you will find that inside the DBMS there are a number of database tables that store data related to the DBMS and the databases, users, security requirements and so on. These are referred to collectively as the **DBMS system schema** and commonly as the **system tables**. These tables contain information relating to the specification of the data managed by the DBMS. They are distinguished from the **user tables** which are the collection of tables storing data for the users' databases. Every user database created within the DBMS will have its description stored in the DBMS schema in various system tables.

In the following activities we will see examples of the system tables using the standard user interfaces to the DBMS – we can even execute queries to retrieve data from these tables.



Note that in the sample databases we have supplied for the course we have created some usernames to enable you to get access to the database. Each activity will usually tell you which username to connect to the database with - if it doesn't, then you should assume that the username is **m359** and the password is **m359**.

ACTIVITY 3.1

Examining the DBMS system tables

Locate and execute the Sybase Central database tool as described in the *Software Guide*.

Now connect to the Walton database using the Connect... option in the Tools menu, or using the keyboard shortcut F11, or the connect button on the toolbar. You should complete the connection prompt window using the User ID **m359** and the Password **m359** in the upper part of the identification page of the connect window. In the lower half of the window highlight the ODBC Data Source name button and choose the name **waltonDSN** from the drop-down list. Then click on the OK button; this should show the Sybase Central application with the folders window on the left-hand side and a right-hand window with a tabbed display. The exact content of the windows will depend on the state of the database but it should be similar to the example shown in the *Software Guide*.

In the left-hand Folders window, expand the folder Adaptive Server Anywhere 9 \M359 Server\waltonDB-M359\Tables. If any part of this folder hierarchy is not visible click on the + symbol to expand the next level of the hierarchy. The left-hand window should now show a list of the names of the tables (and the user who owns them), such as **book** (which is owned by **m359**).

In the right-hand window of the tool the same list of tables should be visible. In the right-hand window double-click on the name of a table (for example, **book**). The right-hand window should change to show a tabbed window with the tabs labelled Columns, Foreign Keys, Referencing Tables, Unique Constraints. Check Constraints, Indexes, Triggers and Data. The window usual opens showing the Columns display which shows details of the definition of all the columns of the table. Clicking on a named tab will change the display to show information related to that aspect of the table. Note that many of the displays will show no information as **book** is quite a simple table – however if you look at the Check Constraints display you should see a list of the columns of the **book** table that have a constraint on them and the definition of that constraint. You can also see the actual data content for a table by choosing the Data window tab.

Examine these different displays – but avoid editing anything – these are the descriptions of the various things the DBMS is managing in order to describe and manage the **book** table and its data.

Now in the left-hand Folders window click on the **waltonDB-M359** folder. The right-hand window should now show a tabbed window with tabs labelled Contents, Connected Users, Table Page Usage, Table Locks and Profile. The information displayed in each of these windows is used by the DBMS for the management of the Walton database.

Clearly the management tool (in this case Sybase Central) is able to get access to the schema descriptions and management information that the DBMS is using – but where is this data actually being held? As we identified above, the system tables contain the internal descriptions for the schema and management information.

Sybase Central can give you access to the system tables. Go back to the left-hand Folders window and right-click on the **waltonDB-DBA** folder then choose Filter Objects by Owner... from the drop-down menu and tick the **SYS** box before clicking on OK. If you now open the Tables folder in the Folders window you will see a large number of tables some of which have names begin with **SYS** and which have the owner **SYS** – these are the system tables. Not surprisingly, the structure of the system tables is quite complex and the data they contain is hard to interpret – but you should note that these are tables just like the user tables in the Walton database (tables such as **book**) and you can see that the tabbed display has the same label for a system table, such as **SYSTABLE**, as it does for a user table, such as **book**.

Since the system tables are SQL tables stored in the DBMS it is possible to manipulate them using SQL queries. Users rarely need to interact directly with the system tables of the DBMS, and to help maintain the database integrity, the system tables usually have very restricted access rights. However, it is possible to show the content of these tables using simple SQL statements.

You should go back to the `WaltonDB-M359` folder and untick the `SYS` box in the `Filter Objects by Owner...` menu item so that the system files are no longer shown. You can close Sybase Central at this point.

Now, locate and execute the ISQL direct entry tool as described in the *Software Guide*.

Now connect to the Walton database using the `Connect...` option in the `SQL` menu, or using the keyboard shortcut `F11`. You should complete the connection prompt window using the User ID `m359` and the Password `m359` in the upper part of the identification page of the connect window. In the lower half of the window highlight the `ODBC Data Source name` button and choose the name `WaltonDSN` from the drop-down list. Then click on the `OK` button, which will return you to the ISQL application with the `Results` window showing a connected message or the window name will change to show the database, username and database server for the connection (in this case `WaltonDB (m359)` on `M359 Server`).

In the `SQL Statements` window type the following SQL query:

```
SELECT table_id, table_name
FROM SYSTABLE;
```

Now use the execute menu button (or the `Execute` menu item on the `SQL` menu or the `F5` function key if you are using keyboard shortcuts) to execute the query, and the result of evaluating the query will be shown in the `Results` window of the tool.

The internal identifier and the names of all the tables in the Walton database will be retrieved from the `SYSTABLE` system table and displayed in the `Results` window. There will be a lot of internal table names in the results window, but somewhere you will see that the user table names, such as `book`, are listed as well as the system tables.

You can close the ISQL tool now.

Incidentally, many of the common management activities have ‘wizards’ available through Sybase Central, although we won’t be using these wizards directly in the course.

ACTIVITY 3.2

Some wizards within Sybase Central

Locate and execute the Sybase Central database tool as described in the *Software Guide*.

Now connect to the Walton database using the `Connect...` option in the `Tools` menu, or using the keyboard shortcut `F11`, or the connect button on the toolbar. You should complete the connection prompt window using the User ID `dba` and the Password `lqs` in the upper part of the identification page of the connect window. In the lower half of the window highlight the `ODBC Data Source name` button and choose the name `WaltonDSN` from the drop-down list.

The left-hand `Folders` window should display the `Sybase Central\Adaptive Server Anywhere 9` folder hierarchy. (If it doesn’t, use the `+/-` markers to move to this level of the hierarchy.) The right-hand window should display a tabbed window with tabs labelled `Servers`, `Utilities` and `Services`. If you choose the `Utilities` tabbed display you will see a list of management utilities and other tools. (Alternatively, use the `Tools` menu item `Adaptive Server Anywhere 9`, or the keyboard shortcut `Alt-T`, `Alt-A`.)



The user **dba** has full administrator rights to the DBMS, including the ability to create a new database. However, you should avoid too much experimentation with the wizards and options that you can see when connected to the database as this user – some of them are quite powerful.

If you click on any of the utilities you will be shown an opening page describing what the wizard does and a series of buttons (including the all-important Cancel button) that will step you through the wizard steps. As an example we will create a new (empty) database.

In the Utilities list double-click on the Create Database item, or use the Tools menu, or Alt-T, Alt-C, Alt-A. Read the opening page information and then click on the Next > button. (We will mainly be using the default options for creating a new database, but we will explain what these options are doing.)

The default option button of Create a database on this computer should be highlighted; if it isn't, click on it to highlight this choice, then click on Next >.

We need to give the database a name and place in the folders on the local computer. Use Browse to go to the ..\M359\Databases folder on your PC, and give the database the name M359newdatabase before clicking on the Save button. Now click on Next >.

We aren't creating a **Windows CE** database, so leave the option unticked and click on Next >.

We next have the option of creating a transaction log, but for our simple purposes we don't require the protection against loss that transaction logging offers, so untick the option and click on Next >.

Untick the Java support options; again, we're not going to need them for this activity. Click on Next >.

The next page allows us to choose some of the options within the database engine – in this case we'll just move on to the Next > page, accepting any options as default.

The next page is an interesting one: it is asking what size of internal storage structure (pages) we want to use to store our data. This is one of the few places in Sybase Central where we can affect the physical storage schema. Without going into too much detail, this option controls the effect of moving data between the hard disk and the main memory of a computer system as a series of pages. The main memory might not have sufficient space for all the data pages to be loaded from the disk when searching a table, so sometimes quite complex movements of pages from disk to memory are required. The efficiency of the database engine in managing pages can be optimised if you know about the amount of data in the tables and the amount of storage required for the rows of the tables. Such choices are beyond the scope of this course, so we're just going to accept the default page option given here.

However, as an aside, you might want to consider the following. Imagine that this course text had been printed on paper pages in a ring binder with a page size of A1 (that's about 59 cm by 84 cm – quite a large sheet of paper!). Each page in the ring binder would hold a large part of the course text, but you would have the problem of handling a very large and unwieldy page. Alternatively, if we had printed the course text with a page size of A6 (about 10 cm by 15 cm, the size of a small notebook), then each page would have very little of the course text on it, but individual pages would be much easier to carry around. Now imagine you work at a desk that has space for a single A1 page or twenty five A6 pages at a time, and you can only read or update pages that are on the desktop – so any other pages that you aren't using would need to be put back into the ring binder for safe-keeping when the desk is full and you need to look at another page.

Spend a few minutes thinking about which of the following tasks would be easier to achieve (involving less paper movement) for the A1 the A6 versions of the course text:

1. Reading the course text from start to finish;
2. Making summary notes of each appearance of the word 'schema' by using an index to locate the pages containing this word, and then copying the sentences from the pages that contain the word.

You might want to use your course conference to discuss this aside with other students.

To make it interesting, think about how the page movements will change if you use a new page to record your summary notes, remembering that this page must also be on the desktop to write on it. These two types of task, linear reading and indexed access for tasks 1 and 2 respectively, are typical of the activities that take place within a database engine. Storage movement from disk to main memory is a very time-consuming activity so the choice of page size for the storage of data can have a significant impact on performance of the database engine.

Returning to our look at some of the Sybase Central wizards, you can click on Next > now, and you should see details about the collation sequence. The collation sequence specifies how comparisons between text, number, character strings and so on occur within the database. We're used to seeing A to Z comparisons when we sort data, but what about if we sort '£' compared with '€' say, or '10' compared with '900'? Or what if the language used included accented characters, where would they appear in a sorted list of characters? The collation sequences are usually industry standard, but you can choose and define your own collation sequence if you have a specific requirement. We'll accept the default shown and click on Next >.

Finally, we are asked if we want the database engine to run as a server and for Sybase Central to connect to the database after the wizard has completed its activity. By default the server will have the same name as the database that you have created. Click on the Finish button to accept these defaults and the wizard will now instruct the database engine to create the database; you will be shown a progress window that you should close when the activity is complete. The wizard window will then close and the Sybase Central window will be updated to show the new database server M359newdatabase and the new database M359newdatabase-DBA, and the usual list of internal data for the database will be shown in the right-hand window. Of course, the database has nothing in it yet (we've defined no tables) so there are no user tables, views and so on, listed. Note that we won't be able to define any tables until we've looked at some SQL data definition statements in *Block 3* of the course.

One final thing that is worth mentioning is the Users & Groups section of the right-hand window – if you don't see the list headed Tables in the right-hand window then open the hierarchy M359 Server\waltonDB-DBA\ in the left-hand window. Now click on the Users & Groups row in the right-hand window – you should see a list with six rows, with dba shown as a User Type, and other rows shown as Group Type. We will learn more about Users, Groups and the Authorities (DBA and RES.) in *Block 3*, but you should notice that a newly created database has only one defined username – the user who created it, in this case user dba. You can create a new username either by using the menu item icon, or by right-clicking your mouse, choosing New and following the wizard instructions and prompts. We'll say a little more about the dba user and authorities in Subsection 3.6 of this block.

You can close Sybase Central now.

3.5 Information system architectures

In the course so far we have taken an abstract view of the existence of user processes, the DBMS, operating systems and so on, without considering that they may be implemented in many different ways.

We have described how the DBMS can be seen to be separate from a user process, but components are not always distinguishable in that way. Some personal computer packages will include a DBMS and a user process which together operate as a single

unit. Such systems are examples of **monolithic systems** in which all the components are interdependent and cannot be separated.

However, modern computer systems are made from separate distinguishable components which communicate with one another through clearly defined interfaces; information systems are no exception to this. For example, in the *Software Guide* you were introduced to the Sybase DBMS and the end user tools ISQL and Sybase Central, which can clearly be seen to be independent processes running within a single computer system and transferring data between them.

The general term for a system in which interactions between component processes are separated by a communication mechanism is a **client-server system**. The **client** process communicates a request for some service to a **server** process, which is responsible for performing the service and returning any results. Comparing this scenario with Figure 3.1, a user process could play the role of a client, with the DBMS playing the role of a server.

A DBMS is sometimes referred to as a **database server** and an application process as a **client application**.

It is possible to have centralised client-server systems in which both the client and server exist in the same computer system, such as on your desktop PC. However, provided that a computer network allows software components on two or more computers to communicate, it is possible to distribute the client and server across a network. The distribution of the components of an information system across a network is more complex than having them all in a single computer system, but our abstract model in Figure 3.1 should be seen to encompass them both. Most of the content of this course does not distinguish between a database that is part of an information system within a centralised system (that is, on a single computer) and a database on a network of computers, which we refer to as a **distributed system**. The course can be written this way because in both cases the information system has just one database that all users share.

In the following subsections we give a brief overview of some of the common distributed data processing architectures and consider ways in which the data can be physically distributed when a network of computers is involved.

Client-Server systems

In the information system context, the client is usually responsible for the user interface and the **business application logic**. The server accepts and processes database requests and is responsible for the DBMS functions described earlier in this block. This is commonly known as a two-tier client-server system. Figure 3.3 shows the client and server executing on different systems, although they could execute as different processes on the same computer and, of course, there could be a general case in which there are several clients interacting with the same server.

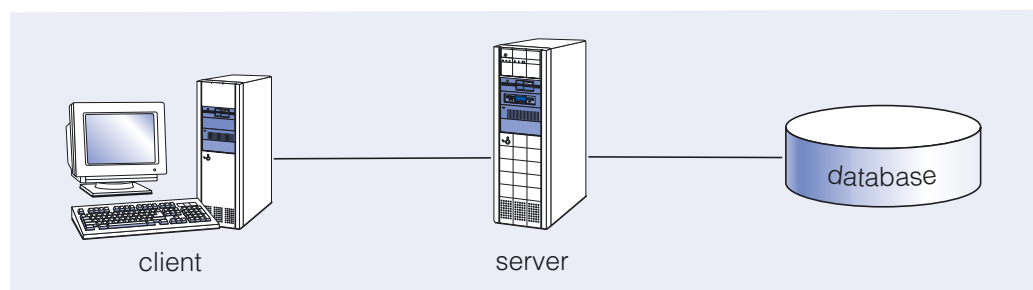


Figure 3.3 Two-tier client-server architecture

A three-tier system is one in which the client is reduced to handling only the user interface (also known as a **thin client**). The client interacts with an application server that is responsible for implementing the business application logic (user process); this, in turn, interacts with a database server responsible for traditional database

management. This configuration is shown in Figure 3.4. Here a single database server might serve multiple application servers which, in turn, can service multiple clients.

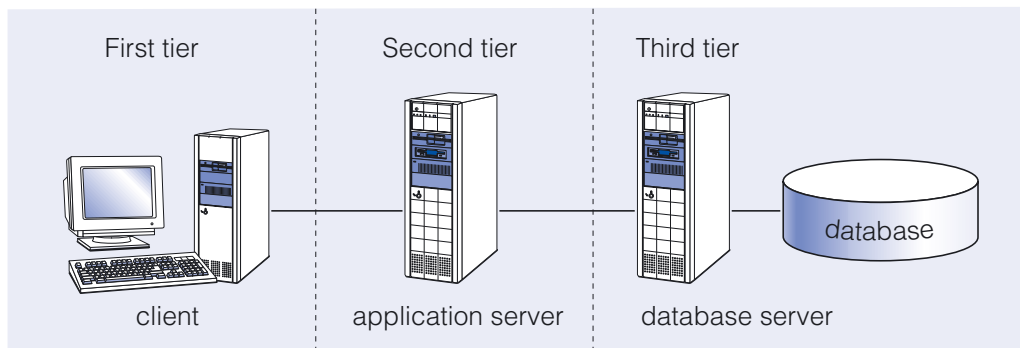


Figure 3.4 Three-tier client-server architecture

The three-tier architecture is common in web environments where the web browser on the end user's PC acts as the thin client, with the web server offering application services. Multiple tiers are also possible where the application level is sufficiently complex and can benefit from the additional flexibility and scalability possible at each tier.

From a database designer's point of view, decisions can be made as to where in the client or server certain processing is performed. An example of this can be seen in calculations involving derived data. For instance, in the supermarket activities in Subsection 2.6, the derived data calculation for the total on a till receipt could be performed in two ways:

- ▶ The DBMS retrieves the required data and performs the calculation, passing the result of the calculation to the client process for display to a user.
- ▶ The DBMS retrieves the required data and passes it all to the client process. The client then performs the calculation for display to a user.

Both methods ensure that users are presented with the same result, but the second method involves all the data having to be transferred to the client process. In a network where communication is much slower than local processing, which is usually the case, the difference in performance for the two cases may be significant.

EXERCISE 3.8

Accepting that network transfer is slow, and all other things being equal, which of the two methods for calculating the till receipt total would be quickest?



Client-multiserver systems

There is another client-server configuration, the **client-multiserver**, in which a single client requires access to several databases which could be managed by different DBMSs. In the client-multiserver approach, the user process needs to serially connect to multiple database servers and combine the results in order to satisfy a particular requirement. However, this is not a straightforward functionality to provide, as we shall see after we've considered the following example.

Suppose a university's logical database is physically split according to the geographic regions in which it has offices and there is a DBMS server in each regional office, as shown in Figure 3.5. Each region physically stores data for only their students and staff in that region's server. However, a user might want to write a query that involves all students in the university, such as a request to count all students on specific courses, or

The same mechanism should also suffice for a local user.

to change staff salaries globally; such a query is possible but requires the client to access multiple physical servers each with its own associated database.

In effect, the user process needs a mechanism to connect to each regional DBMS server to perform queries that can take a logical view of the whole university database that is not confined to a single region. In addition, there must be a way of coordinating any changes to the data in the various databases so that data integrity may be maintained across the whole system.

To achieve the behaviour described above, coordinating changes and maintaining data

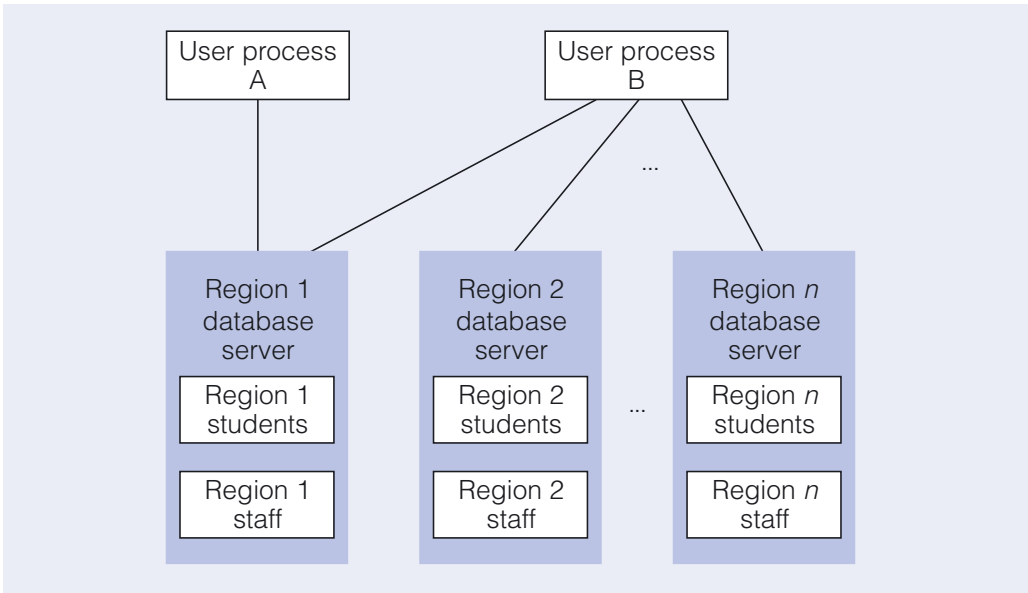


Figure 3.5 A university database implemented on a number of regional DBMS servers

The mechanism described here is based loosely on the SQL connection management facilities, but is typical of systems of this kind.

integrity, we need a mechanism to make and break a client–server connection that also allows a client to communicate with different servers. Most database environments support this way of working by providing SQL statements for **connection management**. We can use the configuration shown in Figure 3.5 as an example. Suppose the university data has been distributed to each of the regions using separate DBMS servers, named region1, region2, and so on. A user process could establish a connection to the database server in region 4. Any subsequent statements are processed by this database server and make access to data available in this database. If the user had previously established a connection with another database server, then that connection would have become dormant since only one connection can be active.

Using the example of separate regions for our university database, as in Figure 3.5, we can see steps that would be required to transfer a particular student between two regional database servers.

- 1 Connect to region1
- 2 Retrieve details of the student that we wish to transfer
- 3 Delete the student details from the region1 database
- 4 Connect to region4
- 5 Insert the student details into the region4 database
- 6 Disconnect from region4
- 7 Disconnect from region1

Figure 3.6 Example of a connection sequence within a user process that transfers a student from region 1 to region 4

How the data is distributed between servers is a design decision, similar to the choice of tables within a database. Decisions would be based on criteria such as ensuring that

data items to be processed together are kept together. However, each database server is autonomous, managing the data under its control and operating independently of any other database server, so that there is no sense in which the totality of the data in the different servers in the university is being managed as a whole.

EXERCISE 3.9

Considering the sequence of events in Figure 3.6, what would be the consequences if the user process crashed after completing steps 1 to 4? Why would transaction management within the DBMSs not help to resolve this problem?



This lack of a global view of the data and the need to look at all of the databases involved in storing related data also create a number of difficulties when we consider the checking of constraints. For example, when a user process executes a statement to insert a new students what is necessary to enforce the requirement that a value for the student identifier is unique for all students? While each database server may enforce a unique constraint for the data it controls, it is necessary for the user process to establish connections to all servers and check the constraint for each one separately so as to check the constraint for all students. This is not a practical proposition and this problem is not easy to resolve.

EXERCISE 3.10

One possible solution to the problem of checking that all student identifiers are unique is that each region is allocated a particular range of student identifiers which can then only be used for students in that region. Why would this assist in checking the uniqueness of identifiers? What practical problems does this solution introduce?



The client–multiserver approach offers little of the management support that we require for data as it fails to treat the data collection as if it were a single database. We can see that the client–multiserver approach requires quite a lot of extra work in managing the connections to the various databases. In addition, the writer of the user process (or the user controlling the user process) needs to know the names of the databases involved and which databases were responsible for which data items. Ideally a user should be able to access distributed data without needing to manage the interactions with the collection of DBMSs. The next systems we consider are designed to relieve the user of these management and location awareness problems.

Distributed, replicated and mobile systems

The key requirement in these systems is that user processes do *not* have to know how data is distributed and so can express database requests as if the data was not distributed.

Distributed systems

In the distributed database approach there is no need for connection management within a user process. Users and the processes they interact with do not have to know anything about where data is stored nor how it is processed. This is referred to as location independence. If the location of data is viewed as a storage characteristic, this requirement for location independence can be considered as just another aspect of data independence. We shall refer to a management system that meets this requirement as a **distributed database management system** (DDBMS).

The requirements for a DDBMS are very similar to the general requirements for data management that we introduced earlier in the course. You will recall that a database has a

logical schema to describe all the data in it. For a distributed database we use the term **global schema**, or **global logical schema**, which describes all the data accessible within the distributed database. Similarly, a local database has a storage schema and the counterpart to this for the distributed database is the **distribution schema** which defines where data is located, that is, in which local database the data is stored.

We want a DDBMS to take full advantage of the distributed nature of data, yet still have the general capabilities of a localised database, and so we expect the following:

- ▶ The DDBMS should present a single view of all data, since users should not have to worry where the data is actually stored; that is, there is location independence.
- ▶ A DDBMS should support transactions across system boundaries and maintain the integrity of the distributed database. A DDBMS must be able to recover from the failure of any particular computer participating in a transaction or a failure in the communication network.
- ▶ There must be a security process that can protect the distributed database from unauthorised access, yet provide uniform access to it for authorised users.
- ▶ A DDBMS should work the same way whether it uses a mainframe or PC and independently of the characteristics of the interconnecting network.

Figure 3.7 shows an example of a distributed database where the user processes communicate with a DDBMS that manages the underlying databases. The DDBMS components of Figure 3.7 should be viewed as cooperating parts of a single DDBMS, and the global and distribution schemas share a single set of definitions. The only way for a user to access the distributed data is to establish a connection to the DDBMS.

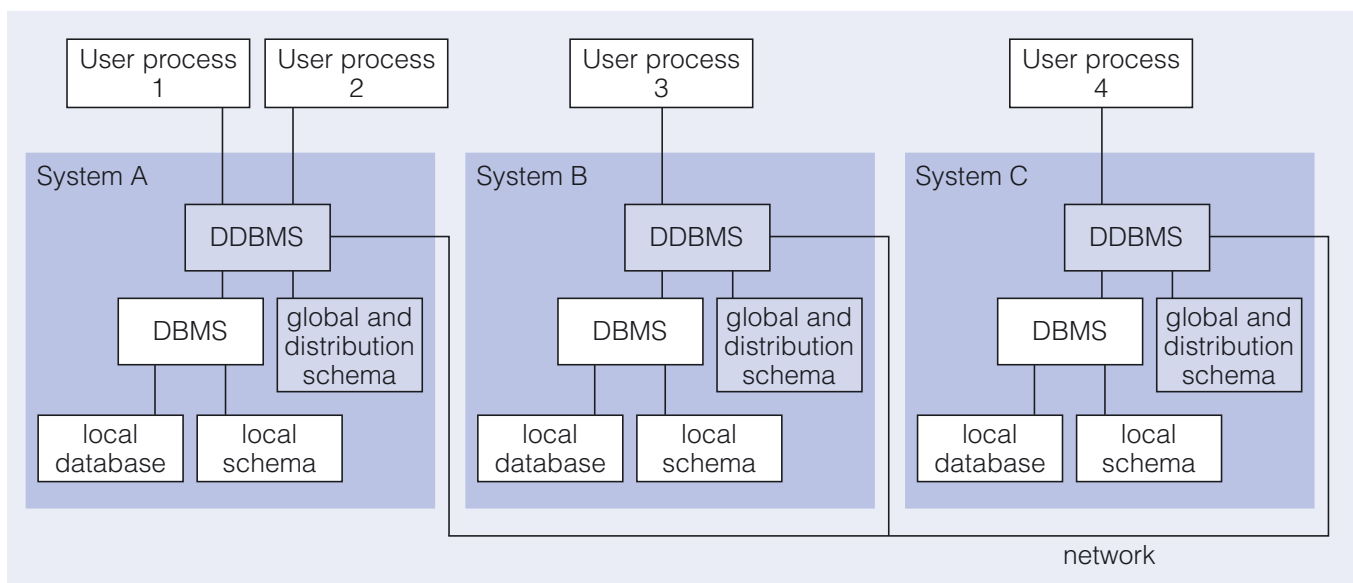


Figure 3.7 Simplified model for a distributed database

A design consideration common to the multiserver and distributed database approaches is the need to determine where to place the data in the different locations. The simplest option when distributing data is to put some tables in one location and others in another. However, most relational DDBMSs include facilities to fragment the tables, so that different parts of each table are distributed to a number of locations. Breaking a table into parts, while maintaining any necessary connection between the parts, is known as **fragmentation**, and it can take different forms:

- ▶ horizontal fragmentation, where a subset of complete rows of a table is stored in one location, such as keeping data for all students and staff in regional databases;

- ▶ vertical fragmentation, where some columns of a table are stored in one location and other columns in another (for example, keeping the names and assessment data for all students in one place and keeping their addresses and other personal details in another);
- ▶ combinations of horizontal and vertical fragmentation.

Moving data between systems will usually be the slowest step in a query. This leads to the oft-quoted fragmentation and distribution design strategy:

Place data so that most processing is performed locally.

EXERCISE 3.11

With the aim of meeting the fragmentation and distribution design strategy, what form of fragmentation of a table is appropriate in the following situations?



- (a) The same processes are performed in each location on different occurrences of data, such as registering students with their region.
- (b) Different processes, requiring different columns of data, are performed in each location on all occurrences of data. For example, a staff table might have a salary column that often needs to be processed from headquarters.

In a DDBMS, the functions we have considered in a local DBMS are present, but have increased complexity. To illustrate this, we will briefly consider transaction management and query processing in the context of a DDBMS.

A transaction that changes data in more than one database server (or location) will require coordination across the multiple servers. It is not sufficient for each server independently to be told to **commit** its work, because some servers may succeed and some may fail; this would violate the 'all or nothing' principle of transactions. One way of supporting transactions for multiple servers is to implement a commit in two phases; unsurprisingly, this is referred to as a **two-phase commit**. In the first phase, all locations involved in a transaction are each sent a request asking them to prepare to commit. Only when all locations have responded that they are ready to commit can the second phase start. In this second phase they are each sent a request to complete the commit. If any one location does not respond to the first request, or responds that it is unable to commit locally, then the global attempt to commit will also fail.

Query processing in a DDBMS involves first finding where the data required is being managed using the distribution schema. The DDBMS must then determine how and where to transfer data between local databases so as to optimise the performance. Since communication is the slowest step, the system might attempt to minimise the transfer of data over the network when trying to optimise the distributed query. However, such a simple description hides a great many complex options which would depend on the sizes of tables at the time queries were executed.

EXERCISE 3.12

- (a) Consider the DDBMS shown in Figure 3.7. System A receives a request to combine data from two databases held in systems B and C. How many different combinations of data transfer and processing can you identify (assuming that there are only locations A, B and C in the system)?
- (b) Suppose the volume of data involved in satisfying the query mentioned above at location B was 1 MB, and the volume of data involved at location C was 2 MB. Also suppose that the result of the query was 0.5 MB. Which of your combinations would result in the minimum transfer of data over the network?



Replicated systems

Replication provides copies of data that are distributed to other database servers so that each user process can perform all processing locally. It makes data available through local access, minimises the use of remote connections, and reduces the dangers of remote system failure or connection failure. Replication can be applied to the entire contents of a database or to selected items within a database. Also, the replicated items can be copied to every system within the replicated environment, or only to selected systems.

As with the DDBMS approach to distributed data, a specialised service is needed to provide the additional coordination of exchanges between participating databases. A replication service is required to ensure that copies of replicated data are maintained in a suitable state of consistency. Ideally all changes to replicated data will be perfectly synchronised within the system. This ideal is unachievable where there are frequent updates to data copies at multiple locations, or in the presence of partial failures of the systems or network. Figure 3.8 shows a simplified connection between databases using a replication server at each system to act as the channel between them. A replication server is appended to each local DBMS – this communicates with the DBMS and is not directly accessible to user processes.

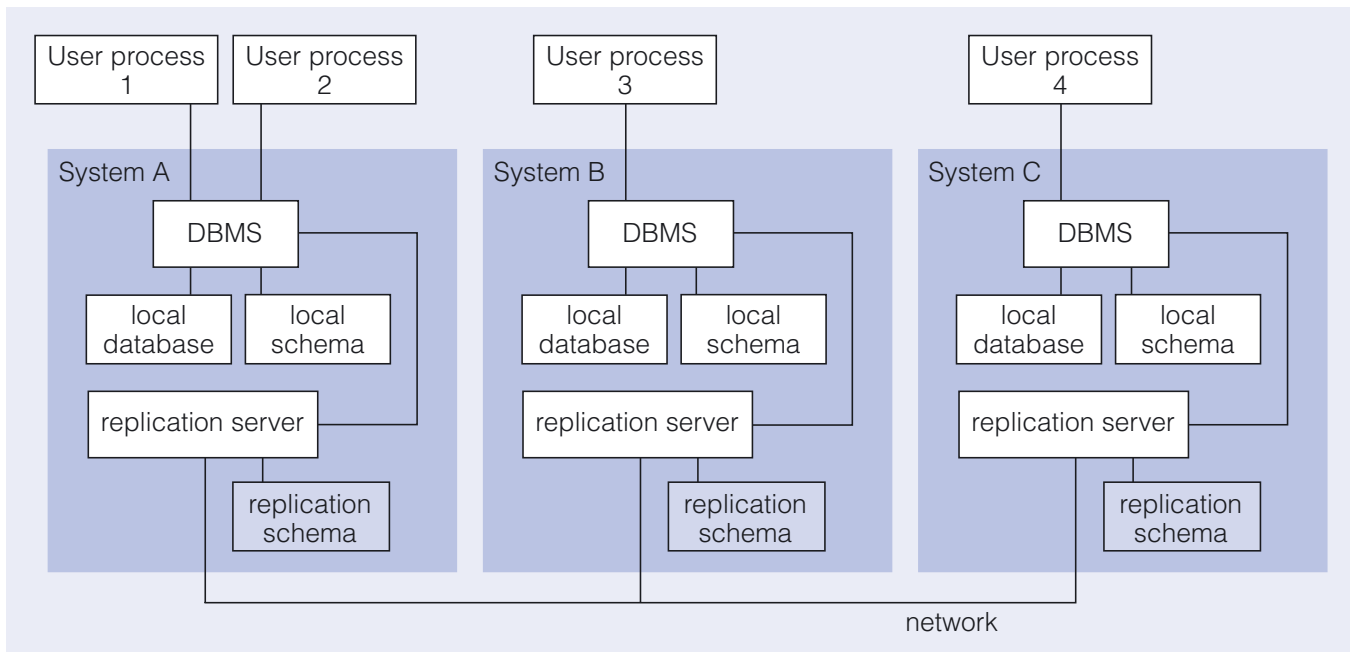


Figure 3.8 Simplified model of replication between three database systems

A **replication server** both sends and receives data and so has two main roles:

- ▶ to receive or collect updates from its local database and pass copies on to other systems that require them;
- ▶ to receive or collect updates from remote systems, via their replication servers, and apply them to the local database.

Push systems are sometimes also called broadcast or subscription services, and **pull systems** are sometimes known as demand-driven systems.

There are two ways in which a transfer of replicates may be initiated: either from the sender or from the receiver. Referring to Figure 3.8, the DBA for System A may declare that System B should receive all changes to certain tables. So, whenever those items are updated on System A, replicates are *pushed* over to System B. In contrast, the DBA for System B may from time to time request copies of any changes to replicated items located in System C. Now, replicates must be *pulled* from System C on demand by System B.

The replication server stores descriptions of any replicated data and descriptions of how to reach other database servers. Users may continue to work with local copies of data, even though there may be a system failure at the remote location that holds the primary copy of the data. For example, in Figure 3.8, user process 1 connected to System A will be able to continue working if System B fails for some reason. Most DBMSs offering replication services will include several different strategies for resolving conflicts of simultaneous replicate updates or for resolving the system state after partial system failures or network outages.

EXERCISE 3.13

- (a) What potential consistency problem could arise if users on Systems A and C simultaneously updated a data item replicated at all three systems shown in Figure 3.8?
 - (b) Does the choice of push or pull replication in these systems help resolve this issue?
 - (c) Suppose a system managing replicated data had been inaccessible due to network failure for a short time. When the network failure is resolved and the system becomes accessible, what would you expect to take place at that time?
-



A database designer will be involved in deciding the most appropriate data items to replicate on each system. The designer will also determine which forms of replication service to use, and the mechanisms to be used to resolve any potential conflicts in consistency management.

Mobile database systems

With the growth in the deployment of wireless-networked portable computers, laptops and powerful **PDA**s, many database vendors now offer mobile DBMS facilities. In effect these are distributed or replicated systems in which parts of a global database are fragmented and/or copied to the mobile devices in much the same way that the fixed networked distributed and replicated systems operate. However, in a mobile DBMS the assumption is that for most of the time these distributed and replicated data sets are not accessible to the global environment. Such systems depend on the mobile components reconnecting periodically in order to exchange data with the other systems.

In mobile environments consistency cannot be maintained at all times. Good fragmentation and replication decisions need to be made for a successful mobile deployment. These must avoid the need for a large number of manual interventions to resolve consistency problems when mobile systems reconnect.

Enhancing DBMS capabilities

The client–server architectures mentioned earlier are based on the separation of components of the information system along functional lines – user interface, application services and database services. However, it is now becoming common for vendors to embed some of the application services capabilities within the DBMS itself and so extend the DBMS functionality. It has long been possible to include program code written in C or COBOL, for example, in a database and to have the DBMS request the host computer to execute this code when required. The growth in standard component services such as **web servers** and **Java virtual machines** has led the DBMS vendors to embed these capabilities directly into their DBMS products. This leads to the so-called ‘enabled’ DBMSs such as Java-enabled, web-enabled and XML-enabled DBMSs. In a Java-enabled DBMS, Java code can be stored, manipulated and executed within the DBMS. In a web-enabled DBMS, a web server interface is able to

permit web pages to directly interact with the content held in the DBMS without the need for an independent application service layer. XML-enabled usually indicates that the DBMS has built-in capabilities to store and retrieve complex XML documents without the need for additional supporting software. (There will be more on Java and XML in *Block 5*.)

3.6 People and databases

We finish this section by looking at two issues in data management systems – the users of the system and their interaction with the DBMS. Remember that one definition of an information system is ‘a computer system whose primary purpose is to support the information requirements of its users’. Clearly it is important that we consider the users in a review of data management concepts. However, the users (sometimes called the **end users**) of the data management system are not the only people involved in a data management system.

In this subsection we will consider different roles for people involved in a data management system – data administrators, database administrators (sometimes referred to as power-users), database developers, application developers, end users, and abusers of the system. There may be situations where all of these roles are performed by the same person or people, and other situations where there are clear divisions of responsibility between the people who fill these roles. As with many of the descriptions in this course the following are illustrations of the types of activities and responsibilities that such roles entail – every organisation and each data management environment will of course have their own variations and conventions.

Data administrators (DAs)

Usually found in large organisations, the data administrator (DA) is a senior manager whose remit is to oversee the information strategy for the organisation. As a manager the DA sets policy and is not involved in the technical management of the data or the data systems. The role covers three major aspects of the organisation's data management: what data is needed within the organisation, how that data gets used to the maximum benefit of the organisation, and (increasingly important) what legal and ethical requirements the organisation needs to fulfil in order to store and manipulate that data. The DA will often act as a management interface between the technical data management division and the business users of the data. In such a role it is a requirement that the DA understands the business domain and its data needs and is aware of the technical possibilities and limitations of the systems that are requested to meet these needs.

A typical DA will oversee the planning and development of new data systems and the maintenance and control of existing systems. This will involve determining the organisation's data needs, deciding how they can be met by data management systems and initiating and overseeing the development of the organisation's conceptual models. Management of the data includes defining and maintaining policies with regard to data acquisition, data safety (loss, damage and disclosure), data security, data maintenance, data archiving, data deletion and data availability to users.

Database administrators (DBAs)

The technical management of the data systems and the responsibility for implementing the DA's policies and conceptual models as well as the day-to-day running of the database systems fall within the remit of the Database Administrator (DBA). The DBA designs and implements the physical database (or at least heads the team which designs and implements the physical database). The implementation will include data

definitions, constraints, security and integrity controls. The day-to-day running of the system will involve performance tuning, data backup management, and system availability monitoring to ensure that end user requirements are met. In addition the policies for data management described by the DA will be implemented, managed and audited by the DBA.

EXERCISE 3.14

In the following list of activities, which do you think would be the responsibility of the DA and which the responsibility of the DBA?



- (a) Educating users on data standards and legal responsibilities.
 - (b) Developing testing strategies.
 - (c) Liaising with users to determine new requirements and resolve data access issues.
 - (d) Keeping up to date with software and hardware upgrades.
 - (e) Determining appropriate security for specific confidential data.
 - (f) Selecting a suitable DBMS.
 - (g) Implementing security access restrictions.
 - (h) Monitoring the performance of end user applications.
-

The technical aspect of the DBA role is reflected in the fact that many DBMSs will have a default username of **dba** associated with the administration rights for the DBMS. We've seen this already in Activity 3.2, where you connected to Sybase Central as the user **dba** in order to create a new database. Then, after creating the database, you saw that **dba** was, by default, a user of the new database who had been granted **DBA Authority and RES. Authority**. The ability to create a new database within the DBMS server is an example of a privilege that makes sense for the technical side of the DBA role, but wouldn't make sense for end users. There are a lot of other privileges that a DBA would be granted that other users should not have. DBAs have a requirement to manage the day-to-day running of the system, set security and create usernames for the system, a requirement that other users do not have. We'll talk more about privileges in *Block 3*.

Not all vendors choose **dba** as the username for the administrator, but there will be something equivalent on every multi-user DBMS.

Other users can be given administration rights (the **DBA Authority** in Sybase Central) so that the ability to administer a database is not restricted solely to the **dba** user. However, it is considered good practice not to give administration rights to users unless it is essential for their work – and even then, users who work as administrators sometimes have two usernames, one with administration rights for when they really need it and another with more restricted rights used when doing more routine work. In this way you avoid the risk of casual users accidentally being given authorities that can damage the security or integrity of the database system.

While mentioning good practice we should also note that DBMS vendors will often supply a default password to go with the default **dba** username. This means that if the password is not changed for the **dba** user then anyone who knows which vendor supplied your DBMS will be able to connect to your database as the DBA. In Activity 3.2 you notice that the password you used was **1qs** – we've followed good practice by changing the vendor-supplied password to one of our own creation. It's not actually a very good password, nor is it very secure (by the final presentation of this course about 6000 people will know that we've used it on all our student software installations!), but it is adequate for the purposes of teaching.

These good practice requirements should, of course, be documented as part of an organisation's Data Administration policies within their CIS.

We've also created some usernames for use with our SQL teaching in *Block 3*. We will generally be using the username **m359** with the password **m359** (again not

very secure, but sufficient for teaching purposes). The **m359** user does not have administrator rights.

Database developers (DDs)

A database developer will be an expert in the technical capabilities of the DBMS and hardware systems within which a database will be implemented. They will be responsible for translating the logical database design into a physical realisation within a specific DBMS product. It is in fulfilling this role that the performance characteristics and physical storage options will become important as the DD must choose a mechanism for implementation that satisfies performance as well as logical requirements. It is important for the DD to know the different implementation options and the consequences of selecting one alternative over another so that they can make informed implementation decisions.

Since the DD is creating the physical database within the DBMS they will also be responsible for implementing constraints, security, backup mechanisms and access mechanisms, and they will often write the scripts, code and stored queries and procedures that the application developers and end users will access.

Application developers (ADs)

The application developers are tasked with building or customising user processes (programs and utilities, report generators and query tools) to meet the specific requirements of the end users, usually working under the direction of the analysts who specify the functionality required by each user or group of users.

Application programs will be written to initialise communication with the DBMS, identify and open appropriate databases, and declare any user identification and authentication information for those databases. Their code will contain requests to the DBMS to perform the required operations and will be responsible for the coding, packaging and presentation of data to and from the DBMS interfaces as well as handling error conditions that the DBMS reports. The AD will treat the database and DBMS (and all the data and any stored procedures within it) as a given, that is, they won't be involved in its design or implementation.

End users (users)

The end users are usually characterised as the people who have the least interest in how the database has been designed, implemented and managed – they are interested in the data. In many cases the end user will neither know nor care that a database is involved in meeting their needs. For example, a supermarket checkout operator scans bar-codes through a bar-code reader and has no need to know what is happening to the data captured by the reader other than knowing it is a requirement if the customer's bill and check-out receipt is to be accurate. In general, end users will require simple mechanisms for interaction with the database.

Unwanted data users (abusers)

These can range from those involved in industrial espionage, hackers and organised criminals (intent on fraud, blackmail, etc.) to those authorised users who overstep their permitted access to certain parts of systems either accidentally or deliberately. In the database environment it is common to find layers of security and protection designed to prevent widespread damage if permitted access boundaries are breached. While database systems are no more or less vulnerable than other computerised systems the threats in database systems are to the (usually) large and valuable data sets and the real world assets that they represent.

The role of the database developer is often subsumed in the role of the DBA.

3.7 User interaction and database access

So far in the course we've used the fairly general label of user process for any software used by a user to gain access to the data in an information system. Here we will look in more detail at the distinction between two types of user process – the application process and the database tool. We will look at the role of SQL in providing a common interface to the DBMS irrespective of the different types of user processes and so show the key role that SQL plays in interaction with a relational DBMS. This ensures maximum accessibility, through a common interface to the content of the database when required.

Application processes

One kind of user process is an application process, such as those you used in your work in Subsection 2.6 with the Walton and Notlaw information systems. The term application process is used to refer to software that involves a specific user view for just one database that is used for some particular set of tasks. In other words, an application process can only be used for one purpose – the 'application' for which it was specially designed and constructed; the user cannot change it in any way. You saw several examples of application processes in the supermarket activities earlier. For instance, the Notlaw till management application cannot be used to control the Walton database, or to meet other requirements of the Notlaw Supermarket. A number of application processes may be packaged together, as you saw for the Walton and Notlaw information systems examples where the supermarket user interfaces are used to give structured access to the individual application processes.

Database tools

The second kind of user process is a **database tool**, such as Sybase Central and ISQL. A database tool is a general-purpose package that can be used with many different databases, one at a time, by any authorised user, to access any data available to that user within a database. Such a tool usually provides a wide range of ways in which data can be processed and displayed. The key issue in a database tool is to offer flexibility to the user, both in terms of the databases they can access and also in the ways in which they interact with the DBMS functionality.

A range of capabilities are typical of database tools which are sometimes referred to by specialist names such as:

- ▶ Query tools, where the emphasis is mainly on the retrieval of stored data.
- ▶ End user tools, where access is not just focused on data retrieval but may also allow updates, constraint definition and access control setting.
- ▶ Reporting tools which provide many facilities for the structured display and printing of retrieved data.
- ▶ Application development tools which allow the production of stand-alone application processes without the need to use conventional programming languages directly. They often allow the generation of form-based or menu-based application processes more quickly and easily than following a traditional programming route.
- ▶ Management tools which focus on the control and management of the database through the DBMS.

Clearly such tools are not used by naive users, such as the checkout operator mentioned earlier. In its most basic form, a database tool might involve direct entry of SQL code (sometimes referred to as a direct entry database tool); it is effectively an SQL editor directly connected to the DBMS. Of course, more complex tools are also possible offering more sophisticated capabilities. To use a database tool effectively

requires an understanding of both the tool's capabilities and the structure and meaning of the data in the databases being accessed. In addition, the capabilities of the tools can usually be customised by the user to build libraries of commonly used procedures and activities.

It is quite common for DBMS vendors and third-party suppliers to offer suites of database tools which can be tailored for specific user requirements. In such tightly integrated environments the distinction between the various types of database tool can become blurred. This blurring can also apply to the distinction between the application process and database tool. The distinguishing feature is usually the primary focus for the software. If database interaction is the primary focus, then it is usually a database tool. If the primary focus is a business function or end user requirement, then it is usually an application process.

Accessing the DBMS

We shall now consider in more detail how the DBMS functions are accessed by user processes. Figure 3.9 shows a number of user processes each connected to a database via a DBMS. The interaction of these components is an important part of the architecture, and we need to examine both the way a request is expressed and how a request is submitted to a DBMS for execution.

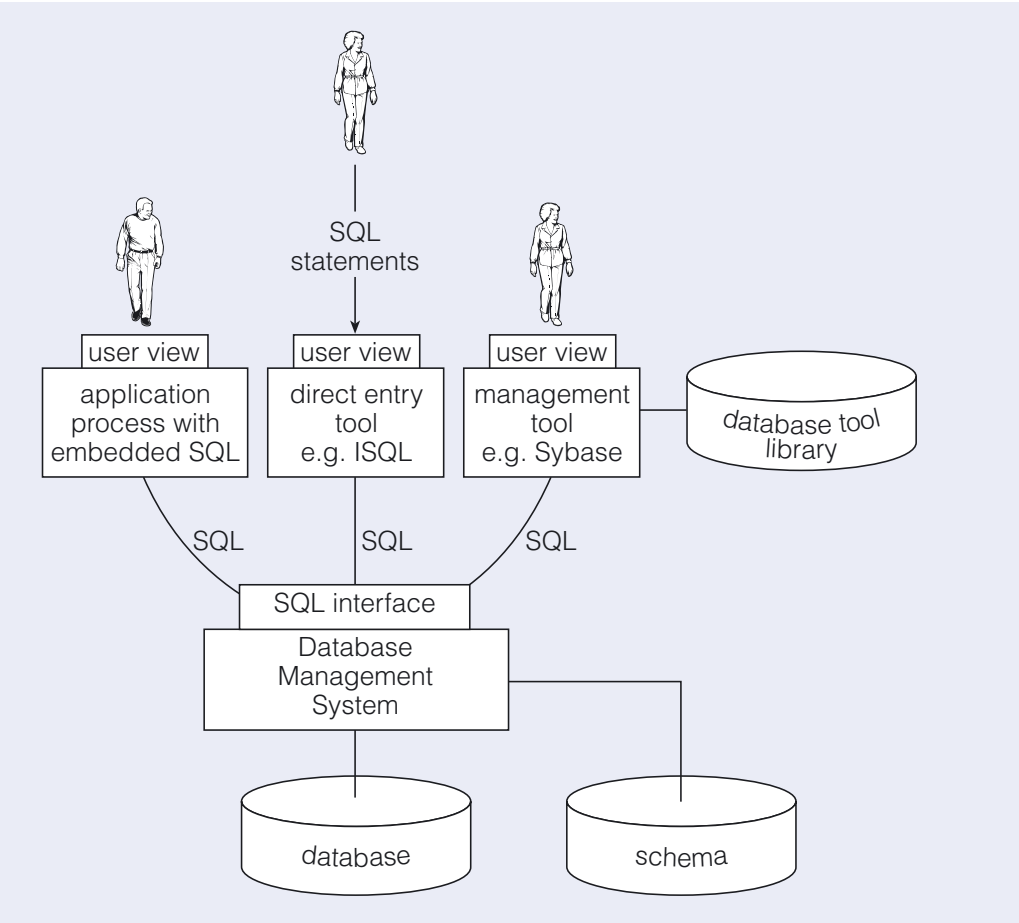


Figure 3.9 Model showing use of SQL statements for interaction with a DBMS

This model differs from the general model we presented in Figure 3.1 because here we make a distinction between application processes and database tools whereas in Figure 3.1 we considered them all to be generic user processes; we also show the DBMS with an SQL interface, showing that SQL can be used by any user process to interact with the DBMS.

Let's look in more detail at the use of SQL within some types of user processes.

Accessing the DBMS via an application process

EXERCISE 3.15

Which DBMS functions would you expect an application process to commonly request? You might want to review Subsection 3.3 to remind yourself of the DBMS functions available.



Solution 3.15 relates to the normal purpose of an application process. There is no intrinsic reason why the data definition function, for example, cannot be requested in an application process, but that is not the usual way it is used. We have included transaction support because that is required for some application processes, but it is data manipulation that is common to all application processes, and that is the focus of the following discussion.

As we explained earlier, a characteristic of an application process is that it can only be used for one purpose, and so the data it processes is restricted to the task it is designed to perform. Whatever the task, the users of an application process play no part in choosing which tables and columns of data are accessed.

For example, as a user of the application processes in the supermarket activities, you were simply not aware of a DBMS request, or what form it took. For this purpose, each DBMS uses a special language, known as a **data manipulation language** (usually abbreviated to DML). Thus, an application process is written by an application developer as a series of statements using a conventional programming language combined with other statements written using the DML specific to that DBMS. The programming language being used is said to host the DML, and may be referred to as the host language. Conversely, a DML statement is said to be embedded within an application process.

Here's an example of a fragment of Java code with an SQL statement embedded in it.

```
...
rowSet.setUsername("m359");
rowSet.setPassword("m359");
rowSet.setCommand("SELECT title, author FROM book WHERE
                    type = 'e-book' ");
rowSet.execute();
...
```

The SQL DML statement is the following text:

```
SELECT title, author
FROM book
WHERE type = 'e-book'
```

You can see the effect of this SQL statement on the Walton Supermarket database if you perform the following activity.

ACTIVITY 3.3

Evaluating an SQL query

Locate and execute the ISQL direct entry tool, as described in the *Software Guide*.

Now connect to the Walton database using the connect option in the SQL menu, or using the keyboard shortcut F11. You should complete the connection prompt window using the User ID m359 and the Password m359 in the upper part of the identification page of the connect window. In the lower half of the window highlight the ODBC Data Source name button and choose the name wa1tonDSN from the drop down list. Then

For an application process prepared using an application development tool, the concept of statements is not always explicit, but the same principles apply.

We don't expect you to know any SQL at this stage. However, you might want to see if you can interpret any SQL examples we do give in this block – what do you think the SQL query means?



click on the OK button; this should return to the ISQL application with the Results window showing a connected message or the window name will change to show the database, username and database server for the connection (in this case: WaltonDB (M359) on M359 Server).

In the SQL Statements window of the ISQL tool, type the SQL DML statement shown above. Make sure you type the statement exactly as shown.

Now use the execute menu button (or the Execute menu item on the SQL menu or the F5 function key if you are using keyboard shortcuts) to execute the query and you will see the result of evaluating the query in the Results window of the tool.

The result should show title and author data from the Walton book table. If the embedded SQL had been evaluated during the execution of the Java program then the values returned by the DBMS would be available for use in the Java program.

You can close the ISQL tool now.

Note that the SQL in the Java fragment always requests the DBMS to execute the same SQL statement whenever the Java code is executed.



EXERCISE 3.16

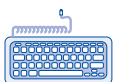
The SQL statement in the Java fragment shown above is always fixed, but the DBMS might return different results each time the application sends the request to the DBMS – explain why.

Accessing the DBMS via a direct entry database tool

You have already seen an example of a direct entry database tool when you used the ISQL software in Activity 3.3. The most important point to remember here is that a database tool is an example of general-purpose software, intended for use with many databases. Therefore a database tool does not contain any fixed embedded DBMS request for manipulating any specific data because requests will be different for each database and each user. When used with some specific database, a database tool requires a separate specification of the particular data manipulation to be requested for that database.

One way of providing a specification of the required data manipulation is by direct user input. This involves a user entering some statement, expressed using what is known in this context as a **query language**, which is immediately executed by the DBMS. This way of working was demonstrated in the previous activity when you typed an SQL statement directly into the **SQL Statements** ISQL window. When the ISQL execute button was pressed, the SQL statement in the window was submitted to the DBMS, with the results being returned to the ISQL tool and automatically displayed.

The majority of the SQL you will be asked to use in this course will be presented to the DBMS through the ISQL direct entry tool.



ACTIVITY 3.4

Using pre-prepared SQL queries in direct entry tools

Locate and execute the ISQL direct entry tool again, as described in the *Software Guide*.

Now connect to the Walton database using the connect option in the SQL menu, or using the keyboard shortcut F11. As before, you should complete the connection prompt window using the User ID m359 and the Password m359 in the upper part of the identification page of the connect window. In the lower half of the window highlight the ODBC Data Source name button and choose the name WaltonDSN from the drop-down list. Then click on the OK button; this should return to the ISQL application with the Results window showing a connected message or the window name will change to

show the database, username and database server for the connection (in this case: waltonDB (m359) on M359 Server).

Now use the Open... option on the File menu (or Alt-F then Alt-O if you use keyboard shortcuts) and locate and open the file ..\M359\Activities\Block 1\Other Activities\showhighvaluestock.sql. This should show the contents of the file (an SQL **SELECT** statement) in the SQL Statements window of the ISQL tool.

Use the execute button on the ISQL menu (or the Execute menu item on the SQL menu, or the F5 shortcut key) to execute the query displayed and you will see the result of evaluating the query in the Results window of the tool.

You can close the ISQL tool now.

The ISQL direct entry tool does not require you to type all your queries directly into the **SQL Statements** window before execution. As this activity demonstrates, SQL statements can be loaded from a stored file and then executed.

The use of such a pre-prepared file is just a convenience – when executing an SQL statement in a direct entry tool, the DBMS does not need to take account of where the statement originated. This allows you to prepare SQL statements using any text editor of your choice and without having to connect to the DBMS until you want to evaluate your queries.

Accessing the DBMS via a management database tool

An alternative way of including the specification of the required data manipulation is as part of a tool's internal definitions of its interactions with a given database. In Sybase Central the menu items, windows icons and data entry and manipulation forms have database actions associated with them. When a user selects a management function, that request is sent to the DBMS typically in the form of SQL statements. So when you click on a management function icon a request is sent to the DBMS to perform that function. If the function requires particular data to be used then that data is requested, usually in a window or form into which the user types the required values. These forms are not evaluated by the DBMS. They are just a way of expressing to the management tool the DBMS functions that are required and the values to be used when the DBMS satisfies the request. The actual request presented to the DBMS will typically be one or more SQL statements.

Users of such tools are not aware of DBMS requests or the specification of the data manipulation for their activities, and in that respect these are similar to the application processes of the supermarket examples. However, the database tool allows you to examine and modify the specification of the required data manipulation, which is not possible in the application process.

Figure 3.9 also shows that a database tool can store tailored actions and procedures, such as forms, menu items, reports and so on, in a file. Such a file is sometimes known as a **library**, since its role is comparable to a program library that stores collections of conventional programs. Its representation in Figure 3.9 is another simplification. We show the library stored on a disk because it enables us to represent this library separately from the database tool. A library is not the only way of storing such procedures – some database tools store forms in the database alongside the data to which they relate. Implicitly, Figure 3.9 is intended to include all the types of customisable definitions used by a database tool.

The *Software Guide* describes other file handling facilities of the ISQL tool.

You saw an example of these kind of management functions when you used the **Create Database** utility in Sybase Central in an earlier activity.



EXERCISE 3.17

- (a) In Subsection 1.5 of this block we said that 'Since the DBMS has sole responsibility for data access, all activities related to data access requirements will reside within the DBMS and be accessed by the user processes'. In effect this represents a library of common activities similar to the library of actions and procedures discussed above for management database tools and shown in Figure 3.9. Suppose you have another management database tool that is different from Sybase Central. Explain whether you should expect to be able to use both the DBMS and Sybase Central libraries with this new management tool.
- (b) Having a library file enables a database tool to store forms for a number of different databases with which it may be used. If this is done, describe a possible problem for the users.
- (c) Why is the separation of the database tool library from the actual database of benefit in a distributed client-server environment?

The SQL interface

Very generally, the SQL interface represents some means of transferring an SQL statement from a user process to a DBMS for execution.

All the requests for DBMS functionality made by the various user processes can be expressed using the database language, SQL: either embedded, directly entered by users, or stored and accessed in tool libraries. There are other database languages used for this purpose, but SQL is an international standard and has become widely adopted. This central role in the database architecture and its widespread use are the reasons why we place so much emphasis on the use of SQL – the subject of *Block 3* of this course.

An SQL statement which has been input to a database tool by whatever means has to be transferred to a DBMS for execution. You may imagine this as a **procedure call** in which an SQL statement is just a **parameter**, which we can illustrate in a very simplified way as

CALL execute_this_SQL (an SQL statement)

The collection of procedures that are available for a given DBMS forms the interface for interaction with the DBMS.

One major complication with the use of such a simple mechanism is that each software supplier chooses to implement the details of each procedure call differently for their DBMS. This is reflected, for example, by having different names for the procedures, different parameters and different formats for returning results. Such an interface is said to be **proprietary** (or **native**) to that DBMS vendor. An immediate problem with having different proprietary DBMS interfaces is that any database tool which wants to interact with a DBMS via its interface has to express all interaction in terms of those proprietary procedures. Consequently, for a tool to have the flexibility of interacting with DBMSs from many different suppliers, it would need to include separate code for each proprietary interface.

However, there is a solution to this problem. A set of general procedures has been defined for common use, and together they are known as **Open Database Connectivity (ODBC)**, because they provide an open (standard) interface to a DBMS. This means that a database tool can use these ODBC procedures to express its interaction with a DBMS in a single standardised way, and so be able to interact with any DBMS that has an ODBC interface. Suppliers who want their DBMS to be accessible to database tools will supply an ODBC interface for their products.

There is an international standard for ODBC, called SQL/CLI (call-level interface), but in practice people refer to ODBC.

Providing a DBMS with an ODBC interface does not require its supplier to replace its native interface – what happens in practice is that a process known as an **ODBC driver** is used to receive ODBC requests from a database tool and translate them into native DBMS requests, as illustrated in Figure 3.10.

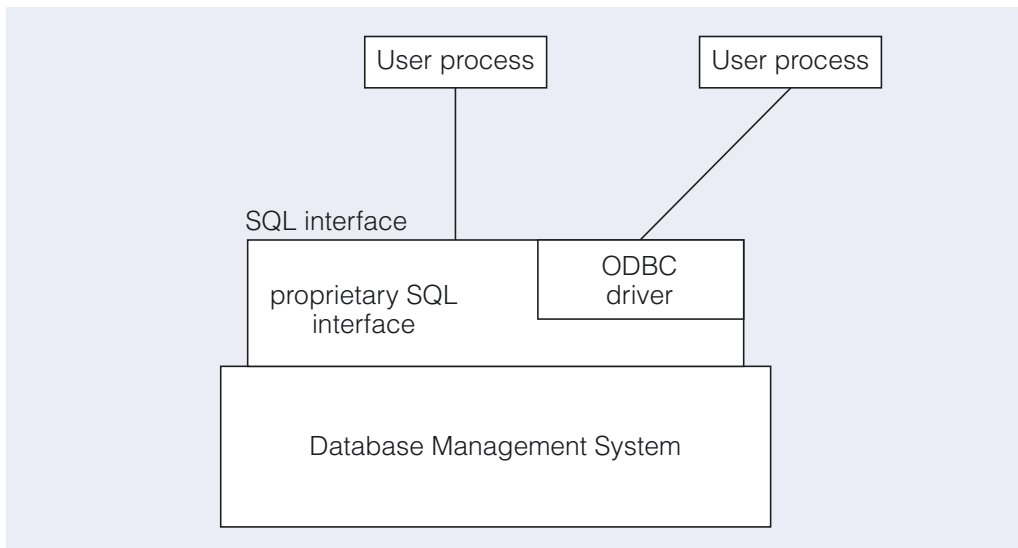


Figure 3.10 DBMS with both proprietary SQL and ODBC interfaces

ODBC is very much concerned with the internal interfaces of the software, and most database users do not have to understand the details, so they are not considered in the course. However, you do need to appreciate the role of ODBC in a database architecture because it is central to the flexible access to a DBMS by general-purpose tools.

EXERCISE 3.18

Describe the main benefit of using ODBC for a database tool like ISQL.



Having examined the way a database tool interacts with a DBMS, we now consider whether the situation is any different for an application process. Of course, you might ask the question: why doesn't an application process interact with a DBMS in the same way as a database tool? The answer is that it can – an application process can be written to use the ODBC interface.

EXERCISE 3.19

Explain why an application process does not derive the same benefit from using ODBC as described for a database tool in Solution 3.18.



So, although an application process can be written to use the ODBC interface, as the solution to Exercise 3.19 indicates there is no real benefit to be gained by avoiding the native interface for an application process that is tailored to a single database.

3.8 Summary

This section has discussed the architectural descriptions of information systems, including the three-schema architecture abstraction for the DBMS and the implementation of the information system components in networked and non-networked environments. We have presented different forms for the distribution of data and processing with a networked DBMS.

We have also seen an outline description of the functionality of a modern DBMS and the relationship between the architectural structure and the functionality and implementation choices within an information system. We discussed why database design and development activities can focus on data requirements (the logical schema development) without the need to be concerned with the realisation of the user processes or the physical storage capabilities.

We finished with an overview of the various user roles in an information system and a description of user processes and their relationship and interaction with a DBMS.

LEARNING OUTCOMES

After completing study of this section you should:

- ▶ Have a fuller knowledge and understanding of the general context and concepts appropriate to the study of information systems.
- ▶ Be able to apply knowledge of the three-schema architecture and information system architectures when describing DBMSs, and to relate the features and facilities available within DBMS products to the architectural abstractions.
- ▶ Be able to recognise the roles and responsibilities of the key individuals involved in DBMS development, management and use, and how their requirements are met by various types of user process.
- ▶ Be able to make informed decisions regarding the appropriate separation of concerns in the description and development of an information system centred on a DBMS.

4

Developing database systems

4.1 Introduction

So far in this block we have considered the role of a DBMS and the properties and requirements for databases to meet user and organisation needs. What we have not considered is the much broader issue of how we decide what should be in our database and how we choose to structure the tables we will include. The remainder of the course is focused almost entirely on these issues.

In this section we are going to look at an outline of the stages involved in the development of a database. Our aim is to give you a basic development method in which you will be able to see how the theory and practice presented in the remainder of the course will fit. We don't attempt to argue for this specific method being the most applicable to any given situation – however, the method we consider is straightforward and allows us to consider all the techniques we cover in this course.

Before we consider the development method in more detail let's discuss why we need to take a formal approach to database development. After all, it is quite simple to use SQL **CREATE TABLE** statements to define tables, or use the facilities of a database tool to define them for you. Once developed, the tables can be manipulated and displayed in many different ways, again using SQL statements, a database tool or an application development tool. However, uncontrolled ad hoc creation of tables by end users leads to an unmanageable and unusable database environment, which can include multiple copies of potentially inconsistent data. This can effectively create islands of data within which the end users cannot find the data that they require.

To recognise why methodical development is an issue, let us look at a very simple example. A hotel provides its clients with accommodation, food and drink and wants to record what each client spends for each cost category so that, as they leave, each client is presented with an itemised bill for all they have spent.

The problem is that there is not just one way in which we can choose tables for this purpose. We can suggest three alternative ways of satisfying the basic requirement of being able to record the data that the hotel has specified. Occurrences of data for two example clients (arbitrarily identified by a code) for each method are shown in Figures 4.1, 4.2 and 4.3. The billing data for both the clients is the same in all three figures, just represented differently.

Bill

Client	Accommodation	Food	Drink
C35	162	75	
C41		38	7

Figure 4.1 First hotel example table

In Figure 4.1 there is one table, Bill, which has a row for each client and a column for each cost category. When a client does not spend any money for a category, there is simply no entry.

Accommodation		Food		Drink	
Client	Cost	Client	Cost	Client	Cost
C35	162	C35	75	C41	7
		C41	38		

Figure 4.2 Second hotel example tables

In Figure 4.2 there are three tables, Accommodation, Food and Drink, corresponding to each of the three cost categories, and each table has a row for a client only if they have spent money in that billing category.

Cost		
Client	Category	Cost
C35	Accommodation	162
C35	Food	75
C41	Food	38
C41	Drink	7

Figure 4.3 Third hotel example table

In Figure 4.3 there is one table, Cost, which has a row for each item of cost, with an associated column describing the category of that cost.

We are not going to say which is the best option to choose, mainly because this decision really involves a lot more understanding of the user's requirements than we have presented here. In particular, it is important to know whether the data may be used for some other purpose (such as monitoring regular clients) and whether there may be a requirement to include additional data at some time (such as the cost of telephone calls).



EXERCISE 4.1

For the three options given for the hotel example, describe how each one would allow for the inclusion of data about the cost of telephone calls.

We can see from Solution 4.1 that any additional type of data can have a different impact on each alternative data recording technique. Databases may be expected to change, so you need to appreciate that making the right choice of tables is important for the long-term success of the database implementation.

The message from this simple example is that a relational database is not just a collection of tables that is created at the whim of a user, but should be seen as a coordinated set of tables that are designed to satisfy some specified requirements. What is needed is a way of developing a database so that the requirements can be clearly identified and so that it is designed in a way that takes those requirements into account.

As we shall see, there are many possible choices that can be made during the design and many rules to guide this work. In trying to decide if some choices are better than others, there are certain desirable properties of a database that need to be considered. Some important properties are as follows:

Completeness	Ensures that users can access the data they want. Note that this includes ad hoc queries , which would not be explicitly given as part of a statement of data requirements.
Integrity	Ensures that data is both consistent (no contradictory data) and correct (no invalid data), and ensures that users trust the database.
Flexibility	Ensures that a database can evolve (without requiring excessive effort) to satisfy changing user requirements.
Efficiency	Ensures that users do not have unduly long response times when accessing data.
Usability (ease of use)	Ensures that data can be accessed and manipulated in ways which match user requirements.

Developing a ‘good’ database with these desirable properties is not easy. Indeed it is quite possible to develop a database which appears to contain all the relevant data but does not have these properties: this has unfortunate consequences as we have already seen.

EXERCISE 4.2

Using the given desirable database properties, very briefly summarise the characteristics that may be expected with a ‘bad’ database.



The importance of database development may be seen simply in terms of avoiding the problems given in Solution 4.2. Database development is not just a matter of creating tables that seem to match the way you see data on forms or reports, but requires a detailed understanding of the meaning of the data and its relationships to ensure that a database has the right properties. This understanding comes from data analysis, which is concerned with representing the meaning of data as a **conceptual data model**. We consider that getting a conceptual data model right is crucial to the development of a good database, and we will discuss this in Section 5 of this block. The rest of this section is about the activities involved in database development.

A conceptual data model is considered in more detail shortly.

4.2 A development life cycle

Database development is just one part of the much wider field of **software engineering**, the process of developing and maintaining software.

A core aspect of software engineering is the subdivision of the development process into a series of phases, or steps, each of which focuses on one aspect of the development. The collection of these steps is sometimes referred to as a **development life cycle**. The software product moves through this life cycle (sometimes repeatedly as it is refined or redeveloped) until it is finally retired from use. Ideally each phase in the life cycle can be checked for correctness before moving on to the next phase. Software engineering is a very rich discipline with many different methods for the subdivision of the development process, and a detailed exploration of the many different ways in which development can be structured is beyond the scope of this course.

It should be noted that in this course we aim to present database development principles and techniques that are common to many development methods, not just the waterfall model.

We start with an overview of the **waterfall model** such as you will find in most software engineering textbooks. Figure 4.4 illustrates a general waterfall model which could apply to any computer system development. It shows the process as a strict sequence of steps where the output of one step is the input to the next and all of one step has to be completed before the next. However, the reality is usually that there is some degree of refinement and feedback as the product proceeds through the development stages. (It would be rare to find that each task is performed perfectly and never needs revisiting – although that is one possibility!)

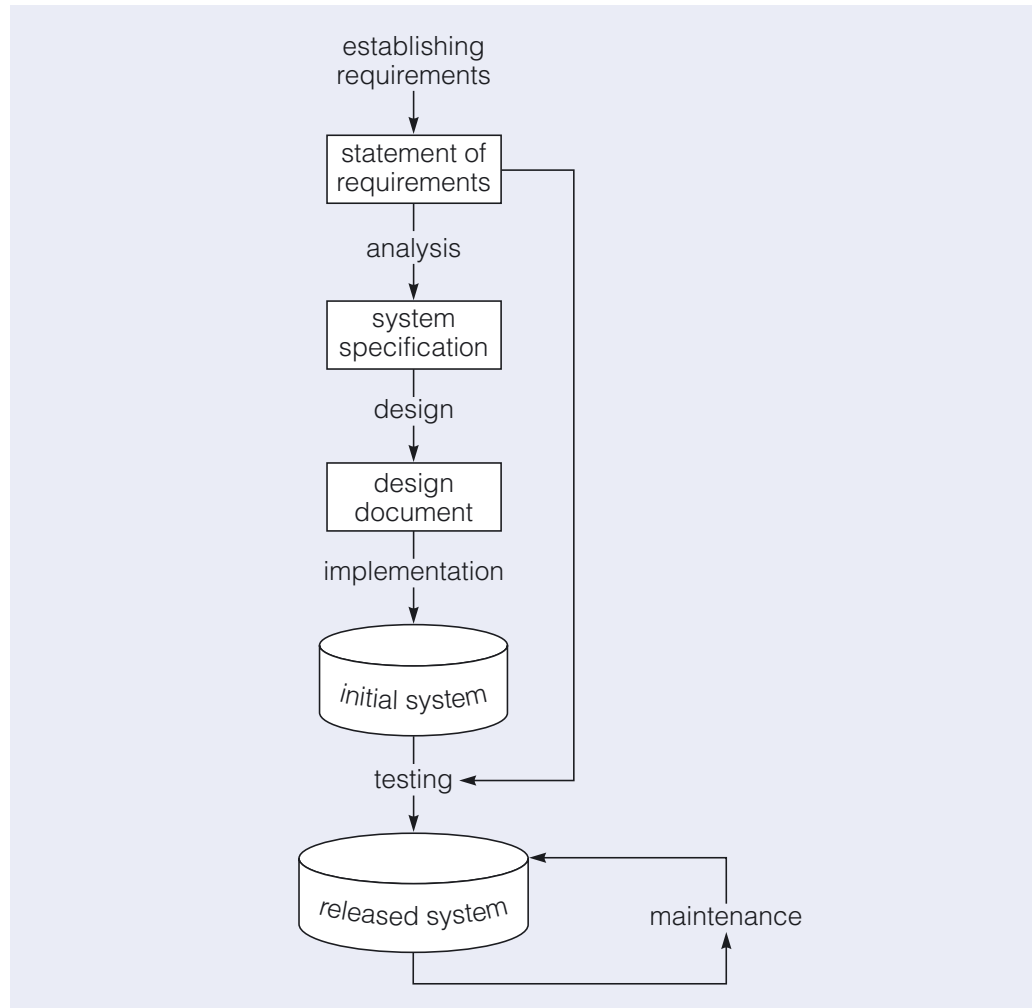


Figure 4.4 A general model of system development

We can use Figure 4.4 as a means of identifying the tasks that are required, together with the input and output for each activity. What is important is the scope of the activities, which can be summarised as follows:

- ▶ **Establishing requirements** involves consultation with, and agreement among, **stakeholders** as to what they want of a system, expressed as a statement of requirements.
- ▶ **Analysis** starts by considering the statement of requirements and finishes by producing a system specification. The specification is a formal representation of what a system should do, expressed in terms that are independent of how it may be realised.
- ▶ **Design** begins with a system specification and produces a design document, which provides a detailed description of how a system should be constructed.
- ▶ **Implementation** is the construction of a computer system according to a given design document and taking account of the environment in which the system will be

operating (for example specific hardware or software available for the development). Implementation may be staged, usually with an initial system that can be validated and tested before a final system is released for use.

- ▶ **Testing** compares the implemented system against the design documents and requirements specification and produces an acceptance report, or more usually, a list of errors and bugs whose correction requires a review of the analysis, design and implementation processes. (Testing is usually the task that leads to the waterfall model iterating through the life cycle.)
- ▶ **Maintenance** involves dealing with changes in the requirements, or the implementation environment, bug fixing or porting of the system to new environments (for example migrating a system from a stand-alone PC to a UNIX workstation or a networked environment). Since maintenance involves the analysis of the changes required, design of a solution, implementation and testing of that solution over the lifetime of a maintained software system, the waterfall life cycle will be repeatedly revisited.

EXERCISE 4.3

The following are problems that have been identified during the testing process in the development of a new system. In which part of the life cycle do you think these problem could have originated and been identified by a thorough review following that stage in the development life cycle?

- (a) The performance of the system is poor – failing to respond sufficiently quickly to meet the stated user requirement of interactive, screen-based use.
- (b) No backup facilities were included to meet the users' requirement of long-term archival of their data.
- (c) No user manuals were provided!

There are many issues outstanding from this brief description, but we do not want to elaborate on the general model any further, simply because it is general and does not explicitly address the development of a database system. However, we can use this framework as the basis for a model of database development which incorporates three assumptions:

- ▶ We can separate the development of a database – that is, specification and creation of a schema to define data in a database – from the user processes that make use of the database.
- ▶ We can use the three-schema architecture as a basis for distinguishing the development activities associated with a schema.
- ▶ We can represent the constraints to enforce the semantics of the data once, within a database, rather than within every user process that uses the data.

Using these assumptions, Figure 4.5 represents a model of the activities and their outputs for database development. It is applicable to any class of DBMS, not just a relational approach.

In the rest of this section we will look at each of these tasks in more detail.



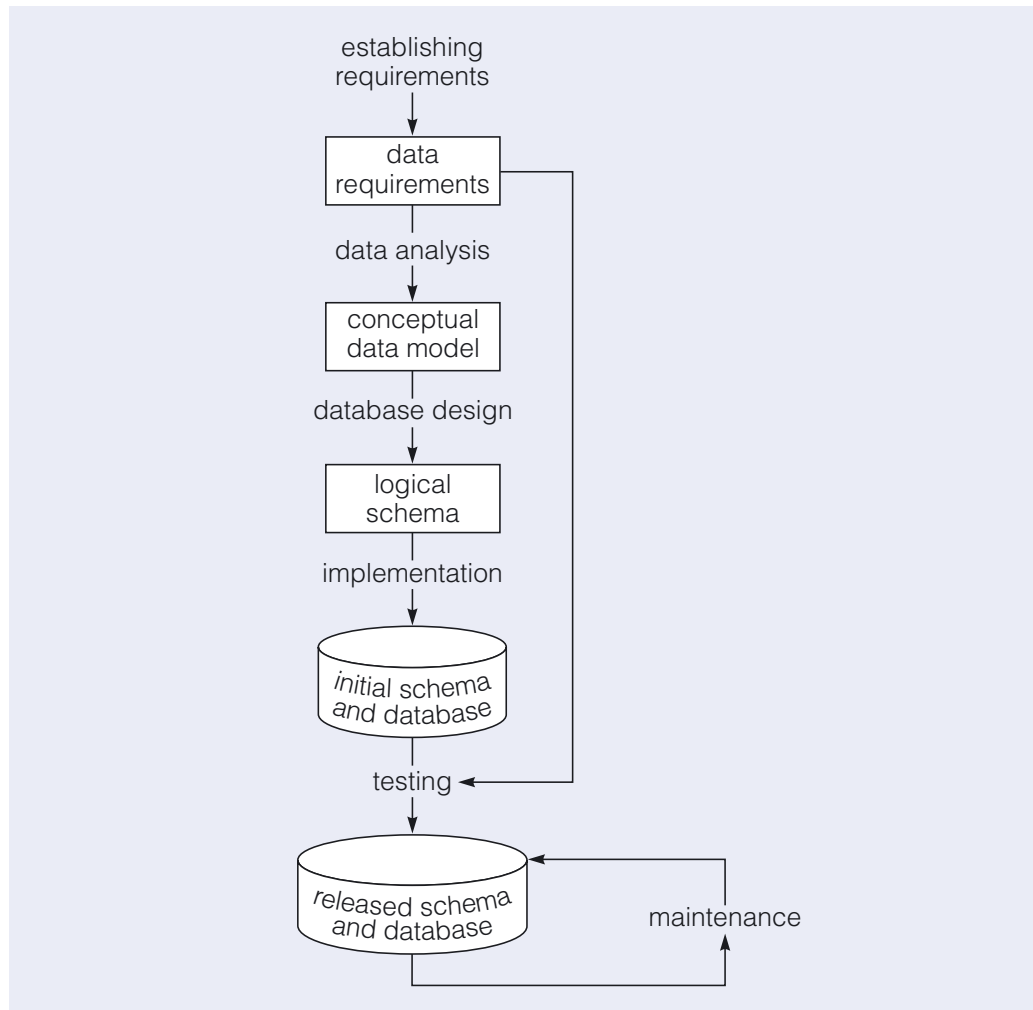


Figure 4.5 Model of database development



EXERCISE 4.4

In the life cycle outline given in Figure 4.5, where do you think the external schema for use by a user process would be involved?

4.3 Requirements gathering

Here we are concerned only with the requirements that relate specifically to the data. Establishing requirements involves consultation with, and agreement among, all the users as to what persistent data they want to store, along with an agreement as to the meaning and interpretation of the data elements. The data administrator has a key role in this process as they overview the business, legal and ethical issues within the organisation.

The data requirements document is used to agree requirements with users, so it cannot be overly formal or highly encoded as it will not be easily understood. The document should give a concise summary of all users' requirements rather than a collection of individuals' requirements as we intend to develop a single shared database.

If you examine the summary database cards, included in your course materials, for the University and Hospital scenarios you will see statements such as the following:

Each course which is available for study is given a course code, a title and a value for credit points – either a 30-point course or a 60-point course. A course may have a quota – the maximum number of students that can be enrolled on the course in any one year; each quota is reviewed regularly and the last date of review is recorded with the quota. A course need not (yet) have any students enrolled on it. Students may not enrol for more than 180 points' worth of courses at any one time.

Notice that the above says nothing about how the data is processed but does state what the data items are, what attributes they have, what constraints apply and the relationships that hold between the data items. For this course, we will usually state the data requirements in this form.

4.4 Analysis

Data analysis starts with the statement of data requirements and produces a conceptual data model. During analysis we want to get a detailed description of the data that will meet the user requirements so we deal with the high- and low-level properties of the data and its use. This includes properties such as the possible range of values that can be permitted for attributes such as the course code, course title and credit points.

The conceptual data model is a concept we haven't yet covered in the course, so the meaning is not likely to be immediately obvious to you. A conceptual data model provides a shared, formal representation of what is being communicated between clients and developers during database development – it is focused on the data in a database, irrespective of the eventual use of that data in user processes or implementation of the data in specific computer environments. That is, a conceptual data model is concerned with the meaning and structure of data, but not with the details affecting how it is implemented.

The conceptual data model is a formal representation of what data a database should contain and the constraints the data must satisfy. This should be expressed in terms that are independent of how the model may be implemented; analysis focuses on 'What is required?' not 'How is it achieved?' Data analysis is a highly skilled task and the analyst has a specialised role that is beyond the scope of this course, where we concentrate on design and implementation. Consequently, we will not consider the analysis task in detail but we will assume that the conceptual data model is the starting point for our database development.

The conceptual data model plays an important role in this course and so we will explain its meaning in detail in Section 5 of this block.

Analyse the data requirements, not the implementation

One of the hardest issues facing an analyst is to perform the analysis without prejudging decisions about implementation. The analysis is purely focused on the data requirements and is not about how those requirements are to be met, or the limitations that might be enforced by the DBMS chosen to host the database. Compromises and enforced limitations resulting from a particular DBMS or computer system should be dealt with during the implementation phase. The requirements gathering and analysis tasks should be performed as if the implementation environment will do everything that needs to be done to satisfy the requirements being specified. Any compromises made at the analysis stage will affect the usefulness of the database and may lead to it failing to meet the user requirements.

You may think of a conceptual data model as being a formal description of the eventual database semantics used to produce a logical schema for a database. Everything in the conceptual data model will appear in the logical schema and everything in the logical schema will be in the conceptual data model. However, a conceptual data model is *not* necessarily expressed in terms of relations or tables, because it will not necessarily depend on the use of a relational DBMS for implementation. Also note that a conceptual data model is a specification used by people for the database design activity; it is not used by any DBMS, nor is it a programming language.

We shall provide examples of conceptual data models in section 5.



EXERCISE 4.5

For each of the following statements decide which processes – requirements gathering or data analysis – would generate the statement as part of the documented output.

- (a) A customer record will allow for the storage of name, UK address, evening and daytime phone numbers, one mobile phone number and as many email addresses as the customer wants to include.
- (b) We need to relate customer orders to their credit card details. If the credit card is invalid we need to know before any orders are accepted.
- (c) A customer must have the opportunity to include a delivery address that is different from the customer's credit card billing address.

4.5 Design

Database design starts with a conceptual data model and produces a specification of a logical schema; this will usually determine the specific type of database system (network, relational, object-oriented) that is required, but not the detailed implementation of that design (which will depend on the operating environment for the database such as the specific DBMS available).

As outlined earlier in this block, our approach on this course will be to use a relational database environment. We can use a relational representation of the conceptual data model as input to the design process. The output of the design stage is a detailed relational specification, the logical schema, of all the tables and constraints required to satisfy the description of the data in the conceptual data model. It is during the design activity that choices are made as to which tables are most appropriate for representing the data in a database, such as for the simple hotel example in Subsection 4.1. These choices have to take into account various design criteria including, for example, flexibility for change, control of duplication and how best to represent the constraints. It is the tables defined by the logical schema that determine what data is stored and how it may be manipulated in the database. We shall consider design criteria in detail in *Block 4*.

When you are familiar with relational databases and SQL it can be very tempting to go directly to implementation after a conceptual data model has been produced. This can be even more of a temptation after completing a course such as this one, where the relational representation of our conceptual data model can be presented in a form that can very easily be converted directly into SQL tables. Such a direct transformation of the relational representation to SQL tables does not necessarily result in a database that has all the desirable properties that we have presented as our data management requirements: completeness, integrity, flexibility, efficiency and usability. A good conceptual data model is an essential first step *towards* a database with these

The relational representation is still independent of any specific DBMS; it is a conceptual data model.

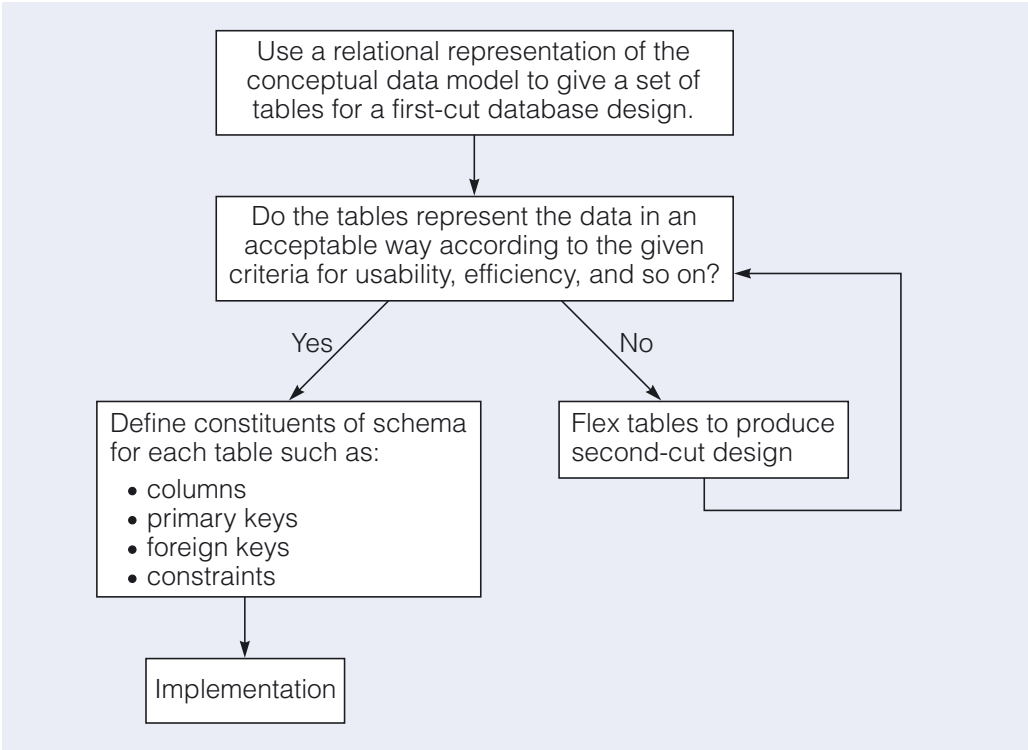
properties, but that does not mean that the direct transformation to SQL tables automatically produces a good database. This first step (sometimes called a **first-cut design**) will accurately represent the tables and constraints needed to satisfy the conceptual data model description, and so satisfies the completeness and integrity requirements, but it may be inflexible or offer poor usability. The first-cut design is then **flexed** to improve the quality of the database design. We will leave a detailed discussion of how the design could be flexed until *Block 4*.

Flexing is a term that is intended to capture the simultaneous ideas of bending something for a different purpose and weakening aspects of it as it is bent.

Using relational theory for a formal design

There will be occasions when you are concerned with the ability to formally prove that a database satisfies given requirements. A relational representation of a conceptual data model can be subject to rigorous analysis using relational theory. This stage, usually omitted in all but the most exacting development environments (such as safety-critical systems), would involve using the formal properties of the relational theory to mathematically prove properties of the conceptual data model that would then be realised in the database design.

Figure 4.6 summarises the iterative (repeated) steps involved in database design, based on the overview given. Its main purpose is to distinguish the general issue of what tables should be used from the detailed definition of the constituent parts of each table – these tables are considered one at a time, although they are not independent of each other. Each iteration that involves a revision of the tables would lead to a new design; collectively they are usually referred to as **second-cut designs**, even if the process iterates for more than a single loop.



Primary keys and foreign keys will be discussed later in the course.

Figure 4.6 Summary of steps for database design

Before we turn to consider implementation there are three general points that you should note as forming the basis of our design approach.

First, for a given conceptual data model it is not necessary that all the user requirements it represents have to be satisfied by a single database. There can be various reasons for the development of more than one database, such as the need for independent operation in different locations or departmental control over 'their' data. In this course we will be considering the development of a single database from a conceptual schema. Our consideration can be applied to the development of several databases, but if the collection of databases contains duplicated data and users need to access data in more than one database then there are further issues related to data replication and distribution that we do not consider in detail in this course.

Second, remember that one of the assumptions about our database development is that we can separate the development of a database from the development of user processes that make use of it. It is based on the expectation that, once a database has been implemented, all data required by currently identified user processes has been defined and can be accessed; but we also require flexibility to allow us to meet future requirements changes. In *Block 4* we will consider how actual data use by user processes might be taken into account during database design and implementation. In developing a database for some applications it may be possible to predict the common requests that will be presented to the database and so we can optimise our design for the most common requests.

Third, at a detailed level, many aspects of database design and implementation depend on the particular DBMS being used. If the choice of DBMS is fixed or made prior to the design task, that choice can be used to determine design criteria rather than waiting until implementation. That is, it is possible to incorporate design decisions for a specific DBMS rather than produce a generic design and then tailor it to the DBMS during implementation.

It is not uncommon to find that a single design cannot simultaneously satisfy all the properties of a good database. So it is important that the designer has prioritised these properties (usually using information from the requirements specification), for example, to decide if integrity is more important than efficiency and whether usability is more important than flexibility in a given development.

At the end of our design stage the logical schema will be specified by SQL data definition language (DDL) statements that describe the database that needs to be implemented to meet the user requirements.

4.6 Implementation

Implementation involves the construction of a database according to the specification of a logical schema. This will include the specification of an appropriate storage schema, security enforcement, external schema, and so on. Implementation is heavily influenced by the choice of available DBMS, database tools and operating environment. There are additional tasks beyond simply creating a database schema and implementing the constraints – data has to be entered into the tables, issues related to the users and user processes need to be addressed and the management activities associated with wider aspects of corporate data management need to be supported. In keeping with the DBMS approach we want as many of these concerns as possible to be addressed within the DBMS. We will consider these concerns briefly in the following material.

In practice, implementation of the logical schema in a given DBMS requires a very detailed knowledge of the specific features and facilities that the DBMS has to offer. In an ideal world, and in keeping with good software engineering practice, the first

stage of implementation should involve matching the requirements of the design with the best available tools for the implementation then using those tools for the implementation. In database terms, this might involve choosing the most appropriate vendor products whose DBMS and SQL variants are most suited to the database we need to implement. However, this ideal world is rarely found and you will frequently find that the hardware choice and decisions regarding the DBMS will have been made a long time before the database design was considered. Consequently, your implementation will involve further flexing of the design to overcome any software or hardware limitations.

EXERCISE 4.6

Summarise the reasons why we want as much of the data management as possible to be addressed within the DBMS.



Realising the design

So far we have been concerned only with the specification of a logical schema. We now need our database to be created according to the definitions we have produced. For an implementation with a relational DBMS this will involve the use of SQL to create tables and constraints that satisfy the logical schema description and the choice of appropriate storage schema (if the DBMS permits that level of control). One way to achieve this is to write the appropriate SQL DDL statements into a file that can be executed by a DBMS so that there is an independent record, a text file, of the SQL statements defining the database. An alternative method is to work interactively using a database tool like Sybase Central (or Microsoft Access), where the forms provided for defining tables avoid the need to remember the syntactic detail of the SQL language. While this may seem to make it easier to realise your database, it will often lead to maintenance problems. In this case, there can be a problem keeping track of exactly how tables are defined and being able to make changes to them, so it is not recommended for large development projects.

Whatever mechanism is used to implement the logical schema, the result is that a database, with tables and constraints, is defined but will contain no data for the user processes.

EXERCISE 4.7

There are some tables in a newly-created database that will contain data. What tables are they and what data do they contain?



Populating the database

After a database has been created, there are two ways of populating the tables – either from existing data, or through the use of the user applications developed for the database.

For some tables there may be existing data from another database or data files. For example, in establishing a database for a hospital you would expect that there are already some records of all the staff that have to be included in the database. Data might also be bought in from an outside agency (address lists are frequently bought in from external companies) or produced during a large data entry task (converting hard copy manual records into computer files can be done by a data entry agency). In such situations the simplest approach to populate the database is to use the import and

export facilities found in the DBMS. Facilities to import and export data in various standard formats are usually available (these functions are also known in some systems as loading and unloading data). Importing enables a file of data to be copied directly into a table. When data is held in a file format that is not appropriate for using the import function then it is necessary to prepare an application program that reads in the old data, transforms it as necessary and then inserts it into the database using SQL code specifically produced for that purpose. The transfer of large quantities of existing data into a database is referred to as a **bulk load**. Bulk loading of data may involve very large quantities of data being loaded one table at a time, so you may find that there are DBMS facilities to postpone constraint checking until the end of the bulk loading.



EXERCISE 4.8

Why would you postpone constraint checking during bulk loading operations?

Those tables for which there is no existing data will be populated using the application processes supporting the normal activities of the organisation, such as enrolling new students on courses or assigning new patients to wards in a hospital.

Supporting users and user processes

Use of a database involves user processes (either application programs or database tools) which must be developed outside of the database development. In terms of the three-schema architecture we now need to address the development of the external schema. This will define the data accessible to each user process or group of user processes. In reality most DBMSs, and SQL itself, do not have many facilities to support the explicit definition of the external schema. However, by using built-in queries and procedures and with appropriate security management it is possible to ensure that access to data by a user process is limited to a tailored subset of the entire database content.

In addition to ensuring that appropriate data access for each user process is achieved, the database developer needs to address other user-related issues. Examples of such issues include: reporting of error conditions, constraint enforcement, automated processing using triggers, grouping of activities into transactions, defining database procedures and functions, and data security (in addition to the general database and user process access control). These topics will be discussed as we progress through the course.

Supporting data management strategies

Most of the development we've covered so far in this block has focused on meeting specific user requirements – that is, ensuring the right data is correctly constrained and made available to the right user processes. However, to support a data management strategy there are various further questions that must be addressed: How frequently should data be backed-up? What auditing mechanisms are required? Which users will be permitted to perform which functions? Which database tools and user processes will be available to users to access data? What level of legal reporting is required? And so on. The data administrator will be involved in setting policy, but the implementer needs to ensure that the right data is being accessed in the right ways by the right users, with appropriate security, record keeping and reporting taking place.

Efficiency: the interaction between design and implementation

When using the three-schema architecture we would like to separate the logical schema, that is, the description of the tables in the database, from the storage schema required for its efficient implementation. This separation represents an ideal that is rarely found in a commercial DBMS. This is most evident when we need to take account of efficiency. When DBMSs lack the ability to separate these concerns it forces efficiency issues to be considered during the database design (by choosing efficient table structures) rather than leaving such decisions until the implementation stage. An initial design for a logical schema may be produced, but its efficiency can be fully evaluated only during implementation. If the resulting implemented database is not efficient enough to meet the processing requirements, it is necessary to return to the database design and consider how the logical schema may be changed to be more efficient. If separation of the logical and storage schemas is possible, and if another storage schema can efficiently implement the logical design, then the logical design may not need revision.

For this reason, some database design methods refer to two separate design stages: *logical* database design and *physical* database design. However, this physical stage is mainly concerned with producing a revised logical schema – that is, specifying SQL tables – so this terminology can be confusing. Efficiency concerns are why the tables of a logical schema may differ from the relational representation of the conceptual data model. (The differences between logical schema description and the relational representation will be considered further in *Block 4*.)

To make design decisions that address specific efficiency concerns requires detailed knowledge of the specific DBMS facilities and hardware systems that will host the database. Consequently, in this course we will address general issues related to design and implementation that can be considered irrespective of the database host environment, and you will need to acquire experience of specific environments by building on this core knowledge.

4.7 Testing

Testing has the aim of uncovering errors in the design and implementation of the database, its structure, constraints and associated user and management support. Testing is usually considered to involve two main tasks – validation and verification. Without adequate testing users will have little confidence in their data processing.

Validation attempts to confirm that the right database has been constructed, with the right characteristics to meet the specified requirements. Validation answers the question: has the right database been developed to meet the requirements?

Verification ensures that the processing steps, constraints and other 'programmed' components of the database (security, backup, recovery, audit trails, etc.) have been correctly implemented and contain no errors in program logic or execution sequences. Verification answers the question: has the database design been implemented correctly?

Of course, testing does not occur only after all the above development steps are complete. Testing is usually applied throughout the stages in the development processes with appropriate reviews to scrutinise the outputs of the development activities. The aim is to identify errors as soon as possible in the development life cycle. This approach will be followed in *Block 4*.

4.8 Maintenance

Databases are one of the more enduring **software engineering artefacts**; it is not uncommon to find database implementations whose use can be traced back for 15 years or more. Consequently, maintenance of the database is a key issue.

Maintenance can take three main forms:

- ▶ Operational maintenance, where the performance of the database is monitored. If it falls below some acceptable standard then reorganisation of the database, usually in the form of adjustments to the storage schema, takes place to ensure that performance is maintained at an acceptable level.
- ▶ **Porting** and implementation maintenance, in which the DBMS, the user processes, the underlying computer system or some other aspect undergoes changes that require the database implementation to be revised. Of course, the extreme form of porting occurs when the database is to be entirely replaced – in which case the entire development life cycle is usually followed using the existing database and its limitations as the starting point for requirements analysis. Adjustments to the storage schema are almost inevitable as new data storage capabilities become available. This can involve both restructuring and reorganisation, depending on the scope of the changes taking place.
- ▶ Requirements change, where the original requirements specification changes, usually because databases are frequently used for purposes for which they were not originally designed. This involves restructuring and typically involves a 'mini life cycle' related to the development of changes to meet the new requirements.

If we follow the three-schema architecture approach we would hope to minimise the impact of change and build systems which are easily maintained.



EXERCISE 4.9

Which of the schemas in the three-schema architecture will be affected by restructuring and which by reorganisation? How could the mapping in the three-schema architecture limit the need for unproductive maintenance when restructuring and reorganisation are performed?

4.9 Summary

In this section we have considered database development as an instance of the waterfall model of the software development life cycle. We have seen that the same activities are required to develop and maintain databases that meet user requirements.

This life cycle will give you the basis for considering the activities and documentation that will be presented in the remainder of the course. We will return to database development when we reach *Block 4* of the course, where we will consider the detailed decision making which ensures that the design and implementation of our database produces a quality product.

In the remainder of *Block 1* we will look at the conceptual data model - this will form the starting point for our development activities. In *Block 2* we look at the development of the relational representation of the conceptual data model before we turn to *Block 3* where we develop our use of SQL for both defining our database implementation and accessing and manipulating data in our database. *Block 4* will consider the specific issues involved in the implementation of the database as well as following the database development life cycle we have presented here.

LEARNING OUTCOMES

Having completed your study of this section of the course you will:

- ▶ Be able to describe the key points of the waterfall model applied to database development.
- ▶ Appreciate the roles of various development artefacts, such as the data requirements document, conceptual data model and such like used to communicate between activities in the database development life cycle.
- ▶ Be able to communicate effectively about aspects of the development of databases.

5

Conceptual data models

5.1 Introduction

The data analysis task involves answering some key questions about the database that we intend to develop:

- ▶ What is the scope of the system we intended to develop?
- ▶ What data is important to support the user requirements?
- ▶ What does that data mean, how is it talked about by the users and what representations do they use?
- ▶ What level of detail is required to adequately model the reality that this data represents?
- ▶ How do the pieces of data relate to each other?
- ▶ What constraints are necessary to maintain the quality of the data?

The answers to these analysis questions lead to the production of a conceptual data model (CDM), which plays an important role in the course so we need to examine it in detail.

We may also use conceptual model as a shorter version of the term conceptual data model.

We use a CDM to describe the data, the associations between the data items and the constraints that limit the data values. We also need to record some of the assumptions and limitations under which the model is valid. If we don't record these at the end of our analysis stage then we may find that the data being modelled, and the database that is produced from that analysis, is used for purposes that the assumptions and limitations would not support.

You can see examples of CDMs on the laminated cards for the University and the Hospital conceptual data models.

A conceptual data model will be a complete, accurate representation of the data requirements of an organisation (or area of interest in an organisation). A CDM is independent of a **physical implementation** that will be used to represent the data – that is, it does not include features of a specific database approach (relational, network, hierarchical, object-oriented, etc.) The model is produced and refined during the analysis stage of the database life cycle and is used to drive the database design stage where the specific database technology is chosen. The conceptual model is intended to give a view of the data without adapting the data appearance to fit into a particular data notation or representation language.

In this course we will use two representations for our CDM: the entity–relationship (E–R) model, which we describe here, and a relational representation of the CDM that we consider in *Block 2*. They both aim to give a clear, unambiguous description of the important aspects of the data. We could use natural language sentences for this description. However, natural language is open to ambiguities and misinterpretation and can result in very long descriptions of the data. The E–R representation we use is intended to be more formal than natural language while still being relatively easily interpreted.

5.2 The domain of discourse and our system

The requirements analysis stage of the development life cycle presented earlier focuses on two key aspects of the database development. It identifies what is to be considered to be of interest and it describes the context and limitations for the data of interest. The aim is to communicate a common understanding of the information represented by the data for those using the data. We considered this in Subsections 1.2, 1.4 and 1.9 of the block, where we highlighted that there can be confusion about the meaning of data when we don't have an agreed context.

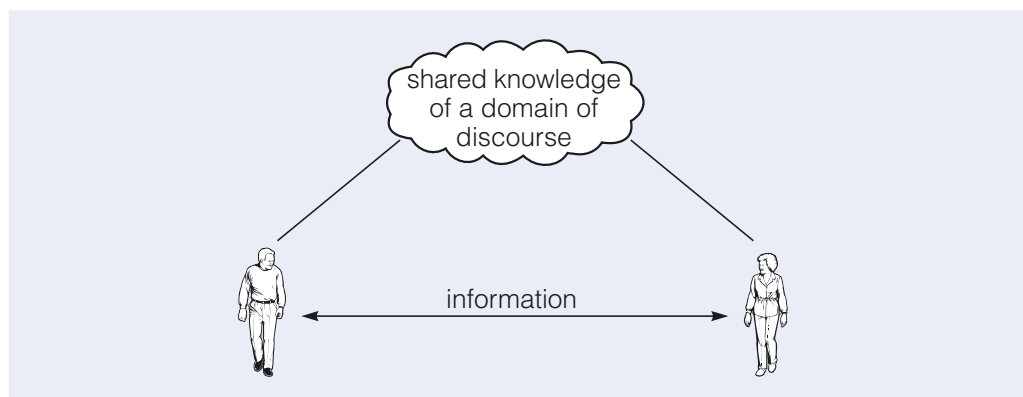


Figure 5.1 The domain of discourse

This context is the **domain of discourse**, as shown in Figure 5.1, a formal phrase that can be simply interpreted as ‘what we agree we are talking about’. The domain of discourse refers to the common understanding that data users will have if they understand the data to have the same meaning. We also mentioned, in Subsection 1.4, that we are very good at deciding on an interpretation for data when one is not supplied. For example, in the sentence ‘Christine is going to Cambridge today’, we will all have made some assumptions that allow us to decide that this is probably about someone travelling to a town on a particular date. However, if we attempt to look deeper at our ‘common sense’ reading of this sentence we might begin to see that we don’t really have sufficient information to be confident in those assumptions. ‘Christine’ might refer to a cat or dog, or even a boat, car or truck. ‘Cambridge’ in the UK, or ‘Cambridge’ in Massachusetts, or is ‘Cambridge’ a verb – as ‘dance’ is in the sentence ‘Christine is going to dance today’? So, one important aspect of the domain of discourse is agreeing how we identify the things we are discussing.

The domain of discourse is also known as the universe of discourse.

We also need to determine the scope of our database – that is, we need to decide what data is of interest to us from all the possible data that we could record. For example, if we decide we need to record facts about an employee then we need to choose what facts we want to record. We could record things like their name, national insurance number, address, phone number, date of birth, salary, date of employment, qualifications, training courses attended, department in which they work, staff number, number of days self-certified as sick days, height, weight, shoe size, glove size, hat size, or any of potentially millions of pieces of data about our employees. The analysis activity will identify the types of facts that need to be recorded and we need to document those so that we know to include them in the database we that develop.

Look at the laminated card showing the University conceptual data model. Even without knowing the detail of what the notation means you should be able to see that the conceptual model is recording details about the domain of discourse and the scope of the database needed to support the university. Obviously the model does not spell

out the most basic issues in the given domain – that is, there is no definition of the meaning of the word university (although in a complex, obscure domain it might be necessary to include such definitions). However, it is also clear that the model is describing the scope and meaning of the data about the university that is of interest. It might also be clear at this stage that this is not a description of every university everywhere – this description includes details of things like telephone tutorials which are not a feature of every university. So the conceptual model on the laminated card is about a specific university – this reflects that our domain of discourse is about a specific university.

After the analysis stage, any data that has not been identified during analysis will be omitted from our data management system and anything that does not have an agreed interpretation is open to ambiguity. A complete conceptual model and supporting documents will include all the data relevant to the user requirements and describe the domain of discourse without any further need for clarification or ambiguity resolution.

Evening Class Register				
Class Registration			Date: 14 - 11 - 2007	
Class	Room	Size	Students	Accepting
Flower Arranging	B4	20	8	No
Name		Student Identifier	Checked	
Jean Blea		S-921	Yes	
Arthur Martins		R-123	Yes	
Fiaz Marzain		N-123	No	
Ariel Daz		S-922		
Trevor Scott		S-923	No	
Sarah Payette		R-234	No	
Faith Barrow		N-129	Yes	
Chi C Cheung		H-110	No	
Jesus Marina		S-300		
Signature: R.T.Narmer				
Date: 7-11-2007				
				Page 1 of 16

Figure 5.2 A page from an evening class register



EXERCISE 5.1

Consider the example document shown in Figure 5.2. It contains a common representation of data that might be of interest to a database developer.

What might the domain of discourse be for this document, and what ambiguities might need to be resolved before a common understanding of the facts that the document represents could be reached?

5.3 Using conceptual data models

Before we move on to the details of how to interpret it, we can look at how the conceptual data model is used.

Our first use for the CDM is to focus attention on the data and its properties rather than the processing requirements of specific applications. One of our key assumptions about the database life cycle is that we consider the data independently of any applications that will use the data. Our conceptual modelling will reinforce this assumption – the conceptual model is focused on the description of the data and the context, constraints and associations that relate to the data. It is not about how any application will process the data, or how we expect the data to be used.

The second use of the CDM is that it provides a working document that is developed during the analysis stage of the life cycle. It usually represents a document that evolves as aspects of the meaning of the data are uncovered and need documenting, or as the scope of the data requirements grows. When an analyst begins collating the data requirements and identifying the data of interest they will make some working assumptions about the data that will need to be clarified with the stakeholders. The conceptual data model will be developed and refined as the data analyst meets with the stakeholders and data users to refine the domain of discourse and determine the appropriate scope for the model. Gradually the assumptions will be replaced with concrete decisions about the model or the limitations of the model. The examples you have seen on the laminated cards are effectively the final version of the conceptual model. As you can see, there are no assumptions remaining to be resolved. There will be additional documents (such as a **data dictionary** and sample forms or reports) that the data analyst will generate and use while talking to the stakeholders, but the conceptual model documentation is the focus of the data analysis activity.

The CDM is also used as a summary document to be passed to the design team. When the analysis is complete any assumptions that the data analyst has made will have been resolved. The model then becomes the descriptive document for the data, constraints and associations between data, which the designer and implementer must realise in a system.

These latter two uses, as a document to review with the stakeholders and as a summary document for the design team, depend on the CDM being used for communication purposes. We can get another view of the role of the CDM by considering the people involved in the development of a database. One of the reasons for identifying the separate activities for system development is that they may be undertaken by different people with different skills and knowledge – in particular, we are concerned with stakeholders, data analysts, data and database administrators, and database developers. There are a number of people who need to communicate about the data to be included in a database. The conceptual model is sufficiently formal to be used by the developer but also sufficiently readable to be used as the basis for discussion with the application developers, end users, security analysts, and others involved in the database development and management roles.

5.4 Understanding conceptual data models

The main aim of this section is to enable you to ‘read’ a CDM. By this, we mean that you understand the notations being used so that you can interpret a given model by describing what it represents. You are not expected to produce a CDM at this stage.

We will now introduce a CDM notation that expresses data requirements in terms of **entity types**, **attributes** of entity types and **relationships** between entity types. This is known as **Entity–Relationship modelling** (abbreviated to E–R modelling), and it is a widely used representation, with some variations in the notations used. Perhaps the main reason for its popularity is the associated entity–relationship diagram (known as an E–R diagram or ERD), which is effective in displaying the important elements of a CDM for communication.

Entity–Relationship model notations

There are several different notations for entity–relationship models; a summary of some notations and their differences can be found on the course website. If you are already familiar with another notation, or see another notation in a textbook you read, then you might find the comparisons in this summary will be useful. If you are not already familiar with entity–relationship models then don’t worry – we will be explaining the notation used in this course in the rest of this section.

We will describe the concepts involved in E–R modelling by means of two extended examples: the University and the Hospital. These are given on the laminated cards titled *University conceptual data model* and *Hospital conceptual data model* that came with your course material. For any statement of data requirements there can be different CDMs that represent the same requirements in alternative ways. In *Block 4* we shall show how some are better than others, but for now we expect you to accept the CDMs we provide without considering alternatives.

5.5 The University conceptual data model

Now would be a good time to ensure that you have the laminated cards featuring the *Scenarios* and the *University conceptual data model*.



We are going to use the University scenario as our **statement of requirements** for the University. There is also a laminated card showing the University conceptual data model; this is the model that we develop from the scenario which we will consider soon. Read through the scenario for the University and be sure that you can understand this as a description of the data requirements that a database for the University will need to meet.

EXERCISE 5.2

Using the text on the laminated card headed *University scenario*:

- (a) Give examples of two facts about each student that the university database will need to record.
- (b) Give one example of a constraint about students and the courses they can take.

Now consider the laminated card for the University conceptual data model. We will work through the essential features of this model as we complete this block of the course – however, for now just consider the various parts of the model shown on this card.

The CDM has five main parts:

Entity–Relationship diagram	This shows the relationships (labelled lines) between the entity types (labelled boxes) that are within our domain of discourse. The properties of the relationships are shown on the diagram, the ‘crow’s foot’ line ending showing relationship degree, and the open and closed circles showing participation conditions.
Entity types	This is a list of the objects of interest (the entity types) in our domain, about which we want to record data. As well as giving a name to the entity types we also name the properties of interest, the attributes, for each entity type.
Additional constraints	The additional constraints record any conditions that, if not enforced, would permit the existence of inconsistent data in the database. Only those constraints that cannot be shown elsewhere in the E–R model are listed here.
Assumptions	These record the working assumptions the analyst has made while developing the model. When working with stakeholders in the ‘real world’ these assumptions would be resolved, but in the academic environment we use the assumptions to record any specific decisions made about places where there may be ambiguity in the requirements.
Limitations	These record the scope of the model. The list of limitations represents a description of expectations about the ‘reality’ of the situation being modelled that indicates the limits under which the entire model is meaningful. They are intended to ensure that the model is only used for purposes that are compatible with the domain of discourse for which it was produced.

We will revisit these components of the model as we consider the various statements in the requirements scenario of the model and use these to introduce the detail in the CDM. You should note that the CDM should be treated as a complete document, that is, the description of data being modelled requires that all parts of the CDM are considered. We will present the CDM components one at a time, for teaching purposes; however, when you are given a CDM you should consider all of the components before attempting to interpret the model.

Along with the CDM there may be additional documents to support the domain of discourse – commonly these include sample documents in use within the domain (such as actual registration forms) and a data dictionary that will include brief descriptions of the items of interest within the domain (such as what a ‘student’ is) and sample data from existing data sources.

Also recall that the method of producing the CDM is through a refinement process. An initial model is gradually constructed and refined until it is considered to be a complete representation of the requirements – so the scenario will be read and reread and the model amended many times before the process is complete.

Let’s start with some straightforward statements in the requirements scenario.

Consider the sentence “The regional offices are known by their reference numbers, 1 through 12; each office has an address, a telephone contact number and an email address.” This sentence tells us a number of things: first, that there is something called

For the Hospital and University examples you will find additional supporting documents installed from the CD-ROM.

regional offices (of which there are several); secondly, that they have a reference number that is used when referring to them, and that we will be needing to record; and thirdly, that for each office we have an address, telephone contact number and email address. Finally it tells us, in a round-about way, that there are at most 12 regional offices (you couldn't have two offices with the same number otherwise they wouldn't be 'known by their reference numbers', and the numbers are limited to the values 1 through 12), which is a constraint that we will need to record in our model.

The thing about which we are interested – the regional office – is an example of an entity type. The details we want to record about regional offices, address, telephone contact number, email address, are known as the attributes of the entity type. The attribute that allows us to refer to a regional office, the reference number, is known as an identifying attribute (or identifier, for short). The fact that the reference numbers are 1 through 12 is a constraint. We'll come back to considering the constraints later in this section.



EXERCISE 5.3

- (a) What attributes of the **Staff** entity type will our database need to record?
- (b) What is the identifying attribute of the **Staff** entity type?

Entity types

Now we can formalise our definitions of entity types, attributes and identifiers. We'll start with a definition of an **entity**:

An entity represents a thing that has meaning in a given context and about which there is a need to record data.

In this case, it is the University that provides the organisational context for the entities that we consider. So a student is a meaningful thing about which we want to record data. We might say one entity occurrence represents a particular student with student identification number s10 who is named Urbach. There will be many students about whom we will want to record information; each student will be represented by a different entity occurrence because the University needs to record data about individual students.

However, a CDM is not concerned with the individual entity occurrence, but with ways of classifying entities for which we want to record the same data. For this purpose we use an **entity type**, defined as follows:

An entity type defines the properties common to all occurrences of a collection of entities.

In other words, exactly the same description is used for each entity of the given type. So we can model the **Student** entity type, which allows us to record any number of entity occurrences of that type.

An entity type is given a name to refer to it. In the University CDM we have identified **Region, Student, Staff, Course, Quota, Enrolment, Assignment, Examination and SpecialCircumstancesForm** as the entity types. So, **Student** is the name of the entity type that defines the need to record data about each student's identifier, name, address, email address and date registered with the University.

Our convention in naming entity types is to use a short but meaningful name with the first letter capitalised. We do not allow spaces in names but capitalise the first letter of any subsequent word if the name of the entity type is formed from several words.

Note that in this context a 'thing' may be an abstract thing, such as a meeting, or a tangible thing, such as a person or place or event.

Note that we could have used the name **RegionalOffice** in place of **Region** if we had chosen to.

EXERCISE 5.4

What do you think that the entity type **Assignment** represents in the University CDM?



While it is entity types that are important to a CDM, and defined by it, that does not mean entities have to be ignored because, after all, it is entities which correspond to the things that really exist – entity types are a convenient device for representing what we want to know about collections of entities. For a given entity type, the individual entities of that type are referred to as occurrences (sometimes called instances) of the entity type.

It is often useful to think of what some occurrences of an entity type might be, as a way of understanding a CDM, even though occurrences do not appear as part of the model. Since entity occurrences represent the actual things in the world that we want to record data about, we can think of each entity occurrence as an **assertion** about the existence of something in the world. In the University CDM we have the entity type **Student**, an occurrence of which represents data about an individual student. For example, Figure 5.3 lists six entity occurrences of entity type **Student** representing six students. The first occurrence of the **Student** entity type shown in Figure 5.3 (the first row in the list) can be seen to be an assertion that there exists a thing that we refer to as a student who has a student identification number s01 and a student name Akeroyd with the stated address, email address and registration date.

Many people use the term 'entity' to refer to both entity types and **entity occurrences**. This is a cause of confusion so we will use 'entity' only to refer to entity occurrences.

An assertion is a statement or claim that something is true.

StudentId	Name	Address	EmailAddress	RegistrationDate
s01	Akeroyd	12 Anystreet, Anytown	Akers@tahoo.fake.com	Nov 23, 1999
s02	Thompson	8 High Street, Lowville	Pjay@thompson.fake.com	Oct 12, 2004
s05	Ellis	34 Globe Road, Smallville	G.Ellis@fake.fake.co.uk	Oct 14, 2003
s07	Gillies	29 Straight street, Angletown	Gillies@address.fake.net	Dec 2, 1993
s09	Reeves	34 The Crescent, Curville	T.Reeves@nnet.fake.com	Dec 14, 2004
s10	Urbach	22 Hilltops, Valley Town	B.Urbach@tnet.fake.fr	May 5, 2003

Figure 5.3 Six entity occurrences of the entity type **Student** (these aren't given in the scenario, but you can find them in the University sample data set in the University supporting document installed from the course CD-ROM)

EXERCISE 5.5

The statement of requirements for the University example includes a number of references to tutors and telephone tutors. Explain how they are represented in the University CDM and give a reason why there is no need for the entity types **Tutor** and **TelephoneTutor**.



The E–R diagram in a CDM is a pictorial representation of entity types and their relationships (we shall look at relationships shortly). In an E–R diagram, an entity type is depicted by means of a round-cornered box with the name of the entity type written inside the box, as in Figure 5.4 for the **Staff** entity type.

As well as being in an E–R diagram, the entity types are also listed with their attributes and may be referred to in the other parts of the CDM.



Figure 5.4 The pictorial representation of the **Staff** entity type



EXERCISE 5.6

It is essential for a CDM to be consistent within itself. Give a simple rule to ensure consistency between the entity types in the E–R diagram and those listed with their attributes in the Entity types section of the CDM.

Attributes

Here is a formal definition of the **attribute** construct:

An attribute is a component of an entity type that represents a single-valued property of entities of that type.

For example:

- ▶ **StudentId**, **Name**, **Address**, **EmailAddress** and **RegistrationDate** are attributes of the **Student** entity type;
- ▶ **StaffNumber**, **Name**, **Address**, **EmailAddress** and **Telephone** are attributes of the **Staff** entity type;
- ▶ **CourseCode**, **Title** and **Credit** are attributes of the **Course** entity type.

Note that each attribute in each occurrence of an entity type can only hold a single value. For example, it would not be possible, in the University model, for a **Student** or **Staff** occurrence to record two email addresses.

The naming of attributes follows the same convention that we used for entity types. In addition, we require that no two attributes of an entity type have the same name, but the same name can be used for attributes in different entity types. If there is potential for confusion then we qualify the attribute name with the name of the entity type. For example, referring to the **Name** attribute in the University model would be ambiguous. In such cases we use **Student.Name** and **Staff.Name** for the **Name** attribute of the **Student** entity type and the **Staff** entity type, respectively, and so avoid the ambiguity.

Appropriate naming of entity types and attributes should enable you to relate these components of a CDM to the data requirements and the domain of discourse, so providing a context for their meaning. However, further description of entity types and their attributes may be necessary to prevent misunderstanding and these can usually be found in a data dictionary or other support documentation for the conceptual model.



EXERCISE 5.7

- (a) By reference to the University statement of data requirements, describe what the attribute **Credit** (of the **Course** entity type) represents.
- (b) Suppose we wanted to record information about the date on which a student was informed of their tutor allocation. To which entity type in the University model would you add the **TutorAllocationDate** attribute? Why did you make that choice?

Note that the term 'attribute' is a type concept, but we do not refer to 'attribute type'. However, as with entity types, we have to be careful to distinguish between an attribute, such as **Staff.Name**, and occurrences of that attribute, which we refer to as attribute values, such as **French**, **Redhead**, **Heathcote**, **Jennings** and **Lai**. We also need to emphasise that the value of an attribute for a given entity is a single value (such as **French** or **Redhead** or **Heathcote** or **Jennings** or **Lai**) and not several values.

One important E–R modelling characteristic is that we specify how to distinguish occurrences of entities of the same type, one from another. This can be done by means of an **identifying attribute** (or **identifier**), which is a particular attribute with the property

of uniqueness such that no two entities of the same type have the same value for that attribute. If no such single attribute can be found, a combination of attributes is specified which together have this uniqueness property. For example:

- ▶ the attribute **StudentId** is the identifier for the **Student** entity type;
- ▶ the attribute **StaffNumber** is the identifier for the **Staff** entity type;
- ▶ the attribute pair (**StudentId**, **CourseCode**) is the identifier for the **Enrolment** entity type.

We shall consider further examples in which the identifier of an entity type needs to be a combination of attributes shortly.

An identifier represents part of the meaning of the data for an entity type – it is a **semantic feature**. There are three roles for an identifier: as an assertion of existence, as a reference and as a uniqueness constraint. For example, an occurrence of **Student** with a particular value, such as **s02**, for the identifier **StudentId**, is an assertion that a student exists who has been registered with that student identification number. Also, if we know a value for the identifier **StudentId**, such as **s02**, we can use it to refer uniquely to the specific student who is registered with that student identification number. Looking at Figure 5.3, we can tell that this particular student's name is **Thompson**. Finally, specifying that **StudentId** is an identifier constrains **Student** occurrences to those where the values of the **StudentId** attribute are different – that is, no two students can have the same student identification number.

During conceptual modelling we want to use the attributes of the entity types that are already in the data requirements, sometimes called **natural attributes**. In particular we don't invent attributes that we add to the entity types solely to act as identifiers. If we did this we might overlook interesting properties of the model that these artificial identifiers have hidden. (In *Block 4* we will consider that the design and implementation might lead us to create additional attributes for efficiency purposes – but first we want to complete our data analysis and data modelling without adding more details than necessary.)

EXERCISE 5.8

Why would **Student.Name** not be a suitable identifier for the **Student** entity type?



In our CDM we use a notation in which an entity type and its attributes (including the identifier) are specified as follows:

EntityTypeName(IdentifyingAttribute(s), NonIdentifyingAttribute1,
NonIdentifyingAttribute2, NonIdentifyingAttributen)

where the identifying attribute, or combination of attributes, is underlined.

For example, here is the **Student** entity type:

Student (StudentId, Name, Address, EmailAddress, RegistrationDate)

This specification allows us to list the data of interest for entities of **Student** type – that is, for each student we expect to record a student identifier, name, address, email address and date of registration at the university. If you review the original requirements specification then you should see they are consistent with the given data requirements.

If you look at the laminated card showing the University CDM you will see the full list of the entity types and their attributes with the identifying attributes underlined.

EXERCISE 5.9

- (a) On the supplied CDM for the University, the identifier for **Enrolment** consists of two attributes of this entity type. By considering the purpose of an identifier, to



distinguish each occurrence of an entity type, show that neither individual attribute of the **Enrolment** identifier can be the identifier by itself.

- (b) Consider the entity occurrence **Enrolment(s01, c4, 'Jan 12, 2005')**. What assertion about the University does this occurrence represent?

We have explicitly made the limitation (1.2 in the *University conceptual data model*) that for each individual student, details of at most one enrolment for a specific course can be recorded. Therefore the combination of **StudentId** and **CourseCode** together is appropriate as an identifier of **Enrolment**, being able to distinguish each occurrence of **Enrolment**, and providing a constraint to ensure that a student cannot be recorded as enrolling on a given course more than once.



EXERCISE 5.10

What is the significance of the existence of a given **Enrolment** occurrence to the university authorities?

A further example of a multiple-attribute identifier is the combination of **StudentId**, **CourseCode** and **AssignmentNumber** used to identify instances of **Assignment**, which ensures that there is at most one occurrence of a numbered assignment for a specific student enrolled on a specific course.



EXERCISE 5.11

Suppose the data requirements for the University are modified to include the following. The date a student submits an assignment on a course is recorded – this will be achieved by introducing a new attribute, **SubmissionDate**. For this new attribute, give the entity type to which it belongs.

Before moving on to relationships we should look at a characteristic of entity types that we will return to later. Notice that the identifier of the **Enrolment** entity type includes attributes that are the identifiers of other entity types. In particular, the **Enrolment** identifier (**StudentId**, **CourseCode**) includes the **Student** identifier (**StudentId**), and the **Course** identifier (**CourseCode**). Entity types like **Enrolment**, where subsets of their identifying attributes are also identifiers of other entity types, are sometimes called **weak entity types**. They are referred to as ‘weak’ because instances of that entity type cannot exist without there being instances of other entity types. An **Enrolment** instance requires that a **Student** and **Course** exist so that the values for the **Enrolment** identifier are meaningful – that is, the identifier refers to the enrolment of a real student on a real course. In an extreme case, if there were no students in the University then there could be no **Enrolment** instances. However, a **Student** might exist without there being an **Enrolment** for that **Student**. **Student** can exist without a dependence on any occurrences of other entity types so it is an example of a **strong entity type**.



EXERCISE 5.12

Weak entity types have attributes in their identifier that are also the attributes of the identifiers of other entity types. Using this description, can you identify other weak entity types in the list of the entity types given for the University CDM?

We will return briefly to weak and strong entity types after we have considered relationships, as there is a subtle interaction between relationships and the values of the identifiers of weak entity types.

We have emphasised that this block does not aim to teach you how to produce a CDM, but Exercise 5.11 is just a simple example of how a model can be developed. Our choice of name for some of the attributes, such as **ExaminationLocation**, may have made it obvious as to which entity type they should appear in. For **SubmissionDate** it may not be so clear, but it helps if you consider how you would identify the entity type it should appear in. You may think of it as being data for a specific student, so we need **StudentId** to be part of the identifier, and on a specific course, so we need **CourseCode** to be part of the identifier, and for a specific assignment, so we need **AssignmentNumber** to be part of the identifier. Taken together, this leads to it being an attribute of **Assignment**, as we gave in Solution 5.11.

To summarise the CDM definitions that we have covered so far, we have identified the entity types, their attributes and identifiers, made some assumptions and recorded some limitations after our first read through the requirements scenario. There is still a lot of information about the data requirements that we need to capture.

We will now look in detail at the associations between entity types. We begin with an overview of relationships and then consider how the properties of the relationships can be identified in the requirements and recorded in our CDM.

Relationships

Here is the definition of a **relationship** as applied to E–R conceptual data models:

A relationship is an association between entities that has meaning in a given context and which needs to be recorded.

In the same way that a CDM includes entity types, which define the properties common to a collection of entities, we want to define properties common to a collection of relationships – that is, a type of relationship. The most basic common property is that of relating entity occurrences, so we have a type of relationship between the entity types **Region** and **Student** to which we have given the name **Manages**. This appears in the requirements scenario as ‘... students are managed by regional offices’. Notice that it is the occurrences of the entity types that are related – a region manages a student. However, in the CDM we include a relationship type, **Manages**, between the entity types, **Region** and **Student**.

The naming of types of relationships follows the same convention as given for entity types. Usually a verb is used for relationship names. Note that relationships have the same distinction between types and occurrences as for entities, but we refer to relationships for both since there is not the same potential for ambiguity. Thus **Manages** is the name of the relationship between the entity type **Region** and the entity type **Student**, representing information about the association that an occurrence of a **Region** manages an occurrence of a **Student**.

EXERCISE 5.13

Identify an association between **Staff** members and **Course** mentioned in the University requirements scenario.



There is, of course, more to a relationship than just its existence. If you read through the scenario looking for relationships you will see that there is more information than just the relationship name and the entities that are associated by the relationship. Two important properties of relationships are their **degree** and **participation conditions**.

The degree of a relationship represents the number of occurrences of each entity type that can be involved in a relationship. The relationship participation conditions represent whether every occurrence of each of the entity types participates in a relationship occurrence, or just some of the occurrences of the entity types participate. We consider these properties of relationships in the following subsections.

Relationship degree

Look at the other statements on the University CDM card that give information about students being managed by regions. ‘Each student is managed by one region’ – this statement gives us some information about the degree of the **Manages** relationship. It tells us that at no time will a student be managed by more than one region. (Note: it also tells us that no student is ever ‘unmanaged’, which we will consider when we look at the relationship participation conditions in the next subsection.) Our common sense also gives us the fact that each region will manage more than one student at a time – otherwise it would be a very small university! Of course, if we weren’t sure of our common sense reading then we could add an assumption to this effect and resolve it later.

If we had details of the instances of **Student** and **Region** and drew connections between the regions and the students they managed, we might get a diagram similar to that in Figure 5.5, known as an occurrence diagram. Note that the diagram shows only the identifying attributes of the occurrences of **Region** and **Student**, but remember that this allows us to identify uniquely both regional office and student involved.

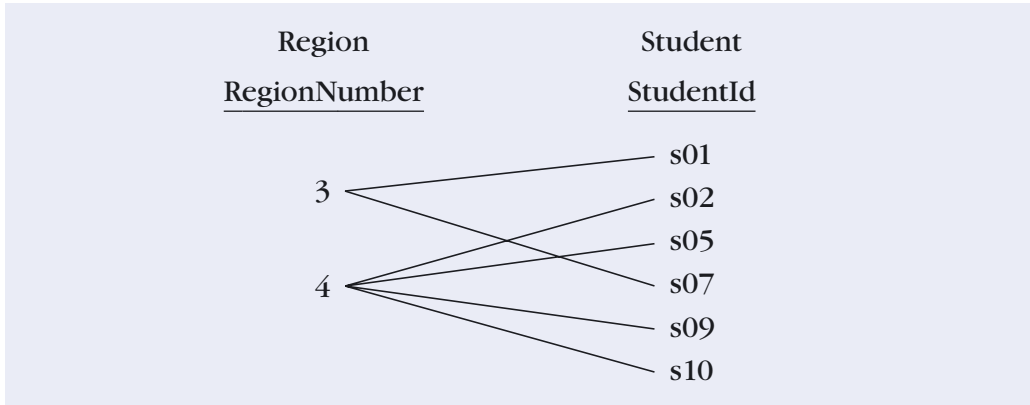


Figure 5.5 Six relationships between occurrences of Region and Student

Figure 5.5 is typical of occurrences of the **Manages** relationship. It shows that each region can be associated with several students, but each student is associated with only one region. The student identified by **s01** is associated with (managed by) the region identified by **RegionNumber 3**; the region identified as **RegionNumber 3** is associated with (manages) the students identified by the **StudentId** instances **s01** and **s07**.

The number of instances of each entity type that can be involved in the association is an important feature of a relationship, known as the **degree** of a relationship. In this case the **Manages** relationship has a degree of **one-to-many** (usually written as 1:n). Thus the relationship has a ‘to-one’ end (written as :1) at the **Region** entity type, and a ‘to-many’ end (written as :n) at the **Student** entity type.

Notice that the 1:n degree of a relationship is directional – one occurrence of **Region** is related to many occurrences of **Student** but not the other way round. We can generalise this by saying that each relationship has two ‘readings’, one for each ‘direction’. So, the degree of the **Manages** relationship written out in concise English requires two sentences, one for each direction:

- Each occurrence of the Region entity type Manages one or more occurrences of the Student entity type.
- Each occurrence of the Student entity type is Managed by one occurrence of the Region entity type.

We usually omit ‘occurrence of’ and ‘entity type’ for the slightly more informal representation:

Each Region Manages one or more Students.

Each Student is Managed by one Region.

The relationships are shown in the E–R diagram as lines between entity types, with the name shown adjacent to the line. The notation we use for indicating that a relationship has

a $:n$ degree at one end is known colloquially as a ‘crow’s foot’. This is shown in Figure 5.6 at the **Student** end of the line depicting the **Manages** relationship. This indicates that the relationship is $:n$ with respect to **Student**, while the lack of a similar symbol at the **Region** end of the line indicates that the relationship is $:1$ with respect to **Region**.



Figure 5.6 Degree of the **Manages** relationship (one-to-many) depicted on the E–R diagram

We usually name relationships in a meaningful way so that they can be ‘read’ from the entity type at the $:1$ end to the entity type at the $:n$ end, which we refer to as the $:n$ direction. For example, the **TelephoneTutoredBy** relationship represents the information that ‘a course is telephone tutored by many staff’. However, it is important to remember that a relationship represents information about an association which has meaning in both directions. That is, viewed in the $:1$ direction, **TelephoneTutoredBy** also represents the information that ‘a staff member is the telephone tutor on one course’. The second direction is not giving us new information, it is just an alternative way of expressing the association between staff and courses, and so should not be represented by a second relationship – this would be unnecessary duplication.

Occasionally the names of the relationships and the attributes can lead to quite clumsy sentences if used directly as taken from the diagram – but their meaning can usually be captured with minor adjustments to the names.

The degree of a relationship is not an observation of the relationship occurrences as they exist at some instant in time, but it reflects an important part of the meaning of the relationship which holds for all possible times. We are modelling not just a snapshot of data at a particular time, but the properties of the data that hold in the domain of discourse at any time. The degree therefore asserts something about the allowable ways in which any occurrences of the two entity types *may* be associated. For example, the $1:n$ **Manages** relationship asserts that a **Region** occurrence *may* be associated with *many* **Student** occurrences via the relationship, but a **Student** occurrence can be associated with no more than *one* **Region** occurrence via the relationship. Note that the first part of this assertion (coming from the $:n$ part of the degree) – specifically, that a **Region** occurrence may be associated with many **Student** occurrences – tells us what is possible but does not constrain the allowable associations between the two entity occurrences, since ‘many’ can be any number (including *zero* – we will address this special case when we consider the relationship participation conditions in the next subsection). In contrast, the second part of the assertion (coming from the $:1$ part of the degree) is a constraint since it forbids any association of a **Student** occurrence with more than one **Region** occurrence (it says that a student can never be associated with two regions by the **Manages** relationship).

Other notations for indicating the degree of a relationship are used by other conceptual modelling schemes.



EXERCISE 5.14

Give two sentences that describe the **Submits** relationship in the University E–R model (ignore the circles on the entity–relationship diagram lines for this question).

Now consider the statement in the requirements scenario about the special circumstances form: ‘A student can submit, along with an assignment or following the examination, a special circumstances form.’ Clearly there are relationships between the **Assignment** entity type and the **SpecialCircumstancesForm** entity type, and between the **Examination** entity type and the **SpecialCircumstancesForm** entity type. (In our model we’ve named these relationships **AssignmentIncludes** and **ExamIncludes** because **Submits** was already in use.) We know that a student gets to submit only one special circumstances form for each assignment and that a special circumstances form is for only one assignment. Figure 5.7 shows some example occurrences of the **AssignmentIncludes** relationship that satisfy these requirements, using some values of the identifier of each entity type to indicate occurrences.

Assignment			SpecialCircumstancesForm
(StudentId, CourseCode, AssignmentNumber)			FormNumber
s09	c2	1	SC754
s10	c4	1	SC458
AssignmentIncludes			

Figure 5.7 Some occurrences of the **AssignmentIncludes** relationship type

If we produced the pair of sentences for the **AssignmentIncludes** relationship we would get a reading in one direction that:

Each Assignment Includes one Special Circumstances Form.

In the other direction we get that:

Each Special Circumstances Form is Included with one Assignment.

This is an example of a relationship of degree **one-to-one** (1:1). If you look at the University CDM entity–relationship diagram you will see that this is shown by a relationship line with no crow’s feet at either end. (Note: we will consider that some assignments do not have special circumstances forms submitted when we consider participation conditions in the next subsection.)



EXERCISE 5.15

Give the pair of descriptive sentence forms for the **TelephoneTutoredBy** and the **Tutors** relationships in the E–R diagram of the University CDM.

Note: *m:n* is written as *m:m* in some textbooks and notations.

There is one more degree of a relationship, the **many-to-many** relationship (also written as *m:n*).



EXERCISE 5.16

At present there are no examples of the many-to-many relationship degree in the University CDM supplied, so let’s extend our scenario requirements with the following additional sentences: ‘A course is part of a degree programme. Each degree programme has several courses, and each course could be on several degree programmes. Degree programmes are identified by their code (e.g. B14) and have an Award description (e.g. BSc Computing).’

Assume that the stated extensions to the University scenario are to be modelled using the relationship named **PartOf**.

- (a) What would an occurrence diagram show for two courses, with course codes **c2** and **c4**, if both of these courses were known to be part of the degrees identified as **B14** and **B16**?
- (b) What would be the readings for the two sentences representing this relationship?
- (c) What would be the E–R diagram representation for the degree of the relationship **PartOf**?

We will return to many-to-many relationships later, in Subsection 5.6 of this block, when we will discover why there are none in the original University model.

There is one final relationship representation that needs to be considered. So far, all our relationships have been between two different entity types. But, what if we had a relationship that referred to the same entity type at both ends? Consider another extension to our University scenario – this time to allow staff members to mentor other staff members. A mentor is an experienced member of staff who can support another staff member. If we assume each staff member has one mentor, and an experienced staff member can mentor many other staff members, then this will make **Mentors** a one-to-many relationship from **Staff** to **Staff**.

Such a relationship, where both ends of the relationship involve the same entity type, is known as a **recursive relationship** and has no special properties that are different from other relationships but it requires more care in its interpretation and description. In addition, when ‘reading’ the relationship we usually add a role description to each entity type. In this case the *mentored* and *experienced* labels will help clarify the roles.

- Each experienced Staff member Mentors many mentored Staff members.
- Each mentored Staff member is Mentored by one experienced Staff member.

The E–R diagram representation is shown below.

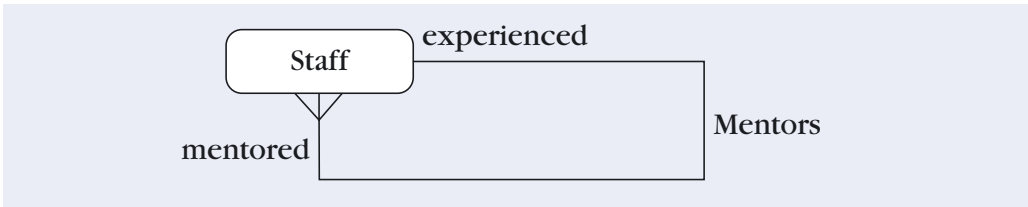


Figure 5.8 Degree of the Mentors relationship

EXERCISE 5.17

Draw an entity–relationship diagram that represents the degree of the relationships between the entity types **House** and **Person** from the following statements that might appear in a requirements document:

Houses can be bought and rented by people (persons). Each house is bought by one person. Each person buys one house. Each house is rented by many persons. Each person rents one house.

(Note: don’t try to reinterpret the given statements to better match your understanding of reality. You were asked to represent the given statements, not to argue about the domain of discourse.)



Before we look at relationship participation conditions there is one final aspect of relationships representation in the CDM to note. Earlier in this section, weak and strong entity types were mentioned. Look at Figure 5.9, which shows an example occurrence diagram for the **EnrolledIn** relationship. Notice that each occurrence line begins and ends on instances of **Student**, on the left-hand side, and **Enrolment**, on the right-hand side, that have the same values for **Student.StudentId** and **Enrolment.StudentId**. Now think about what the weak-strong association means – if there is an enrolment which includes a student identifier, then there must exist a student with that identifier.

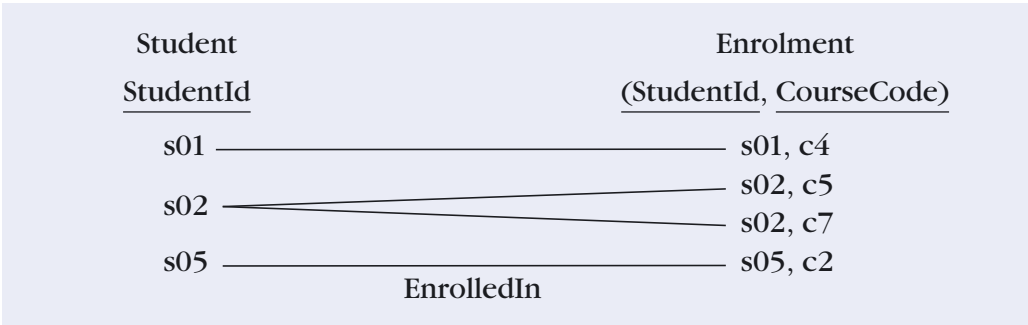


Figure 5.9 Occurrence diagram for the **EnrolledIn** relationship

So what exactly does the relationship **EnrolledIn** mean? Well, it means that a student may have enrolled on a course and the **Enrolment** instance represents that student on that course. Now look at Figure 5.10.

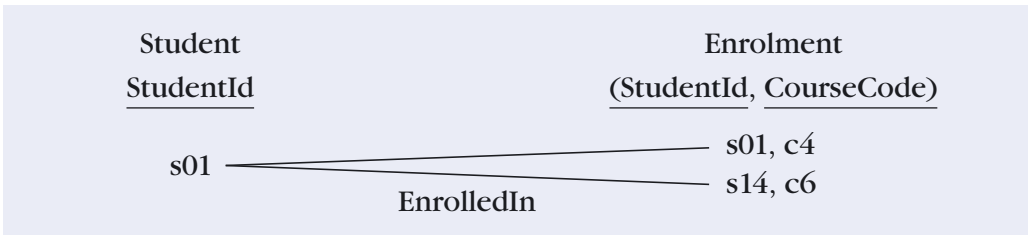


Figure 5.10 Invalid occurrence diagram for the **EnrolledIn** relationship

Could we ever get into a situation described by the occurrence diagram in Figure 5.10? It implies that a student can enrol on a course and be another student! Does this mean student **s01** is also student **s14** when they enrol on the course **c6**? In general this isn't a sensible thing to allow and in fact we would want to constrain the relationship occurrences so that this doesn't happen. You can see examples of the wording (admittedly quite convoluted wording) of the additional constraints in the University model that ensure situations similar to Figure 5.10 don't occur. Constraints c.11, c.12 and c.13 prevent weak entity occurrences from being related to inappropriate instances of the strong entity types on which their existence depends. We revisit this issue in *Block 2*.

Relationship participation conditions

We can now consider the filled and open circles that appear on the relationship lines on the University E-R diagram (refer to your University CDM card). These symbols represent another aspect of the meaning of a relationship, specifically related to whether every occurrence of each of the entity types participates in a relationship occurrence, or just some of the occurrences of the entity types participate.

Our occurrence diagrams have, so far, shown all occurrences of both entity types involved in the relationships. But consider the additional information we have about, for

example, staff members being telephone tutors: ‘Each staff member ... may or may not act as a telephone tutor for a course.’ If we had a staff member who wasn't a telephone tutor, how would this appear in an occurrence diagram? This situation is shown in Figure 5.11.

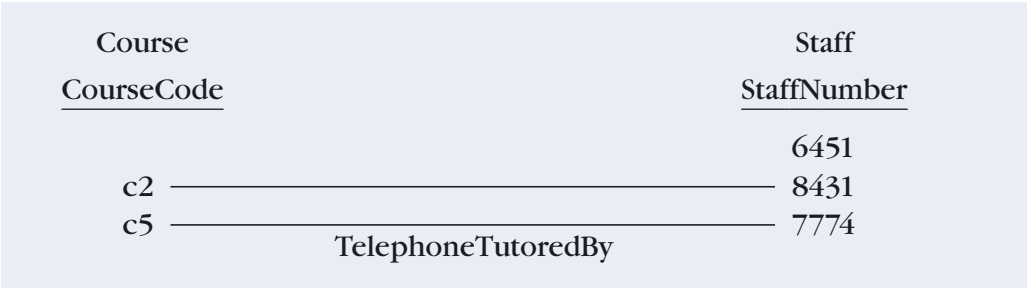


Figure 5.11 Occurrence diagram for the TelephoneTutoredBy relationship

Figure 5.11 shows that the staff member known as **6451** is not involved in the **TelephoneTutoredBy** relationship (there is no line connecting that staff number to a course code). But we haven't any unattached instances of **CourseCode**. So, could an unattached course code ever occur? According to the scenario the answer is no; we are told in the scenario that every course has a telephone tutor allocated. Clearly there is something significant we need to record about the involvement of instances of an entity type with the relationship.

For each of the entity types in each relationship there is what is known as a **participation condition**. We say that an entity type has a participation condition of **optional** if entities of that type *need not* be involved in an occurrence of the relationship; so the **Staff** member involvement being optional with respect to the **TelephoneTutoredBy** relationship. Conversely, a participation condition of **mandatory** means that each entity of that type *must* be involved in an occurrence of the relationship. For example, **Course** has mandatory involvement in the **TelephoneTutoredBy** relationship.

An optional participation condition is depicted on an E–R diagram by an open circle on the relationship line while mandatory is depicted by a filled circle. Look at the **Manages** relationship shown in the University CDM: we can interpret this as saying that **Manages** is mandatory with respect to **Student** (a filled circle at the **Student** end of the line depicting the relationship) and optional with respect to **Region** (as defined by the open circle at the **Region** end of the line). So there are some regions that might not have any students to manage, but if there is a student there must be a region that manages that student.

As with the degree of a relationship, participation conditions reflect part of the meaning of a relationship which holds for all possible occurrences, asserting whether any occurrence of each of the related entity types is allowed to be *not* involved. **Manages** being optional with respect to **Region** means that a **Region** entity may be associated with a **Student** entity via the **Manages** relationship, but does not have to be. **Manages** being mandatory with respect to **Student** means that a **Student** entity must be associated with a **Region** entity via the **Manages** relationship. This represents the constraint that a student always has at least one managing region.

Notice that a participation condition of optional is not a constraint – it does not forbid anything, it merely states what may be the case. In contrast, a participation condition of mandatory for a relationship *is* a constraint – specifically, that an entity of the type concerned cannot exist without being involved in an occurrence of the relationship.

Our sentences that capture the meaning of the relationships shown in the E–R diagrams now need to be more complex than those given earlier, in order to account for the participation conditions. Optionality is usually stated in the natural language sentence by explicitly including a zero along with the degree of the relationship, and when a :1 relationship is mandatory it is indicated by stating ‘exactly one’.

The **Manages** relationship with both the degree and participation conditions stated as shown in the University CDM becomes:

- Each Region Manages zero, one or many Students.
- Each Student is Managed by exactly one Region.

The **EnrolledIn** relationship with both degree and participation conditions can be written as:

- Each Student is Enrolled in zero, one or many Enrolments.
- Each Enrolment Enrols exactly one Student.

Occasionally these are reduced even further by using 0, 1 and *m* for zero, one and many, respectively.

- Each Region Manages 0, 1, *m* Students.
- Each Student is Managed by 1 Region.
- Each Student is Enrolled in 0,1, *m* Enrolments.
- Each Enrolment Enrols 1 Student.



EXERCISE 5.18

- (a) What is the full reading, in both directions and showing both the degree and participation properties, of the **TelephoneTutoredBy** and **Tutors** relationships in the University model? You should try expressing this using both natural language sentence forms and the reduced 0, 1, *m* forms for your answer.
- (b) Consider the following additional information about the **Mentors** relationship shown in Figure 5.8. All staff members must be mentored. Experienced staff members can choose to be mentors but do not have to be mentors. Show the participation conditions on the E–R diagram for the **Mentors** relationship.

Now that we’ve completed our look at the entity types and the entity–relationship diagram, we can consider the remaining sections of the CDM.

Additional constraints

As we have just seen, we are able to represent many aspects of the **constraints** in data requirements using the E–R diagram and the entity types with careful choice of identifiers and by specifying the properties of relationships.

EXERCISE 5.19

What constraints can be represented using identifiers and relationship properties?

Not all constraints can be recorded in ways identified in Solution 5.19 – instead we simply write them out as a descriptive entry in the *additional constraints* part of a conceptual data model, as shown in the University CDM. Recording constraints in this informal manner is not as precise as the way constraints can be represented by the choice of identifiers and the properties of relationships. However, it ensures that all

This section of the CDM is named ‘additional constraints’ to remind the reader that these aren’t the only constraints that apply to the model – they are in addition to those shown elsewhere in the model or those accepted as part of the domain of discourse and so unstated.



aspects of the meaning of data given by a statement of requirements are represented somewhere in the CDM.

For example, there is the rule described in the scenario that a student may not enrol for more than 180 points' worth of courses at any one time. How could this rule be represented as a constraint? It would need to express the condition that each **Student** occurrence must not be enrolled on courses having a total of credit points more than 180. Expressing this constraint requires the sum of the credit points (from **Course**) for all **Enrolment** occurrences associated with that **Student** occurrence. Thus it would involve three entity types (**Student**, **Enrolment** and **Course**) and two relationships (**EnrolledIn** and **StudiedBy**). Constraints of this complexity cannot be accommodated using the notations described for the E-R diagram or for the entity types.

As well as such complex constraints there are also simpler constraints on attribute values. For example, the university data requirements state that the value of the credit points for a course can only be 30 points or 60 points. This is represented as a constraint on the values of the attribute **Credit** (of **Course**). Within the University example, this constraint is important because it reflects the policy of the university.

If you look at the additional constraints stated for the University CDM you'll notice that not all the constraints are explicitly stated in the scenario. Some constraints, such as the ordering of the various dates within the University model, are common sense but should be stated to ensure the domain of discourse is properly understood and the data properly protected. Similarly, the constraints that would be obvious in a given domain of discourse are not usually stated, although they could be if you wanted to fully document a situation. In this case we don't include constraints on people's ages, since it is clear from the University domain of discourse that the normal range for people's ages would apply.

The additional constraints in the CDM examples on the laminated cards include both informal and formal wording for the constraints.

EXERCISE 5.20

What would be the potential data integrity problem that might arise if the constraint c.3 in the University CDM was not enforced (This constraint specifies that a **Student's** registration date must be before the **EnrolmentDate** of the same **Student** for any **Enrolment**.)



In keeping with our requirement not to include unnecessary duplication in our data we do not record constraints that can be represented somewhere else in our model – so we don't include a written constraint that says that each **StudentId** value is unique within all the **Student** entity type occurrences as this constraint is already represented by the identifying attributes of the **Student** entity type being underlined.

Assumptions

It is not uncommon when modelling a complex situation to start with a large number of **assumptions** – remember, you are refining the domain of discourse while undertaking the modelling activities so it is not surprising that there is an initial period of uncertainty in the definitions, descriptions and conditions being modelled. However, the developer will produce an interim model based on these assumptions until such time as the stakeholders can clarify the situation. So assumptions will be replaced by definite statements of requirements when the refinement process is completed. That is, these assumptions will be removed as the development work is completed. The University CDM is clearly incomplete as it has an assumption still to be resolved.

Using assumptions in TMAs and Exam questions

As students you do not get the opportunity to refine your model by interviewing or interacting directly with the question setter. If you identify aspects of a scenario or question in a TMA or Exam that you feel are either ambiguous or incomplete you should use the assumptions section of the model to explain how you have interpreted the question. In this way you can inform your tutor of your thinking.



EXERCISE 5.21

Look at the E–R diagram in the University CDM and in particular at the degree of the **Manages** relationship. Now reread the University scenario. State an assumption that might have been made (and resolved) in order for the two documents to be consistent.

Limitations

In any model of a real situation there will be limits under which the model is meaningful, and if you step beyond those limits the model will not be an adequate representation of the real world. If you attempt to use the model, or anything designed using that model, in a situation in which one or more of these limitations is ignored then there will be a mismatch between the model and reality.

Limitations in engineering

A similar list of limitations can usually be found with engineering designs. You may recall the O-ring failure that led to the destruction of the space shuttle Challenger. In a subsequent inquiry it was identified that the working limits for the O-rings had a minimum and maximum temperature – the limitations for the use of the O-rings was that the environment temperature would never go below or above these limits. Accordingly, this engineering solution should only have been used where the temperature would be within the specified temperature range. If you wanted to work outside of these temperature limits then another design would be required (another type of O-ring, for instance). The O-rings used on Challenger had not been designed to be used at the low temperatures that occurred on the night prior to the fated launch, and the O-rings failed.

Entries in the **limitations** part of a conceptual data model record any aspects of the modelling that may not be otherwise explicitly represented, in order to enable the detail of the E–R diagram and the entity types to be understood. In the case of the University CDM, one of the limitations stated is that only current enrolments are recorded (I.2). This is important in understanding the choice of identifiers for **Enrolment** and **Assignment**.

Imagine that we are recording details of past, as well as current, enrolments. It would then be possible for a student to enrol on the same course more than once – in the current year and in some previous year (we will assume that a course is presented only once a year). The chosen identifier of **Enrolment** – (**StudentId**, **CourseCode**) – would then be inadequate, since there would be two **Enrolment** occurrences with the same value for the identifier. To overcome this problem, the entity heading would need to be modified to include a new attribute (say, **EnrolmentYear**) indicating the year of enrolment. This new attribute would then form part of a new identifier for **Enrolment**: (**StudentId**, **CourseCode**, **EnrolmentYear**). The same argument would apply to

Assignment: its identifier would need to be extended to (**StudentId**, **CourseCode**, **AssignmentNumber**, **EnrolmentYear**). Of course, assuming we are not trying to model past enrolments, the identifier (**StudentId**, **CourseCode**) for **Enrolment** is perfectly acceptable. However, we do need to make the reason for our choice of identifier clear, and this is the purpose of the stated explicit limitation. This limitation acts as a reminder that the model is not intended to be used as a model for any university where a student can enrol on the same course more than once in a given year, and never to record past enrolments.

EXERCISE 5.22

Suppose that limitation I.4 in the University CDM did not hold in the university being modelled and that students could sit the same examination several times (not a common occurrence in the UK, but repeated attempts at examinations are possible in some education systems). What changes would be needed to accommodate this requirement in our University CDM?



This is a good time to remind you that the CDM should be interpreted as a complete model, not by considering one component at a time. You should appreciate that relationships can be constrained by additional constraints or their interpretation affected by the limitations placed on the model. Your understanding of the data being modelling will be complete only if you read and incorporate all components of the model.

5.6 The Hospital conceptual data model

Find the laminated cards for the Hospital scenario and Hospital conceptual data models. Read through the requirements as described by the Hospital scenario and attempt to construct a CDM from the supplied scenario. When you've finished, look at the sample model and see how well you identified the important information given in the scenario. You might have thought of some different assumptions to those made when the sample Hospital CDM was written so don't be surprised if you have a slightly different end model.

EXERCISE 5.23

- (a) In the Hospital requirements scenario, what aspects of the data requirements are represented by the participation conditions and degree of the **OccupiedBy** relationship shown in the Hospital CDM? Explain how these aspects are represented.
- (b) Explain the meaning expressed by the identifier for **Treatment**.



Most aspects of the Hospital CDM are directly comparable with the University CDM but there are some aspects of the model that we will examine in more detail. You might have noticed these issues as you read through the Hospital scenario.

One point that you may have noticed as you read through the requirements and the CDM for the Hospital is that there are no $m:n$ relationships. This is not untypical, because $m:n$ relationships are relatively rare in a completed CDM. This does not mean that they are unimportant, because they are extremely useful in developing a CDM and in comprehending a data model, such as that for the Hospital.

This figure does not include participation conditions because they are not relevant to the discussion.

Relationships of degree $m:n$ – revisited

Looking at the statement of data requirements for the Hospital, you may reasonably conclude that an $m:n$ relationship does, in fact, exist. There is the statement ‘Each doctor may provide treatment for several patients and each patient may receive treatment from a number of doctors ...’. This statement describes the association between doctors and patients, which is the treatment of a patient by a doctor, and it suggests that an $m:n$ relationship, named (say) **Treats**, exists between **Doctor** and **Patient**, as depicted in Figure 5.12.



Figure 5.12 $m:n$ relationship **Treats**

Figure 5.13 illustrates this situation by showing some example occurrences of the **Treats** relationship, using some values of the identifier of each entity type to indicate occurrences.

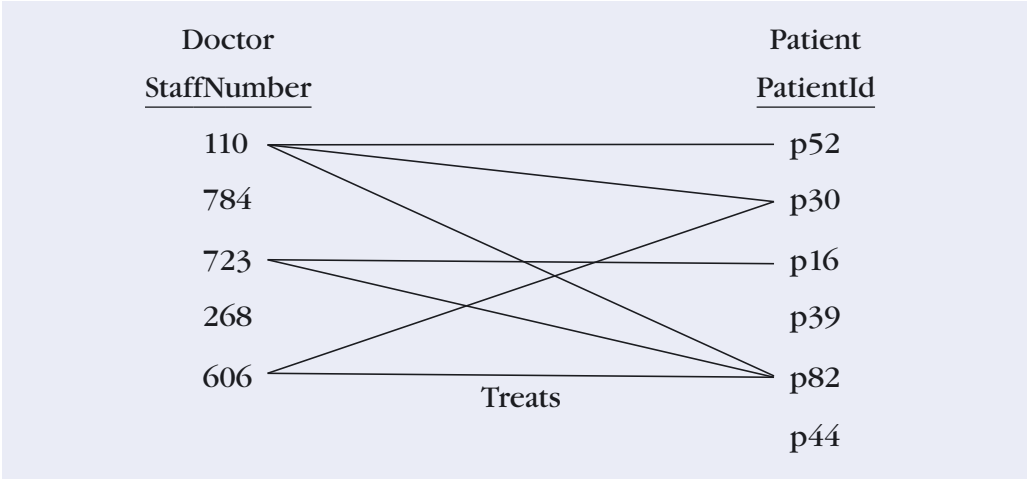


Figure 5.13 Some occurrences of the **Treats** relationship

So why do we not include this $m:n$ relationship **Treats** in the Hospital CDM? The reason is that it would not be a *complete* model of the situation. There is a further statement that ‘A patient may receive more than one treatment from each doctor, for which the *start date* and the *reason* (e.g. a chest infection) for the treatment are recorded’. The start date and reason for a treatment are not data about a doctor (a doctor can prescribe many treatments); likewise, they are not data about a patient (a patient can receive many treatments). Rather, they are data *about* the relationship involving the treatment of a patient by a doctor, which the model has represented by the **Treatment** entity type and its constituent attributes. An alternative way of looking at this is that the value of the attribute **StartDate** is determined by reference to two entity occurrences from different entity types – a specific patient and a specific doctor. So start date is an attribute that relates to the relationship between a doctor and a patient. However, only entity types have attributes; relationships do not. So the relationship **Treats** becomes a special type of entity type, an **intersection entity type** which we could call **Treatment**. The start date attribute can now be shown as an attribute of the **Treatment** intersection entity type. Remember from our definition of the entity type that it is possible for an entity type to describe events as well as physical things – **Treatment** is just such an entity type.

Consider that Figure 5.14 shows the same **Doctor** and **Patient** occurrences as those in Figure 5.13, but now includes example **Treatment** entity occurrences and **Provides** and **Receives** relationship occurrences.

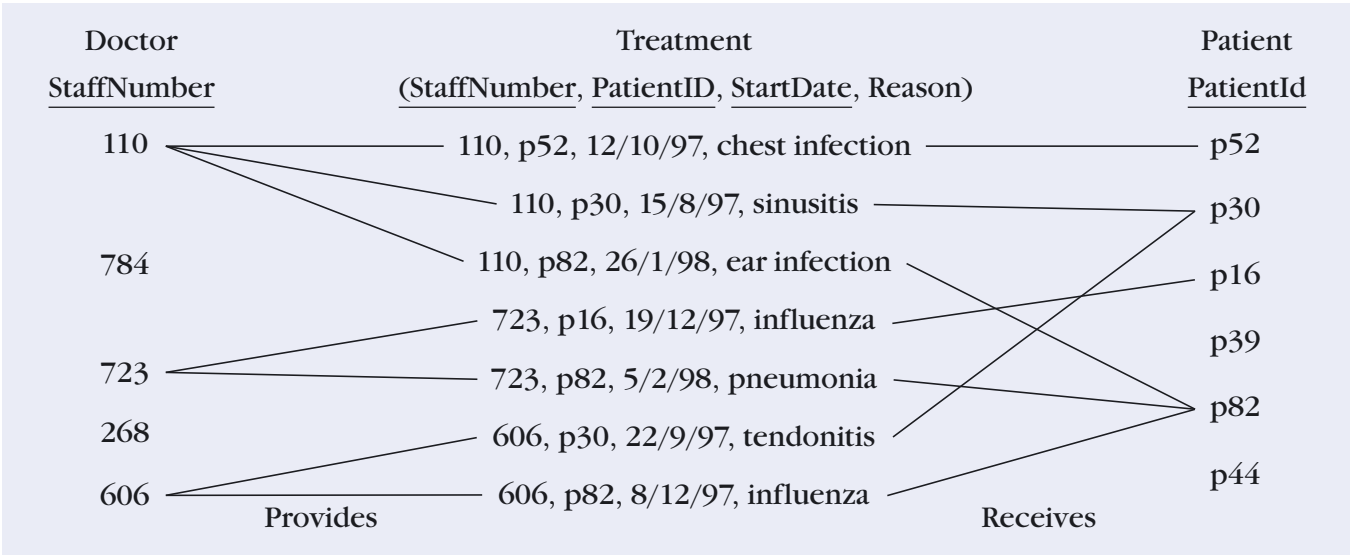


Figure 5.14 Doctor and Patient occurrences from Figure 5.13 with Provides, Treatment and Receives occurrences

You should note that the situation depicted in Figure 5.14 represents the same occurrences between doctors and patients as were shown in Figure 5.13. For example, the **Doctor** occurrence with a **StaffNumber** value of 110 is still associated with three **Patient** occurrences, those with **PatientId** values of p52, p30 and p82. However, this association is now via occurrences of the **Provides** relationship, the **Treatment** entity type and the **Receives** relationship. The **Patient** occurrence with a **PatientId** value of p82 is still associated with three **Doctor** occurrences, those with **StaffNumber** values of 110, 723 and 606. However, this association is now via occurrences of **Receives**, **Treatment** and **Provides**.

You should also note that the **Treatment** intersection entity type is a weak entity type dependent on the **Doctor** and **Patient** entity types. The identifier values for occurrences of **Treatment** depend on the existence of occurrences of **Doctor** and occurrences of **Patient**. This is typical for intersection entity types, since they replace relationships that depend on the existence of occurrences of related entity types.

Thus the data requirements that were originally perceived as the *m:n* relationship **Treats** are better expressed, by an intersection entity type **Treatment** with additional relationships **Provides** and **Receives**. This transformation is sometimes referred to as *resolving an m:n relationship*, and is the reason why *m:n* relationships occur only rarely in a completed CDM – *they exist only when there is no data to be recorded about the relationship*.

There is a general pattern that can be applied when producing an intersection entity type from a relationship representation. In general an *m:n* relationship, when decomposed, becomes a new entity type and a pair of relationships. The new relationships have the degree pattern 1:*m*, *m*:1 around the new intersection entity type. The participation conditions of the :*m* ends are mandatory, and the participation conditions at the :1 ends are the same as the original entity types had in the original relationship. The process of producing the new relationships and entity type is shown diagrammatically in Figure 5.15.

If you can think of sensible names for the new relationships (the domain of discourse might give a meaningful name) then use those names, otherwise invent something that can be used as a descriptive name – such as ANewRelationship and BNewRelationship.

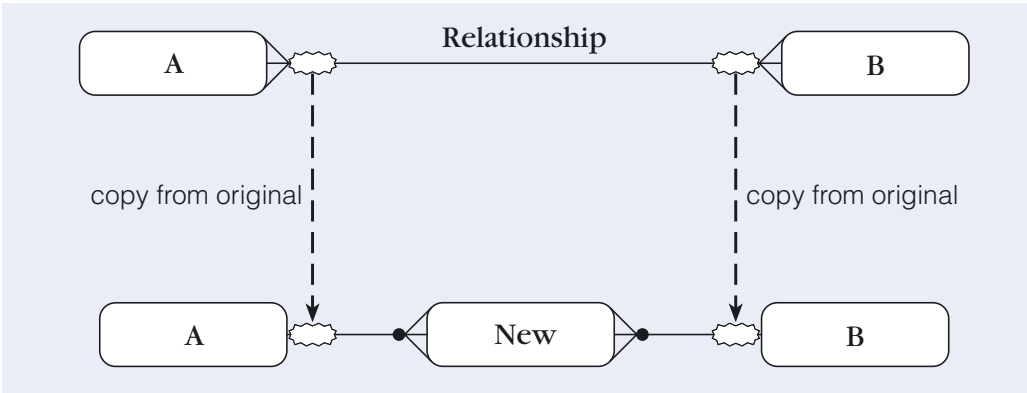


Figure 5.15 The process of resolving an $m:n$ relationship



EXERCISE 5.24

There is another situation in the statement of the Hospital data requirements that suggests an $m:n$ relationship. Describe the situation and explain how it has been resolved in the Hospital CDM.

Further relationship properties

In an entity–relationship conceptual data model, relationships represent many aspects of the meaning of the data. In particular, the properties of relationships – their degree and participation conditions – are a way of expressing various kinds of constraints. There are other properties of relationships that can be expressed in terms of constraints.

For example, the role of doctors in the two relationships **ConsistsOf** and **HeadedBy** varies according to the value of the attribute **Position** – only a consultant may participate in a **HeadedBy** relationship, and only registrars and junior doctors participate in a **ConsistsOf** relationship. These two constraints can be expressed by requiring that **HeadedBy** and **ConsistsOf** are **exclusive relationships** for the entity type **Doctor**. This simply means that an occurrence of **Doctor** may participate in one of these relationships but not in both. If you look back at the University model you will see that constraint c.8 describes the exclusive relationships **ExamIncludes** and **AssignmentIncludes**.

The opposite kind of constraint is when relationships are **inclusive**. This occurs where an entity that participates in one relationship must also participate in the other. We do not have such an example in the Hospital model but you could imagine that it would arise if we extended the model to include outpatients. The **OccupiedBy** relationship would then clearly not be mandatory and we might also make the assumption that the **IsResponsibleFor** relationship applied to all inpatients but not to any outpatients. **OccupiedBy** and **IsResponsibleFor** would then become *inclusive* relationships because a patient then could participate in both of these relationships (an inpatient) or neither of them (an outpatient).

In our modelling we have chosen to represent inclusive and exclusive relationships in the additional constraints section of the conceptual model. Some CDM notations will use additional notation in the E–R diagram to show the same properties.

An outpatient is one who is not allocated to a ward and doesn't have a specific doctor responsible for them.

5.7 Summary

In this section we have seen that, given a statement of data requirements, data analysis is concerned with representing those requirements in a conceptual data model. This CDM is expressed in terms that are independent of its realisation – that is, the model is independent of how the eventual database is going to be implemented. There are different ways in which a conceptual data model may be expressed, and in this course we use an entity–relationship (E–R) data model.

Database design starts with a conceptual data model as representing the given data requirements, and produces a logical schema for a database that can satisfy those requirements. Implementation is concerned with constructing a working system, which requires a storage schema as necessary.

We have learnt that an E–R conceptual data model is built up from three fundamental constructs, entity, attribute and relationship, and can be described by means of an E–R diagram, together with a specification of entity types and any constraints, assumptions and limitations. The distinction between type and occurrence is important for all three constructs. The ability to distinguish one occurrence of an entity from another occurrence of the same type is achieved by means of an identifier, which is one or more attributes whose value is different for each occurrence. Properties of relationships are specified in terms of their degree and participation conditions. These various aspects of an E–R conceptual data model have been illustrated using the University and Hospital examples, and we will continue to use and develop these scenarios as we progress through the course.

LEARNING OUTCOMES

Having completed this section, you should be able to:

- ▶ Describe the role of the conceptual data model in database development.
- ▶ Relate the conceptual data model and the domain of discourse that the model captures.
- ▶ Interpret example entity–relationship representations of conceptual data models and explain their content.
- ▶ Communicate, using basic E–R conceptual data modelling notation, the data requirements underpinning database design.

Block summary

The purpose of this block has been to set the scene for, and give context to, the database theory and practice that is the focus of the M359 course.

In Sections 1, 2 and 3 we have given some background to the fundamentals of data management. These sections covered the historical developments and underlying issues, and the business importance of data management, and introduced the terminology and technologies that will be developed throughout the remainder of the course.

A framework for developing a database system, using the waterfall life cycle from general software engineering development, was introduced in Section 4, and will be considered in further detail when you study *Block 4* of the course.

Section 5 introduced the conceptual data model (CDM). The CDM will be a key communication document throughout the course and it is important that you are able to read and interpret the diagramming notation that we use, and to relate this to the lists of entity types, additional constraints, limitations and assumptions that make up the CDM.

In *Block 1* we have briefly mentioned relational theory – this will be developed further in *Block 2*, while the SQL language will be the focus of *Block 3*. After studying these underpinning topics of relational databases, *Block 4* will look at the implementation and use of SQL in practice. With *Block 5* we will complete the course by looking at two slightly more advanced features of databases: XML and application development.

Solutions to Exercises

SOLUTION 1.1

The issues raised include durability, agreed interpretation, trust, the use of structured data and general purpose encoding, and the need for efficient processing mechanisms.

SOLUTION 1.2

When changes are made to the data, for example when a student moves house, it is possible that the values could become inconsistent if changes are made to data in one of the files but not in the other.

SOLUTION 1.3

- (a) The file-based approach depends on the application code using the physical structure of the data in the file, so if the structure of the data file were to change, the application code would need to be changed accordingly.
- (b) The database approach allows data to be accessed by referring to its logical name and hides the translation of this named data into the physical data structure. This allows the physical storage of data to change and does not require the application programs that use it to be altered.

SOLUTION 1.4

The assumptions we need to make for the context in which this data is going to represent an invalid date of birth could include:

2065 represents the year and not, for example, the time.

The current date is before 2065.

The date is represented in the Gregorian calendar.

Time travel has not been invented – I imagine you missed that one, okay a little superfluous to state this. But if you haven't made this assumption then you can't be 100% certain that someone from the future hasn't started working at the OU so making this a valid date of birth.

SOLUTION 1.5

- (a) You may have identified another problem, but one possible example might be if we had a patient or doctor whose name was Clare. The search system might identify the occurrence of the word Clare anywhere in the same statement as the word Ward as satisfying the request. So, a statement such as 'Patient Marie Clare in Marchmont Ward received treatment by Dr Clare on 13 May 2005' could be retrieved, possibly twice!
- (b) The ward name would appear in a specific location within the structured record so there should be no confusion with the appearance of the word Clare in the patient's name, or doctor's name, or anywhere else in the record.

SOLUTION 1.6

A database is a stored collection of data, not a computer program, so cannot be 'run'. Programs within a database management system, such as the database engine, may run on Windows and UNIX platforms, but the database itself can only be stored in such systems.

SOLUTION 1.7

Any user process that made use of the removed field would need to be changed to remove any reference to that field as that data was no longer present in the database.

This is productive maintenance; if the data is removed then all the requirements for that data in all applications must also have changed.

SOLUTION 1.8

(a)

<i>Identifier</i>	<i>Name</i>	<i>Address</i>	<i>Postcode</i>
<i>R272982388</i>	<i>Martin McGavin</i>	<i>23 Smooth Lane</i>	<i>MK92RG</i>

(b) Yes. Some requests can be satisfied in different ways by different sequences of relational operations – this is a handy formal theoretical feature which can be used to optimise a user request by rewriting their original query in a more efficient form. Although relational theory itself is not concerned with efficiency, it is important when implementing efficient relational systems.

(c) Yes. You can ignore the first selection and just apply the second selection of those students whose first name was 'Martin'. In this example, the first selection is unnecessary if it is only the final result that you require (Martins are a subset of people whose first name begins with M). This is the kind of efficiency gain that a good DBMS is capable of identifying and applying automatically to optimise a query.

SOLUTION 1.9

(a) Data independence, which separates the logical description from the physical description of data, is comparable to encapsulation. Encapsulation, which separates the internal implementation of an object from the external access to that object (i.e. the messages used to access the internals) allows the internal implementation to be changed without affecting the user processes that access the object.

(b) Our definition of the database, given in Subsection 1.4 of this block, is that it is a collection of structured, persistent, shared data. Persistence and sharing are also aims in the development of a persistent programming language, so they have common aims.

SOLUTION 1.10

You could tag the example text with the following (closing marker omitted for brevity):

```
<admissionrecord>, <patientfirstname>, <patientsurname>, <wardname>,
<doctorsurname>, <dateofadmittance>, <reasonforadmittance>.
```

(Your names for the tags or the amount of structuring you used may differ from the examples listed here.)

SOLUTION 1.11

(a) OLAP – a data warehouse of all the known factors related to students who enrol and study on courses. Data mining techniques can then be applied to data to identify correlations between sets of factors and students who re-enrol.

(b) Multimedia database system – you wish to integrate different data in different representations into a single collection of data.

(c) A combination of OLAP systems to allow data warehousing and data mining ('Where am I likely to find these birds during a warm May?' is a data mining type of question), and GIS to put the analysis results onto a map.

SOLUTION 1.12

In Subsection 1.3 we summarised these principles as:

- ▶ The data records must be durable.
- ▶ The data records must be interpretable.
- ▶ The data records must be trustworthy and secure.
- ▶ Agreeing the structure and content of data collections can enable data sharing.
- ▶ Structured data can be processed automatically.
- ▶ Data structure representation must be flexible.
- ▶ Data access must be simple and logical, rather than tied to physical representations.
- ▶ Data collections can be used for many purposes; their processing must be timely to be useful.

SOLUTION 2.1

Because it is possible to over-engineer a solution – but the increased costs can be high. An inappropriate solution to a business need will usually have an impact upon an organisation. Purchasing excessively complex software or equipment, for example, will have an impact on staffing costs. (A Cray supercomputer cluster could be used to manage a local store's stock levels, but so could a desktop PC of a more modest specification.) A potential problem in the information technology environment is overly complex solutions – leading to high costs for equipment, trained personnel, management overheads, and such like.

SOLUTION 2.2

Information management is complex. It can be easy to lose sight of the various objectives that an organisation has in relation to the ownership and management of its data and information. By developing strategies focused on specific roles and objectives it becomes possible to clearly distinguish between the why, what and how in this complex environment. Even smaller organisations can achieve clear advantages by developing an information strategy, and asking questions such as: Why is this needed and what do we want to do with it? What will meet this need and is it adequate? How can we use and manage this technology and is it appropriate?

SOLUTION 2.3

Some ways in which costs might arise: the cost of recreating the data when required, the cost of flawed decision making in the absence of that data, the cost of the time an individual spends looking for that data, the cost of the legal actions if that data was required under legal obligation, loss of customer confidence and subsequent loss of future sales (losing an order, payment, delivery note) and the subsequent need for customer support to retain that customer.

SOLUTION 2.4

- (a) These costs are hidden in the time and effort needed to get members to fill out forms, extracting information from existing records, tracking accounts information against orders and payments made, and entering all of this into the home computer.
- (b) These are usually printing, photocopying and postage costs, and again the time taken to perform these activities. The printer itself could be treated as an execution cost as the printer is needed to process the data into a form that can be communicated to the members; more usually a printer is considered to be a standard item of equipment cost for a home user (but not always for an office-based user). If the club chooses to make a website accessible to members then there will be costs associated with the creation and hosting of the site, upload and download costs, web page maintenance costs and so on.

SOLUTION 2.5

- (a) Accuracy – the data isn't an accurate value for my telephone number.
- (b) Timeliness – the posting date does not allow sufficient time before the payment due date for the customer to receive and act on the bill.
- (c) Accuracy of the domain of discourse – however, this will depend on the context in which it is being used. The lack of flexibility of the appointment system may lead to inaccurate data. If, however, you only ever work in half-hour meetings and only ever work between 9 and 5, then the data that this appointment system represents may be an accurate reflection of your reality.
- (d) Relevance and trusted – although the data is probably accurate, it is not meant for me, so I haven't received data relevant to me. It also means I will have lost trust in the data that the surgery does hold about me – they've confused mine and this other person's records once, might they not do it again?
- (e) Completeness and understandable – there is no indication of which fund this applies to, and the '25' is not a meaningful reference number in relation to distribution periods identified by end dates.
- (f) Understandable – most people would want to know the exact dimensions for a floor rug – 'large' is far too vague in this context. A 10 metre by 10 metre floor rug would be large in my house, but possibly not in a stately home! Large doesn't really mean anything unless there is additional information given somewhere.

SOLUTION 2.6

When the Data Protection Act was introduced, corporations needed to report what personal data was being held throughout their organisation – this required the companies to first find out what data storage and processing systems were in place. Such investigations frequently resulted in the companies finding data islands that had been constructed by individuals to support their specific work activities. Legally, these data islands were part of the corporate data protection responsibilities and consequently needed to be taken back under central control and reporting.

SOLUTION 2.7

A possible prevention of the NASA data loss problem would involve a data retention policy which ensures that data is not archived solely in limited formats – long-term storage and data access issues need to be addressed.

SOLUTION 3.1

- (a) An external schema defines what data is required by a user process, and is represented in the same way as a logical schema, so it should be considered as defining a logical representation of data.
- (b) A logical schema defines constraints, which are not included in an external schema because they are used just by the DBMS and not by user processes.

SOLUTION 3.2

Data independence is achieved because all existing external schemas have been defined without the added column and remain unchanged when it is added, thus providing logical data independence for all user processes that make use of those external schemas.

SOLUTION 3.3

If another user updates data of interest to you, retrieval of the data before and after the update would make you aware that the data had been changed by someone else.

SOLUTION 3.4

Either the funds are transferred (the transaction succeeds), or the funds are not transferred (the transaction fails).

SOLUTION 3.5

A single-user computer may fail, and after any failure it is necessary to recover all completed transactions, but incomplete transactions should have no effect on the database. A correctly implemented transaction support mechanism will ensure correct behaviour, entire transaction success or failure, even in the presence of computer failures.

SOLUTION 3.6

No update relating to an incomplete transaction should be recovered, otherwise there is likely to be some inconsistency. The database contents should be as if the interrupted transaction had not started.

SOLUTION 3.7

- (a) A schema contains the definitions of data in a database, constraints on the data and users' access rights for that data.
- (b) A user process may request data definition, data manipulation, transaction support, constraint definition, access control definition, and restructuring and reorganising functionality.
- (c) The automatic functions of a DBMS are: constraint enforcement, access control, prevention of inconsistencies due to failures of software or hardware, concurrency support and recovery.

SOLUTION 3.8

The first method, with the DBMS performing the calculation, would be quickest. The calculated total will be a smaller amount of data than the list of the prices of all the items on the till receipt – and calculation is faster than communication.

SOLUTION 3.9

If the user process crashed after step 4, the student's details would have been deleted from the database for region 1 but not added to the database in region 4 – in effect they would have disappeared from the university's records.

Transaction management takes place within a single DBMS and would not manage a series of requests distributed over different DBMSs.

SOLUTION 3.10

The allocation of particular ranges of student identifiers assists the checking for uniqueness because it is only necessary to check for uniqueness within each database separately.

A practical problem occurs when a student moves to a different region: the student identifier needs to be changed and so it will be necessary to record previous identifiers for a student. Another problem occurs if the number of students exceeds the allocation of unique identifiers in a region.

SOLUTION 3.11

- (a) Horizontal fragmentation: some rows of the student table would be distributed to each region.
- (b) Vertical fragmentation: the salary column of the staff table would be distributed to the headquarters location.

SOLUTION 3.12

- (a) The three obvious alternatives are:
- 1 Transfer the data from B to A, transfer the data from C to A, and combine the data at A.
 - 2 Transfer the data from B to C, combine the data at C and send the result to A.
 - 3 Transfer the data from C to B, combine the data at B and send the result to A.
- (b) Transfer the data from B to C (1 MB), combine the data at C and send the result to A (0.5 MB) – 1.5 MB is transferred in total.

SOLUTION 3.13

- (a) Since it would take a small amount of time for each update to be distributed across the replication servers, it is possible that whichever updated first (say, System A) would not be seen by the other (System C) at the time the latter (System C) performed its update. System B would then be issued with two instructions to update the local copy of the replicated item – one which reflected the System A update, the other reflecting the System C update. Until Systems A and C resolve the order in which the updates should be applied, System B would not have a consistent value for the data.
- (b) No, a choice of push or pull replication in these systems does not help to resolve the issue. A system being a push or pull system would determine when the conflict in ordering of the updates might occur, but would not help resolve the basic consistency maintenance issue.
- (c) Since local copies of data held at other systems may have been updated during the network failure, the system would need to make its local replicated data consistent with the rest of the system. If the local system had continued processing during the network failure, it would also need to communicate its local changes to any systems holding copies of the data it updated during the network outage.

SOLUTION 3.14

DA: (a), (c), (e).

DBA: (b), (d), (f) – note that a DA might define the requirements for data management but it would be the DBA who identified a DBMS capable of fulfilling those requirements – (g), (h).

SOLUTION 3.15

An application process involves data being retrieved or updated, so it requests the use of the data manipulation function. It may also involve transactions, in which case it will include requests for the use of the transaction support function.

SOLUTION 3.16

The result returned depends on the contents of the database – the actual data stored at the time the SQL statement is evaluated.

SOLUTION 3.17

- (a) The DBMS library will be usable by both database tools provided they can both make the necessary requests to the DBMS for the activities stored in the library. However, the new database tool is not likely to organise and store its descriptions in the same way as Sybase Central, and so you should not expect to use the database tool library with another database tool.
- (b) Each form is designed for use with the data in a specific database, and so it simply will not work with any other database. Thus, users of the database tool would be shown that forms are available but have to know which forms may be used with which database.

- (c) The client, which is the database tool in this case, could be on a different computer system to the database. In keeping with the principle of keeping the data where it is being used, the database tool library should be kept near the client.

SOLUTION 3.18

Using ODBC enables a database tool to interact with any DBMS that has an ODBC interface (i.e. an ODBC driver).

SOLUTION 3.19

An application process is written for use with just one database, which is managed by one DBMS, and so there is no need for an application process to interact with any other DBMS.

SOLUTION 4.1

To include data about the cost of telephone calls, the first method needs a new column added to the table Bill. The second method of recording the data needs a new table, which could be named Telephone, with the same columns as the other tables for this method. The third method does not need any change, rows just being added for this data with an appropriate category for the cost.

SOLUTION 4.2

A bad database would: not satisfy all user requirements; contain inconsistent and invalid data; require excessive effort to change; be slow and be clumsy to use to achieve a desired outcome.

SOLUTION 4.3

- (a) The user requirement for interactive, screen-based use forms part of the system specification, but actual performance can only be identified at implementation time, when the system has been built and is being evaluated. In the development of some systems, attempts at performance prediction can occur during the design stage; however, with database development it is normal to validate such design predictions at the testing stage.
- (b) The missing feature was in the requirements, so during analysis, design or implementation someone has overlooked this requirement. Without further information it is impossible to say when the feature fell out of the development life cycle.
- (c) No user manuals – were they asked for as part of the requirements? If they were then see the answer to (b). If they weren't part of the requirements then such a basic omission should have been identified at the requirements gathering – user documentation should be considered as a standard requirement for any new system.

SOLUTION 4.4

If a user process's requirements were known in advance then the external schema description required for it to access the data would be known and used to define the data requirements for the database development. Our separation of concerns could then permit us to ignore the needs of the user process until the implementation, when the external schema would need to be realised in the implemented database.

SOLUTION 4.5

- (a) This is quite a detailed description of what data will be recorded about a customer so it is likely to be data analysis output – it might be included in the requirements analysis (a lot of information about data items usually is), but it will form part of the more formal conceptual data model.
- (b) This statement is about the requirements for recording a valid order, so it is part of the requirements specification. In effect, it is saying that an order can be recorded

only if the credit card details are valid. If this appeared in the data analysis output we would need to include answers to several more questions: What is a valid credit card? What is an order? Can a customer use more than one credit card per order, or none? And so on.

- (c) This is another requirements specification statement – it says nothing about the data requirements in any detail, consequently there would be a lot more questions that need to be asked about the statement and things it references before the data analyst would be able to document this statement.

SOLUTION 4.6

In a typical database system we will be sharing data among different users and user processes – the alternative to putting the data management in the DBMS with the data is to duplicate the data management requirements in every application that uses the data. The more we can put in the DBMS, the less we have to implement in our user processes.

SOLUTION 4.7

The schema, or system, tables used to store the schema itself must contain data representing all the table definitions and constraints.

SOLUTION 4.8

Some constraints will involve comparisons between data stored in several tables. Since importing data is usually performed one table at a time, these constraints will fail and the import would be prevented. Only when all the bulk data has been loaded should the constraints be checked because, at that time, the data in the database should be consistent.

SOLUTION 4.9

Restructuring will affect the logical schema directly, with changes (see below) to the external and storage schemas and mappings under some circumstances. Reorganisation only affects the storage schema and the mapping to the logical schema.

The mapping between schemas allows certain changes to be hidden from the user as the underlying storage or logical schema is remapped for presentation to the user. Some changes, such as deleting a table from the database, cannot be hidden from those users who access that data. Changes such as the addition of new tables or adding columns would not affect the external schema for existing users but could involve the creation of new or amended external schema for those users who need to access the new data. New tables will also require storage schema adjustments to create storage for the new data. All changes involved in reorganisation, concerned with the representation of the underlying physical storage, can be made by adjusting the mapping to the logical schema which hides these changes from the external user.

SOLUTION 5.1

This is really an open-ended question – but it highlights the fact that documentation alone is insufficient to determine a domain of discourse; discussion with a stakeholder is an essential activity in reaching agreement about the meaning of data represented by documents such as this.

Clearly this is a document about some kind of evening class (it says it is!), but beyond that it doesn't tell us much about the domain of discourse. What do the two dates represent, and why are they different? The headings Size, Students and Accepting don't tell us what the values under the headings represent. We can make some guess as to the headings Class and Room, but does B4 represent a building and room number code, or just a room number? How many classes are there – just Flower Arranging? The document shows it is page 1 of 16; are there 16 classes, or are the 16 pages all for flower arranging classes? The table with headings Name, Student Identifier and

Checked looks fairly sensible as a list of students in the class, but what does Checked mean? And what is the significance, if any, of those students with empty entries for Checked? In fact, is this actually a student list? We might have assumed that the Name and Student Identifier were for the same person, but what if the Name is the tutor name and the student is only identified by their Student Identifier? Who signs the form, and what does the signature represent? Did you notice that the header says the class has 8 students, but that there are 9 names? And so on.

SOLUTION 5.2

- (a) Facts about a student include: their identification number, their name, address and date of registration, the regional office that manages them, their enrolment on courses, their tutor and so on.
- (b) A student cannot enrol on more than 180 points' worth of courses at any one time.

SOLUTION 5.3

- (a) We are told in the requirements scenario that 'Information recorded for each member of staff includes the staff number, name, address, telephone number and one email address. In addition, knowing which region employs the staff member is important.' This gives us the attributes staff number, name, address, telephone number and email address.

The region employing them is *not* an attribute of the **Staff** entity type. We've already said that regional office (region) is an entity type. Later we will see that the association between **Staff** and **Region** is a relationship, not an attribute.

- (b) The identifier of the **Staff** entity type is the staff number. This isn't stated explicitly in the requirements scenario. It's one of those implicit pieces of information that forms our common knowledge – staff numbers can be used to refer to members of staff to identify them explicitly. If we were unsure whether or not this was a sensible assumption to make then we could record it in our model and later seek confirmation from our stakeholders at the University.

SOLUTION 5.4

Assignment is the entity type that is used to show that each occurrence of the assignment entity will record the mark awarded to a student who submits an assignment for a particular course. Note it is not the actual assignment – it's the mark for the assignment that is being recorded.

SOLUTION 5.5

Tutor and **TelephoneTutor** are considered to be instances of entities of type **Staff**. There is no need for the entity types **Tutor** and **TelephoneTutor** because the properties common to these entity types are also common to the **Staff** entity type.

Don't worry if you were initially wondering why we had not listed **Tutor** in our earlier list of entity types. If we had listed it as an entity type and looked at its properties we would soon have noticed that it had the same properties as the **Staff** entity type. Part of the refinement of the model would then be to decide if these represented the same entity type.

SOLUTION 5.6

All entity types shown in an E–R diagram must also be listed in the Entity types section, and vice versa.

SOLUTION 5.7

- (a) **Course.Credit** represents a value for the credit points for a specific course.
- (b) This would be added to the **Enrolment** entity type, as shown below.

Enrolment (StudentId, CourseCode, EnrolmentDate, TutorAllocationDate)

The attribute relates directly to a specific student being tutored on a specific course, and the information about specific students on specific courses is part of the **Enrolment** entity type.

It wouldn't be added to the **Student** entity type as each student could enrol on several courses and so would have a tutor allocation date for each course, in which case there would be several different **TutorAllocationDate** values for each student.

Similarly it couldn't be added to the **Course** entity type as each course can have many students enrolled on it and so many tutors would be allocated for each course.

SOLUTION 5.8

Student.Name could not be guaranteed to be unique within all the possible occurrences of the **Student** entity type. For example, it would be quite possible for several students with the **Name** value **Thompson** or several students with the **Name** value **Chen** to be registered with the University.

SOLUTION 5.9

- (a) An occurrence of **Enrolment** represents a specific student enrolled on a specific course. Each student may enrol on many courses, so there may be many occurrences of **Enrolment** with the same **Enrolment.StudentId**, so **Enrolment.StudentId** does not distinguish occurrences of **Enrolment**. Similarly, each course may have many students enrolled on it, so there may also be many occurrences of **Enrolment** with the same value of **Enrolment.CourseCode**. Thus neither attribute by itself can uniquely identify a single occurrence of this entity type.

- (b) The assertion could be stated as:

There is an enrolment on the course identified by the course code c4 made by a student in the University identified by the student identifier s01 that was made on 12 January 2005. Note that this is an assertion about an enrolment in the University, not about students or courses. However, for this assertion to be valid there should also exist an occurrence of a student with that student identification number and a course with that course code. Otherwise the University would be enrolling students that they didn't record any details about on courses about which they had no information.

SOLUTION 5.10

An occurrence of **Enrolment**, with a value for each of the attributes **StudentId** and **CourseCode**, will only exist when the student identified by the **StudentId** value is enrolled on the course identified by the **CourseCode** value. Conversely, a student is not studying a course if there is no corresponding occurrence of **Enrolment**.

SOLUTION 5.11

SubmissionDate will be an attribute of **Assignment**.

SOLUTION 5.12

Assignment and **Examination** are weak entity types. They both have identifiers that include **StudentId** and **CourseCode** – that together are the identifier for **Enrolment**, and individually are the identifiers for **Student** and **Course**. The entity types **Assignment** and **Examination** also fulfil the common sense reasoning that you wouldn't, for example, receive an **Assignment** submission from a student who wasn't enrolled on a course! That is, the **Assignment** entity instances depend on the existence of **Enrolment** instances.

SOLUTION 5.13

Acting as a telephone tutor – ‘Each member of staff ... may or may not act as a telephone tutor on a course.’

SOLUTION 5.14

This is a 1:*n* relationship from **Enrolment** to **Assignment**:

Each Enrolment Submits many Assignments.

Each Assignment is Submitted by one Enrolment.

SOLUTION 5.15

TelephoneTutoredBy:

Each Course is Telephone Tutored by many Staff members.

Each Staff member Telephone Tutors one Course.

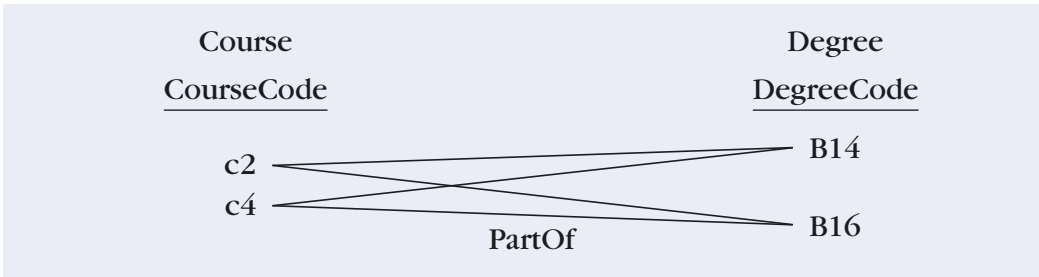
Tutors:

Each Staff member Tutors many Enrolments.

Each Enrolment is Tutored by one Staff member.

SOLUTION 5.16

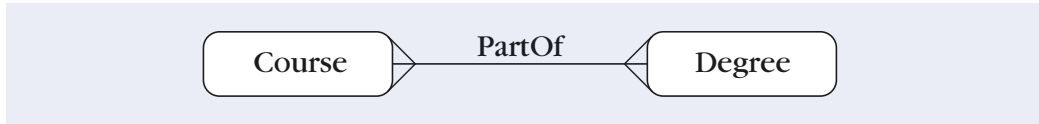
(a)



(b) Each Course is Part of many Degrees.

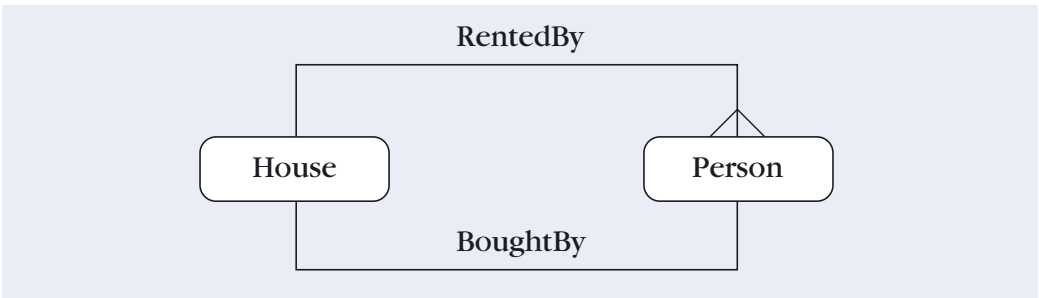
Each Degree is made up of many Courses.

(c)



SOLUTION 5.17

Note that having multiple relationships between different entity types *is* possible. Also, you might have slightly different naming than the solution given here.



SOLUTION 5.18

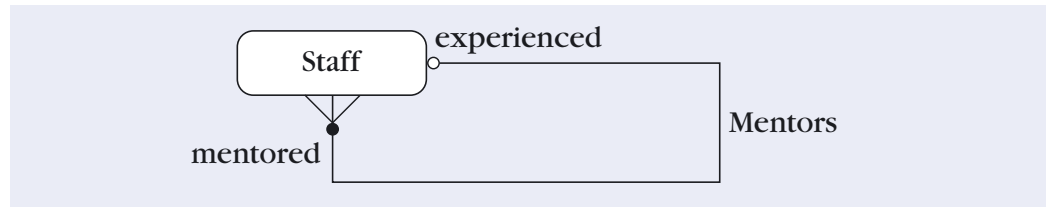
- (a) Each Course is Telephone tutored by one or more Staff members; or Each Course is Telephone tutored by 1, m Staff members.

Each Staff member Telephone tutors zero or one Courses; or Each Staff member Telephone tutors 0, 1 Courses.

Each Staff member Tutors zero, one or more Enrolments; or Each Staff member Tutors 0, 1, m Enrolments.

Each Enrolment is Tutored by zero or one Staff member; or Each Enrolment is Tutored by 0, 1 Staff member.

- (b)



SOLUTION 5.19

The identifier enforces a uniqueness constraint. So, for all instances of the entity type, each value of the attribute chosen as the identifier can appear only once.

Having a degree of :1 at an end of a relationship constrains the occurrences of the entity type at that end to being involved in only one occurrence of the relationship.

Having a mandatory participation condition forces every occurrence of the entity type to be involved in at least one occurrence of the relationship.

SOLUTION 5.20

Someone may mistype a date when entering details of a student enrolment, which might permit a student to be recorded as enrolling before they have registered.

SOLUTION 5.21

There is nothing in the scenario that explicitly states that a region can have no students, or even that a region might have more than one student at a time.

Questions leading to assumptions that might be stated (and subsequently resolved) would be: Does a region manage more than one student at a time? Can there be times when a region has no students to manage (such as at the start of a new year before any students have registered)? The given model has assumed yes to both of these and so the **Manages** relationship is optional for **Region** and : n at the **Student** end.

Notice that this assumption is essentially a common sense interpretation – but if there is any doubt about the interpretation then the assumption should be stated and resolved. (For instance, I know of a cat called Christine that is treated by a vet in Cambridge!)

SOLUTION 5.22

The new scenario changes our definition of an **Examination** entity type and impacts on the chosen identifier and the relationship with **Enrolment**.

An additional attribute of **Examination** will be required so that there is an identifier for each examination that a student on a specific course takes. One resolution for this would be to add an assumption (and eventually replace this with the limitation) that no student takes an examination for a course more than once on any given date – and would add the **ExamDate** attribute to the identifier of **Examination**.

The **Takes** relationship also needs adjustment. The optionality for **Enrolment** and **Examination**, as originally shown, is acceptable but the relationship degree

(currently 1:1) must be changed to 1: m from **Enrolment** to **Examination**. Each Examination is Taken by exactly one Enrolment (of a student on a course); each Enrolment Takes zero, one or many Examinations. (And, of course, the stated limitation I.4 should now be removed!)

SOLUTION 5.23

- (a) The fact that each ward may be empty or may contain one or more patients is expressed by **OccupiedBy** being optional with respect to **Ward** and being : n with respect to **Patient**. The fact that each patient is assigned to a single ward is expressed by **OccupiedBy** being mandatory with respect to **Patient** and being :1 with respect to **Ward**.
- (b) A patient in hospital receives at most one treatment from any one doctor starting on the same date. (Otherwise there would be more than one **Treatment** entity with the same identifier.)

SOLUTION 5.24

'Such a treatment may require several prescriptions ... A drug may appear on different prescriptions.'

This suggests an $m:n$ relationship between **Treatment** and **Drug**. However, there is a requirement to record the quantity and dosage of a drug for a prescription, and so the $m:n$ relationship needs to be resolved and represented by the entity type **Prescription** and the two relationships **Requires** and **IsUsed**.

Index

A

- access control 75
- analytical processing 36
- ANSI/SPARC architecture 26, 69
- application 22
 - custom-built 68
 - developer (AD) 58, 90, 92, 95
 - development tool 93
 - process 17, 92–93, 95, 99
 - program 17, 112
 - server 82
- application process 93
- archive maintenance 63
- artificial intelligence 36
- assertion 123, 125
- assumptions 135
- attribute 107, 120, 122
 - naming 124
 - natural 125
 - value 124, 135
- authorised user 75
- automated data processing 11

B

- back-end system 47
- backup and recovery 51, 76
- base table 72
- business application logic 82

C

- centralised system 82
- change management 76
- client 82
 - multiserver 83, 85
 - server connection 84
 - server system 82
- client–server
 - three-tier 82
 - two-tier 82
- closure property 31
- Codd, E.F. 29
- Code of Practice for Information Security Management 46

- collation sequence 81
- commit 74, 87
- common
 - interface 23
 - language 23
- communication mechanism 82
- completeness 49
- component processes 82
- computer error 63
- computer-aided design (CAD) 40
- conceptual data model 103, 107–108, 110, 116, 119, 125, 134, 140
 - additional constraints 121, 134
 - assumptions 116, 121, 135
 - entity types 121
 - interpretation 119
 - limitations 121, 136–137
- concurrency 75
- conflict resolution 89
- connection management 84
- consistency 19, 23–24, 26, 42, 58, 88
 - between interface and application 14
 - management 89
- consistent data 42
- constraint 20, 24, 61, 69, 75, 85, 108, 110, 116, 122, 125–126, 132–134, 137
 - definition 75
 - management 75
- corporate information strategy (CIS) 44
- cost 46, 54
 - communication 47
 - execution 47
 - origination 47
 - system 47
- costs, delivery 47
- crow's foot 129–130

D

- data 18
 - abuser 92
 - accessibility 42, 48, 59
 - accuracy 19, 49
 - acquisition 47, 50
 - administration 65
 - administrator (DA) 90, 106
 - asset 51, 65
 - bulk loading 112
 - cleaning 47, 50, 56
 - context 20, 41
 - controller 52
 - correctness 24, 42
 - definition 27, 69, 73, 75
 - dependence 14, 69, 71
 - derived 73, 83
 - dictionary 23, 119, 121
 - distribution 110
 - duplication 16, 58, 110
 - external representation 42
 - facts 19
 - file 12–13
 - fit for purpose 49–50
 - import and export 112
 - inaccurate 48
 - inconsistency 58, 121
 - independence 17, 23–24, 28, 32, 69, 71–72
 - index 69, 71
 - information content 20
 - integrity 49, 75, 84, 103
 - interpretation 19, 41, 49, 58, 118
 - logical 24
 - manipulation 73
 - manipulation language (DML) 95
 - meaning 33–34, 41, 49, 58–59, 103, 107
 - mining 36
 - misinterpretation 20
 - physical 24
 - poor quality 50
 - privacy 49
 - processing 33
 - properties 69, 75
 - quality 49–50, 54
 - relevance 49
 - reliability 75
 - replication 110
 - representation 19

- requirements 125
- security 54
- semantics 105
- shared 14, 20–21, 26, 33
- sharing 18, 48, 58
- storage 12, 18
- structure 107
- structured 13, 18, 21, 33–34, 36
- tagging 34
- timeliness 49
- trusted 49
- understandable 49
- user view 26
- valid changes 24
- validity 19, 24
- value 48, 54
- vs information 18
- data analysis 57, 103, 116
 - activity 119
- data island syndrome 64
- data management 6, 18, 39, 44, 63–66
 - legal and ethical issues 51
 - strategy 51, 64, 112
- data mining 36
- data processing
 - environment 12
 - file-based 13, 16, 24
- Data Protection Act 51–53, 60
- data quality 49
- data record
 - durable 18
 - interpretable 18
 - secure 18
 - trustworthy 18
- data requirements 107, 124
- data subject 52–53
- data warehousing 36
- database 21
 - administration 65
 - administrator (DBA) 77, 90
 - approach 16, 22, 24, 26
 - design 108–110, 116
 - designer 22, 59
 - developer (DD) 90, 92, 112
 - development 44, 101, 103, 105, 110
 - engine 21, 23
 - implementer 22, 59
 - life cycle 116, 119
 - post-relational 27, 32
 - pre-relational 27–28
 - properties 102, 110
 - recovery 76
 - reorganisation 114
 - restructuring 76, 114
 - schema 70, 110
 - scope 117
 - semantics 108
 - testing 113
 - tool 111–112
 - transaction 74
 - type 108
 - web-enabled 40
 - XML-enabled 40
- database design
 - logical 113
 - physical 113
- database management, requirements 108
- database management system (DBMS) 16, 22, 24, 68–69, 81, 92, 110
 - advantages 26
 - components 68, 77
 - design decisions 110
 - disadvantages 26
 - functionality 93
 - functions 72
 - logical access 17
 - role 68
 - separation of concerns 25
 - SQL-based 39
 - system schema 77
 - tools 77
- database servers, multiple 83
- DBA 90
- DBMS 16
- decision support 36
- derived table 72
- design
 - decision 84, 113
 - document 104
 - first-cut 109
 - flexed 109
 - second-cut 109
- development life cycle 67, 103, 113
- direct entry database tool 96
- direct user input 96
- disclosure 75
- distributed
 - data 85
 - data processing 82
 - database 40
 - system 82, 85
- distributed database management system (DDBMS) 85
- documentation 59
- domain of discourse 41, 117, 124, 129, 135
- drill-down 36
- drill-up 36
- duplication 108
 - redundant 23
 - unnecessary 135
- durability 41
- E
- efficiency 103, 113
- encapsulation 33, 40
- encoded information 12
- end user 22, 68, 90, 92
 - tool 93
 - training 47
- entity 122
 - occurrence 122
- entity type 120, 122, 125, 131, 134, 138–139
 - intersection 138
 - strong 126, 132
 - weak 126, 132, 139
- entity–relationship (E–R)
 - conceptual data model 127
 - diagram 121, 123, 133–134
 - model 116, 120
- ethical
 - issues 54
 - practices 51
- external schema 70, 110, 112
- F
- file system 21
- flexibility 42, 103, 108, 110

- flexible data structure 18
- formal representation 107
- fragmentation 86
- free text document 34
- free text storage system 21
- Freedom of Information Act 51–53
- front-end system 47
- G
- Geographic Information System (GIS) 37, 40
- H
- hierarchical system 28–29
- historical record keeping system 6
- Hollerith tabulation system 12
- Hospital conceptual data model 137
- host language 95
- HTML 34
- I
- identifier 122, 124–126, 134–135
 - artificial 125
 - multiple-attribute 126
- image data tag 39
- impedance mismatch 32
- inconsistency 23, 42, 61
- information 19
 - Commissioner 52
- information management strategy 45
- information strategy 63
- information system 68, 93
 - strategy 45
- information systems 18
- information technology strategy 45
- information vs data 18
- inheritance 40
- interactive request 35
- interface 13, 82, 98
 - logical 17
 - physical 17
- internal storage 80
- interoperable system 42
- J
- Java virtual machine 89
- K
- knowledge 18, 36
- L
- legacy 28, 40
- legal
 - limitation 42
 - obligation 54
 - requirement 54
- legislation 52
- library 94, 97
- life cycle 114
- limitation 126
- loading/unloading data 112
- location independence 85
- logical
 - data independence 71
 - description 25, 31
 - independence 42
 - interface 17
 - properties of data 69
 - representation 23, 42
 - structure 21, 29
 - view 26, 84
- low-level storage management 26
- M
- machine learning 36
- maintenance 14, 62, 105, 114
 - problems 111
 - productive 14
 - unproductive 14, 16–17, 23
- management database tool 97
- management function 97
- management tool 93
- manual data processing 10
- mapping 17, 24, 26, 32, 69–71, 77
- markup language 34
- meaningful data 19
- menu-based application process 93
- metadata 38
- Microsoft Access 111
- mobile, data management 40
- mobile DBMS 89
- monolithic system 82
- multi-user systems 73
- multimedia 12, 34, 38
 - database 38, 40
 - storage 38
 - system 40
- N
- native interface 99
- network system 28
- O
- object 33
- object-oriented 32
 - database 27, 38, 40
 - database management system (OODBMS) 34, 40
 - technology 33, 40
- object-relational database management system (ORDBMS) 34
- online analytical processing (OLAP) 36
- online transaction processing (OLTP) 35, 38
- Open Database Connectivity (ODBC) 98
- operating system 22, 69, 81
- operational maintenance 77, 114
- optimisation 110
- organisational context 122
- P
- partial data 48
- participation condition
 - mandatory 133
 - optional 133
- pattern identification 38
- performance 114
- persistence 41
- persistent
 - data 21

- programming language 33
- personal information 53
- physical
 - implementation 31, 116
 - independence 42, 71
 - interface 17
 - representation 18, 23, 42, 69
 - storage 17, 25, 28, 77
 - structure 29
 - structure of a file 23
 - structure of data 14
 - view 26
- physical and intellectual property risks 51
- planning new systems 53
- porting and implementation maintenance 77, 114
- principles of good information handling practice 52
- principles of good practice for information management 46, 60
- privacy 52, 75
- process 22
- processing correctness 75
- Professional
 - Association of Computing Machinery (ACM) 52
 - British Computer Society (BCS) 52
 - codes of conduct 52
- program 22
- properties of data 73
- proprietary procedure 98
- punched-card system 13, 16
- Q
- quality management 49
- query 22
 - language 96
 - optimiser 32
 - processing 87
 - tool 93
- R
- recovery process 76
- redundant duplication 26
- refinement process 121, 135
- relation 29–30
- relational
 - database 21, 27, 29, 108
 - database design 44
 - database model 27
 - DBMS 71, 73, 111
 - logical structure 32
 - model 32, 40
 - representation 108, 116
 - specification 108
 - technology 71
- relational database, model 29
- relationship 32, 120, 127, 131, 137, 139
 - degree 127–129, 140
 - degree, many-to-many 130, 137–138
 - degree, one-to-many 128, 131
 - degree, one-to-one 130
 - exclusive 140
 - inclusive 140
 - meaning 133
 - naming 129
 - participation condition 127, 132–133, 140
 - properties 127, 134, 140
 - recursive 131
 - role 131
 - type 127
- relevant data 48
- remote connection 88
- reorganisation 77
- replicated
 - database 40
 - system 88
- replication, server 88
- reporting tool 93
- requirements
 - ambiguity 121
 - analysis 117
 - change 73, 114
 - end users 92
 - establishing 106
 - specification 125
 - statement of 104, 120
- resolving an m:n relationship 139
- restructuring 114
- retrieving and updating data 73
- rollback 74
- S
- schema 69, 73, 76, 105
 - distribution 86
 - global logical 86
 - logical 69, 108, 110–111, 113
 - modification 73
 - storage 69, 80, 110–111, 113–114
- securing data assets 50
- security 86
 - access restriction 24
 - enforcement 110
 - function 51
 - limitation 42
 - management 73, 75
 - risk 51
 - threat 51
- semantic
 - feature 125
 - hierarchy model 33
- semantics 33
- sensitive information 53
- separation of concerns 26, 42, 72
- separation of data storage from application processes 17
- server 82
- single-user system 73
- snapshot 36
- software engineering 103, 110, 114
- software limitations 111
- specialist information system 40
- SQL 30, 111
 - data definition language (DDL) 69, 110
 - database 40
 - embedded 96
 - interface 98
 - statement 111
 - table 79
- stakeholder 104
- standards bodies 41
- strategic decision making 36
- strategic value 48

structured data 34	transactions, all or nothing principle 87	V
structured message document 34	trend analysis 38	validation 59, 113
Structured Query Language (SQL) 30	two-phase commit 87	verification 113
Sybase Central 111	U	view 72
synchronised replication 88	uniqueness 125	W
system	University conceptual data model 117, 120	waterfall model 104
abuser 90	unproductive maintenance 26	web
catalog 23	usability 103, 109	environment 83
specification 104	user 90, 92	server 89
table 77, 79	application 22	wizard 80
system catalog 23	process 22–23, 70, 81–82, 92–93, 98, 111–112	X
T	program 22	XML 34, 40
testing 105, 113	requirements 25, 110, 112, 116	data management 35
thin client 82	tables 77	data representation 40
third-party supplier 94	view 69–70, 93	database 27, 40
three-schema architecture 69, 71, 77, 105, 112	user processes, multiple 24	schema 35
transaction 35, 74, 86		tags 34
management 87		
support 74, 95		